Концептуальное введение в информатику

В.В. Рословцев

Москва
2026

История версий

1	0.9.0	17.01.2026	Подготовлена собранная и частично проработанная версия. Но большая часть глав все еще требует редактуру
2	0.9.1	—	вычитывание и дописывание главы 1

17 января 2026 г.
v. 0.9.0

Реферат

Настоящее пособие является введением в информатику, предназначенным для того, чтобы познакомить читателя с широким кругом понятий, необходимых для успешной работы в разработке программного обеспечения. Концепция изложения предполагает изучение и активное применение различных формальных построений и то, как они могут применяться на практике.

Курс нацелен на сбалансированную подготовку, сочетающую как теоретическую грамотность, так и владение практическими навыками и приемами.

Содержание

Введение 9

1 Программа как инженерный объект 11

1.1 Понятие алгоритма 11

1.2 Композиционные свойства алгоритмов 19

1.3 Программа. Парадигмы программирования . . . 21

1.4 Основы программирования на языке C 25

1.4.1 Массивы 26

1.4.2 Циклы 27

1.4.3 Указатели на функции 29

1.5 Понятие абстрактного типа данных (АТД) 30

1.6 Понятие абстрактного типа данных (АТД) 32

1.7 Программный интерфейс (API) 35

1.7.1 Синтаксический аспект 36

1.7.2 Семантический аспект 37

2 Полиморфизм 38

2.1 Понятие полиморфизма 38

2.2 Полиморфизм в языке C 40

2.2.1 Void. Реализация полиморфизма с помощью void 41

2.2.2 Макросы и полиморфизм 43

3 Классификация структур данных 45

3.1 Классификация структур данных 45

3.2 Линейные структуры данных 47

3.2.1 Массивы 48

3.2.2 Связные списки 53

3.2.3 Очереди и стеки 57

3.2.4 Последовательности 62

3.2.5 Адаптивные последовательности 64

3.3 Нелинейные структуры 66
3.3.1 Ветвящиеся структуры 66
3.3.2 Неупорядоченные структуры 67

4 Обработка ошибок 68
4.1 Возврат специального значения/кода ошибки . . 70
4.2 Использование глобального флага ошибки . . . 70
4.3 Возврат значения вычисления и структуры с описанием ошибки (ErrorInfo) 72
4.4 Объединение результатов вычисления и обработки ошибок (ResultInfo) 74
4.5 Использование исключений в C++ 76

5 Общие операции над коллекциями (map-reduce) 79
5.1 Отображение: операция map 79
5.2 Свертка: операции reduce и fold 80
5.3 Операция flatMap 81
5.4 Фильтрация: операция where 82
5.5 Сцепление и распаковка: операции zip, unzip . . 82
5.6 Агрегирующие функции 83
5.7 Функциональные списки 84
5.8 Лямбда-функции в C++ 85
5.9 Применение операций для матричной и векторной арифметики 86
5.10 Монады 89
5.10.1 Понятие идиомы в программировании . 89
5.10.2 Монада как идиома 89
5.10.3 Реализация монады для обработки ошибок: ResultInfo 90
5.10.4 Расширения DSL 92

6 Синтаксический подход к объектно-ориентированному программированию 95
6.1 Предпосылки 95
6.2 Программирование, ориентированное на объекты 97

6.3 Класс. Поле. Метод 98
6.4 Наследование 100
6.5 Полиморфизм и Подтипизация 101
6.5.1 Ad-Hoc полиморфизм (Перегрузка) . . . 102
6.5.2 Параметрический полиморфизм (Шаблоны) 102
6.5.3 Полиморфизм подтипов (виртуальные методы) 103
6.5.4 Подтипизация 103
6.5.5 Зависимости Типов 104
6.6 Объектная модель и абстрактные типы данных . 105
6.7 Перегрузка операторов 107
6.8 Глобальные константы 108
6.9 Обработка ошибок и еонцепция ResultInfo . . . 109

7 Семантический, или осмысленный, подход к объектно-ориентированному программированию 110
7.1 Предпосылки 111
7.2 Объекты данных и процессы 112
7.3 Объекты как глобальные константы 116
7.4 Отсутствующие значения: альтернатива NULL . 120
7.5 Неоднородные коллекции: дизъюнктивная сумма 120
7.6 Подтипизация и дизъюнктивная сумма 121
7.7 Косвенные связи между объектами 121
7.8 Снова об обработке ошибок: композиция вычислений 121
7.9 Концепты как средство реализации алгебры типов 121

8 Замыкания и семантика передачи параметров 122
8.1 Передача параметров по ссылке и по значению . 122
8.2 Различия между указателем и ссылкой 123
8.3 Семантика копирования 124
8.4 Семантика перемещения 126
8.5 Замыкание 128

8.6 Умные указатели и предпосылки к ссылочным типам 129

9 Ленивые последовательности 134
9.1 АТД LazySequence 134
9.2 Ординалы. Арифметика ординалов 144
9.3 Арифметика бесконечных ленивых последовательностей 148
9.4 Потоки (stream) 162
9.5 Обработка событий и принятий решений 169

10 Древовидные структуры данных 173
10.1 Деревья и абстрактные синтаксические деревья (AST) 173
10.2 Неоднородные структуры и универсальные деревья 174
10.3 Пути в деревьях, обходы и рекурсия 176
10.4 Деревья поиска 177
10.5 Обходы дерева и поиск 178
10.6 Бинарные деревья поиска (BST) 179
10.7 Улучшение работы деревьев поиска 179
10.7.1 Балансировка деревьев 179
10.7.2 Прошитые деревья 180
10.8 Бинарные кучи (Binary Heaps) 180

11 Неупорядоченные структуры 182
11.1 Концепция словаря 182
11.2 Хеш-таблицы 186
11.3 Альтернативные варианты реализации словарей 187
11.4 Построение алфавитного указателя 189
11.4.1 Объектная модель документа: страницы, строки, слова 190
11.4.2 Разделение ответственностей: лексер, пагинатор, сборщик 191

11.4.3 Эффективные Структуры Данных: Словари и Хеш-Таблицы 194
11.5 Кеширование и индексирование данных 195

12 Анализ ситуаций с помощью древовидных структур 199
12.1 B-деревья 199
12.2 Задача о рюкзаке 201

13 Графовые структуры данных 205
13.1 Объектная модель графа 205
13.2 Обходы графа: в глубину и в ширину 214
13.3 Динамическое программирование. Мемоизация частичных результатов 222
13.4 Оснащение метаданными 225
13.5 Продвинутые идеи при обходе графов. Алгоритмы A* и лойда-Уоршелла 228
13.6 Обобщение объектной модели 229
13.6.1 Декомпозиция и абстрагирование 230
13.6.2 Использование подтипизации без наследования 235
13.7 Материализация элементов графа по требованию 236

Введение

Информатика обычно рассматривается как область или дисциплина (в т.ч. учебная), занимающаяся изучнием способов структурирования и обработки информации, включая сбор, хранение, передачу, проебразование, анализ, оценку и ряд других задач. На фундаментальном уровне изучает теорию вычислений, элементы моделей вычислений, теорию алгоритмов и ряд других разделов, позволяющих в совокупности сформировать понятийный базис и понимание логики проектирования и эффективной реализации программных систем. На основе этих базовых идей разрабатываются методологии разработки программных систем, а на прикладном уровне – создаются информационные системы, системы обработки данных, системы искусственного интеллекта.

Настоящее введение в предмет мы назвали концептуальным потому, что акцент, в отличие от большинства существующих курсов и пособий, акцент делается на изучении различных концепций и идиом, как более низкоуровневых, так и более абстрактных, а те или иные алгоритмы и структуры используются скорее как примеры, на которых показывается применение объясняемых идей.

Особенностью подачи материала можно считать то, что мы существенным образом опираемся на математику, но в то же время показываются и способы реализации обсуждаемых концепций, приводятся примеры программного кода, иллюстрируется связь между теорией и практикой.

Например, в разделе **??** обсуждаются продвинутые идеи, касающие отчасти проектирования программных систем, но в большей мере – методологии собственно реализации, использование таких понятий, как объекты данных, объекты, реализующие процессы, и др. Впоследствии неоднократно эти

понятия используются для обсуждения того, какие решения при реализации тех или иных алгоритмов предпочтительнее и почему. В разделе 9 обсуждается достаточно необычная концепция ленивых последовательностей и особенностей их реализации, что позволяет не только на примере обсудить, как может быть реализована спецификация, заданная в виде математического определения, но и показать некоторые приемы, как можно в некоторых случаях обходить аппаратные ограничения.

В последних главах большое внимание уделяется алгоритмам на деревьях и графах. На этих примерах обсуждаются приемы анализа ситуаций и принятия решений на основе идентификации и сопоставления альтернатив, что составляет значительный пласт задач, в информационных, информационно-аналитических, интеллектуальных системах.

1. Программа как инженерный объект

1.1 Понятие алгоритма

Алгоритм является одним из центральных понятий в области программирования и информатики в целом. Неформальное определение алгоритма говорит о том, что это любой корректно определенный набор инструкций, исполнение которого приводит к требуюмому результату. Иногда конкретизируется, что речь идет о вычислительной процедуре, которой на вход подается некоторая величина или набор величин и на выходе получается искомая величина и набор значений. Также говорят, что алгоритм является корректным, если для любых корректных входных данных результатом его работы являются корректные выходные данные. К выработке представления об алгоритме можно подходить различными способами.

Один из таких подходов, с которым мы, в силу исторических причин, обычно знакомимся еще в школе, причем не только в рамках изучения информатики, рассматривает алгоритм как последовательность действий, которые исполнитель должен совершить для достижения заданного результата.

Определение 1.1.1. *Алгоритм – <u>последовательность</u> предписаний для конкретного исполнителя, представленная в понятной этому исполнителю форме и предназначенная для решения конкретной задачи.*

Существенно тут то, что речь идет именно о последовательности действий, а значит, за один условный (или буквальный) такт выполняется одно действие. Рассматривать развитие какого-либо процесса потактово – это в целом довольно универсальный метод; это позволяет выделить отдельные, достаточно мелкие этапы, в рамках которых рассматривается выполнение отдельных, обычно атомарных, действий или

шагов. Само слово «такт» выступает тут как способ абстрагироваться как от конкретной величины его продолжительности (мы просто считаем, что это достаточно короткий интервал времени), так и в принципе в ряде случаев от концепции продолжительности во времени. Например, в случае преобразования алгебраических выражений за один такт можно выполнить одно или несколько элементарных действий над этими выражениями. И при «обычном» определении понятия алгоритм считается, что за один такт выполняется как раз точно одно действие.

Второй значимой составляющей определения понятия алгоритма является явное или неявное указание на то, что упомянутые действия должен совершать некоторый (но в каждом случае конкретный) исполнитель. В широком смысле в роли исполнителя могут выступать объекты различной природы, необязательно компьютеры, и требуется уточнять, и что считается доступными (или допустимыми) для данного исполнителя действиями, и то, в какой форме они могут или должны быть представлены. Например в упомянутом случае с преобразованием выражений исполнителем может быть человек. На субклеточном уровне в организмах наблюдаются процессы, похожие на те, что присходят при работе компьютера: есть некое хранилище, в котором содержатся, по сути, команды, составляющие (под)программы. Есть части и механизмы, считывающие эти команды. Исполнителями являются органеллы или биологические механизмы или системы внутри клетки, или клетка в целом. Изучается и вопрос того, как сделать исполнителем химические соединения – а точнее, как получить т.н. «программируемую материю», способную, упрощенно говоря, в зависимости от получаемых команд менять форму или свойства.

добавить ссылку на статью про программируемую материю

Это наводит на мысль, что разные исполнители, вообще го-

воря, способны выполнять различное количество действий за один такт; в т.ч. два и более действия. А Значит, в приведенном выше определении можно заменить слово «последовательность» на «упорядочение». Напомним, что частичный порядок – это пара $\langle M, \preceq \rangle$, где M – некоторое непустое множество, называемое носителем, а $\preceq$ – бинарное отношение, обладающее свойствами рейфлексивности, транзитивности и антисимметричности. Частичным такие упорядочения называются потому, что в общем случае в них бывают пары элементов, не сравнимые между собой, т.е. для таких элементов x и y неверно как $x \preceq y$, так и $y \preceq x$. В этом случае, если два элемента (действия) сравнимы, то одно из них (например, то, что «меньше») должно быть выполнено раньше другого. Если же они несравнимы, то они могут выполняться независимо друг от друга, в т.ч. и одновременно (параллельно). Такой подход часто используется в рамках т.н. функционального подхода к программированию.

Определение 1.1.2. *Алгоритм – порядок предписаний для конкретного исполнителя, представленная в понятной этому исполнителю форме и предназначенная для решения конкретной задачи.*

Можно пойти еще дальше и отказаться, по крайней мере, на первых порах, и от понятия упорядочения действий.

Определение 1.1.3. Алгоритм – *это система предписаний, задаваемая исполнителю для достижения определенной цели или в определенной ситуации.*

Предписания в этом контексте могут иметь различную форму: команды в императивном стиле, системы равенств или функций и т.д. В целом же алгоритмы характеризуются несколькими свойствами, которыми они должны обладать (см. 1.1).

Понятность – предписания и инструкции должны быть, очевидно, понятными тем, для кого предназначены. Это касается и алгоритмов, хотя тут возникает проблема: в то время как

Свойство	Описание
Понятность	Алгоритм представлен в форме, понятной исполнителю
Завершаемость	Алгоритм выполняется за конечное время
Полнота	Алгоритм обрабатывается максимально широкое пространство входов, при котором не нарушается логическая замкнутость и его целостность (плохо формализуется)
Корректность	Алгоритма дает правильные результаты на всей области определения, явно информирует об ошибках в точках, где результат не определен (например, при делении на 0)

Таблица 1.1 – Свойства алгоритмов

мы, программисты, являемся существвами биологическими и используем для описания идей естественный или формальный язык, основанный на символах (буквах, иероглифах, звуках, жестах), используемые нами в данный компьютеры – это электронно-цифровые устройства, работающие с цифровыми данными и использующими электро-магнитные структуры для представления и передачи информации. Для преодоления этого концептуального разрыва были придуманы языки программирования, которые позволяют сегодня программистам записывать свои идеи в относительной удобной для человека форме, и компиляторы, которые преобразуют записанные программистами конструкции в набор инструкций, которым

может оперировать испольнитель – компьютер, в данном случае.

отметить где-то множественность таких концептуальных разрывов, например, разрыв между спецификацией требований и программной реализацией

Решение этой задачи во многом позволило бы автоматизировать написание тестов и их валидацию, однако на практике это трудно достижимо. Возникает ряд вопросов, определяющих сложность этой задачи:

– Все ли аспекты корректности охватываются существующими тестами, или часть из них остаётся непроверенной?

– По каким критериям следует оценивать полноту и качество тестов?

– Относительно чего именно должна проверяться корректность программы – только ли в рамках заданных требований, или в более широком контексте возможных ситуаций?

Кроме того, возникают и конкретные препятствия. Например:

– концептуальный разрыв между тем, на каком языке описаны требования, и тем, на каком языке написан программный код, который эти требования должен реализовывать;

– в принципе сложность формализации требований, усугбляемая тем, что требования бывает как функциональные (т.е. относящие к тому, что система должна *уметь* делать) и нефункциональные (т.е. *как* система должна это делать – например, треботвания к скорости работы или формату представления результатов).

Как правило, на практике тестирование сводится к проверке соответствия программы заранее сформулированным требованиям, однако такой подход не гарантирует исчерпывающей оценки корректности алгоритма.

Завершаемость означает, что алгоритм должен завершать выполнение за конечное время. Это свойство тесно связано с понятием разрешимости — возможности ответить на вопрос, существует ли алгоритм, который за конечное время выдаёт результат. Например, применительно к задачам алгебры логики это возможность устанвоить, выводима ли та или иная формула или доказуема ли та или иная гипотеза. Применительно к вычислительным опреациям это проверка на то, что тот или иной алгоритм заврешается.

Касательно вопросов разрешимости существует три категории задач:

1. Разрешимые. Для них мы всегда можем сказать, что программа завершится за конечное время или нет. В частности, в логике высказываний можно построить таблицу истинности.

2. Неразрешимые. Для таких задач нельзя точно указать будет ли найден ответ за конечное время.

3. Полуразрешимые. Это системы, в которых положительный ответ гарантированно достигается за конечное время, а отрицательный ответ может уходить в бесконечность.

Наиболее примечательный тип из перечисленных – полуразрешимые, так как можно заметить, что мир устроен похожим «полуразрешимым» образом: когда что-то возможно, то за конечное время это можно найти, в худшем случае, перебором, либо построить, создать и так далее. При этом доказать, что что-то невозможно, обычно очень проблематично. В частности,

любые попытки переборов часто приводят к бесконечным проверкам, которые по сути ничего не значат, так как может быть следующий элемент окажется тем самым искомым.

Следующее свойство алгоритмов – **полнота**. Она характеризиует охват алгоритмом возможных (в адекватных пределах) ситуаций. Например, для сортировки критерием полноты является то, что она можем работать в том числе на пустых наборах данных или на очень больших. Полная универсальность практически недостижима, поэтому важно явно определять границы применимости алгоритма и документировать исключения.

При этом приходится констатировать, что имеющиеся в нашем распоряжении возможности типизации данных и вычислительных блоков не всегда достаточны для точного задания области определения и и области значения переменных. Например, обычно в языках и средах программирования предусмотрены знаковые и беззнаковые целые, а также несколько разновидностей вещественных чисел. Но формального способа представить встроенными в язык типами множества четных, нечетных чисел или чисел из заданного диапазано – как правило, нет. Это и приводит к тому, что при том, что формальная проверка типов пройдна, в те или иные вычислительные блоки могут поступать данные, для которых работа соответствующего алгоритма не определена.

В результате вопрос полноты алгоритма отчасти смыкается с вопросом о его корректности и способности работать с на некорректных входных данных.

Корректность алгоритма сама по себе означает означает, в первую очередь, способность алгоритма выдывать верный (ожидаемый) результат при верных входных данных, но иногда сюда же относят обработку внештатных ситуаций. Вообще-то, источников таких ситуаций может быть два:

(a) поступила конбинация значений входных данных, для которой «поведение» или ход вычислений по какой-либо причине не предусмотрен;

(б) из-за воздействия внешней среды:

(б1) на инфраструктурном уровне, например, при принудительном или аварийном заверешнии потока или процесса, в т.ч. в параллельных вычислениях,

(б2) на уровне обмена данными за счет либо нарушения изоляции, либо испоьзования эффекта глобальных переменных в том или ином виде.

Из всего перечисленного п. (б1) занимает несколько обособленное положение в том плане, что подобные ситуации нежелательны с точки зрения логики работы алгоритмов (поскольку иногда приходится при их реализации учитывать возможность возникновенния подобных ситуаций, а это нарушает некоторые важные принципы разработки программных систем, о которых речь пойдет далее). Но иногда с этим приходится мирится из-за особенностей или ораничений среды исполнения, а иногда подобное поведение является осознанным выбором разработчика обеспечивающей среды, например, с тем чтобы в чем-то упростить реализацию.

Проблематике, связанной с инкапсуляцией, изоляцией и другими важными принципами грамотной разработки, посвящена значительная часть главы 7. О пункте (а) мы частично уже поговрили, а частично продолжим беседу в главе 4. Пока что отметим, что считается полезным применять принцип защитного программирования, при котором каждая структурная единица вычислений (грубо говоря, каждая функция) должна в самом начале работы проверять входные данные на предмет корректности. Например, если по смыслу функция ожидает, что будет передано, скажем, положительное четное число, а формальный тип параметра – знаковое целое, то прежде, чем приступать к каким-либо вычислениям, следует проверить, что поступившее число действительно больше 0 и является четным. Что делать, если это не так, мы довольно подробно обсудим в главе 4.

1.2 Композиционные свойства алгоритмов

Понятие алгоритма напрямую связано с вопросом, что такое вычисление. Существует несколько теоретических моделей, описывающих вычислительные процессы.

Одна из них диктуется так называемыми бесконечными автоматами, в частности, машиной Тьюринга. Машина Тьюринга состоит из бесконечной ленты, на которой символы из заданного алфавита, управляющей головки, которая может считывать или записывать символы на ленту. На каждом шаге работы машина Тьюринга выполняет одну из двух основных операций: перемещает головку на одну ячейку влево или вправо, изменяет символ в текущей ячейке (или оставляет его без изменений). Программа машины Тьюринга представляет собой набор таких инструкций, полностью определяющих, как изменяются состояние машины и содержимое ленты при обработке входных данных.

Есть похожая модель – алгоритмы Маркова. В этой модели вычисления рассматриваются как процесс переписывания строк символов по заданным правилам. В отличие от машины Тьюринга, лента здесь может рассматриваться как потенциально растяжимая: при замене подстрок длина строки может увеличиваться или уменьшаться. То есть в ней можно взять символ и заменить его на несколько других или, наоборот, взять несколько символов и заменить его на один.

Другая концепция – конечный автомат. Он представляет собой абстрактное устройство, которое может находиться в одном из конечного числа состояний и изменять своё состояние в зависимости от входных символов.

Описанные модели — фундаментальны, но не исчерпывают всех аспектов алгоритмических систем. Возникает вопрос: как объединять отдельные алгоритмы в составные системы, формализовать их взаимодействие и проверить корректность комбинаций?

Даже для программ, описанных в терминах машины Тью-

ринга или конечного автомата, задача определения их поведения и результата в общем случае неразрешима. Кроме того, у программ часто есть ”побочный эффект”. Например, есть утилита, которая принимает 2 пути к файлу: один файл, который существует, а второй, который нужно создать. При этом попутно создается какой-нибудь третий файл с журналом, например.

Довольно часто программу можно рассматривать как совокупность взаимосвязанных функций, каждая из которых выполняет определённую вычислительную задачу. Для функций, как правило, известны входные данные и выходное значение, причём типы этих данных могут быть строго определены средствами языка программирования.

Однако этого зачастую недостаточно для полного описания поведения функции. Рассмотрим пример функции сортировки. Формально можно сказать, что она принимает на вход последовательность и возвращает последовательность. Но такое описание не отражает сути вычисления: из него не следует, что возвращаемая последовательность должна содержать те же элементы, что и исходная, и отличаться только порядком их расположения. Не указано также, что в результате сортировки не должны появляться новые элементы и не должны теряться существующие.

Вариантов функции, которая принимает последовательность и возращает последовательность существует множество. Поэтому наличие более специализированного описания частично позволяет формально и, что более важно, автоматически валидировать корректность комбинации тех или иных функций.

Эта проблема приводит к метауровневой цели: получить систему, в которой можно формализовать аспекты комбинирования или композиции конструктивных элементов систем. Под конструктивными элементами системы могут пониматься функции, классы или что-то подобное.

Решение этой задачи имеет большое значение, поскольку

оно открывает путь к созданию более надёжных, проверяемых и предсказуемых программных систем.

1.3 Программа. Парадигмы программирования

Программа – это совокупность исходных текстов, написанных на заданном языке программирования, или совокупность исполняемых файлов, образующих целое. Программа может рассматриваться как в математическом, так и в инженерном смысле.

В математическом смысле программа представляет собой совокупность исходных текстов, структура которых определяется концепцией языка программирования. Например, программа на языке С состоит из функций, которые взаимодействуют друг с другом.

В инженерном смысле программа — это совокупность исполняемых файлов, структура которых определяется концепцией исполнителя (операционной системы или среды исполнения). Например, программа на языке С компилируется в исполняемый файл, который может быть запущен на конкретной операционной системе.

Следует отделять понятие программы от понятий программного модуля, программной системы и программного продукта. Программа, по своей сути – код, исполняемый компьютером. Программный модуль – часть программы, взаимодействующая с другими модулями. Программная система – программа + конфигурационные файлы и базы данных. Программный продукт же это программная система + документация.

Если рассматривать математический аспект более детально, то программа может быть определена как целостная конструкция, подлежащая исполнению. Возникает вопрос: каким образом это целое строится? Основные задачи программной инженерии могут быть сформулированы как процесс формализации

требований пользователя в виде формальной спецификации и их преобразования в программный код. Промежуточной задачей при этом является разработка технологии построения программ.

Программы, как правило, строятся из изолированных компонентов, каждый из которых выполняет определённую функцию. Основополагающим принципом их организации является разделение ответственности: один объект — одна функция. Такие функциональные блоки можно рассматривать как абстрактные типы данных.

Парадигмы программирования — это стили или подходы к написанию программ. Основные парадигмы включают: структурное, процедурное, объектно-ориентированное, императивное и функциональное программирование. Эти парадигмы развивались исторически, отражая переход от низкоуровневых машинных кодов к современным высокоуровневым языкам.

Первые языки программирования были низкоуровневыми: команды в них задавались в виде числовых кодов. Позднее такие коды стали заменяться мнемоническими обозначениями, что привело к появлению ассемблера. Выполнение программ на ассемблере рассматривалось как линейная последовательность команд, однако на практике нередко возникала необходимость в повторении определённых участков кода. Для этого использовалась команда перехода GOTO. Подобный подход привёл к созданию плохо читаемых программ, в которых было сложно проследить алгоритмическую структуру и логику выполнения. Решением этой проблемы стало появление концепции структурного программирования, предложенной как альтернатива хаотичному использованию переходов.

Структурное программирование предполагает отказ от использования оператора GOTO и структурирование алгоритмов с помощью конструкций if-then-else, for, while и т.п.

Впоследствии возникла необходимость повторного использования отдельных частей кода. Без выделения процедур или

функций говорить о программной композиции было невозможно. Так сформировалась новая парадигма — процедурное программирование.

Процедурное программирование основано на использовании процедур (или функций), каждая из которых выполняет конкретную задачу. Ключевыми элементами функции являются её сигнатура, определяющая способ взаимодействия, и тело, описывающее реализацию. Программа в данном случае рассматривается как совокупность функциональных модулей. Функции можно интерпретировать как процессы преобразования входных аргументов в результат.

Дальнейшее развитие идей привело к осознанию необходимости работы с неатомарными структурами данных, то есть такими, которые состоят из нескольких полей. Например, рассмотрим структуру Студента:

```
struct Student {
    string firstName;
    string middleName;
    string lastName;
    DateTime birthDate
}
```

И ей предшествуют несколько функций, которые осуществляют операции с этими данными, например вычисляют возраст. Стремление объединить данные и функции, их обслуживающие, послужило предпосылкой к формированию новой парадигмы программирования.

Дополнительный импульс развитию этой идеи был связан с исследованиями в области искусственного интеллекта, активно развивавшимися в 1960-е годы. В то время нейронные сети рассматривались как попытка моделировать поведение нервной системы человека. В рамках этих исследований сформировалась область представления знаний — попытка формализовать процессы человеческого восприятия и мышления. Одним из ключевых результатов стало понимание того, что человек

воспринимает окружающий мир через объекты и их атрибуты. Что такое атрибут? Атрибут (от лат. *attributio* «приписывание») – признак, приписываемое качество или свойство.

Параллельно развивалась идея стандартизированных моделей данных. В 1969 году Эдгар Кодд предложил реляционную модель, в основе которой лежало представление данных в виде отношений между объектами и их атрибутами. Именно эта концепция во многом повлияла на формирование объектно-ориентированного программирования (ООП).

Объектно-ориентированное программирование (ООП) описывает программы в терминах взаимодействия объектов (в основе объекты, обладающие атрибутами и методами). Состояние объектов и их поведение инкапсулированы в замкнутое целое. В языке С ООП реализуется с помощью структур и функций. ООП включает принципы инкапсуляции, наследования и полиморфизма.

Последующие парадигмы программирования отражают по большому счету взаимно ортогональные подходы к описанию вычислительных процессов.

Императивное программирование основано на последовательном выполнении команд. Ключевые его элементы: присваивание, изменение состояния, использование исключений.

Функциональное программирование основано на чистых функциях без побочных эффектов, где основное внимание уделяется вычислению выражений, а не изменению состояния (т.е. фокус на неизменяемости данных и композиции функций). Побочные эффекты при этом могут проявляться через изменения внутреннего состояния объекта, изменения его окружения или появлением исключений. Характерным для последнего является то, что меняется состояние вычислительной среды, обеспечивающей выполнение.

1.4 Основы программирования на языке C

Программа на языке C состоит из системы функций, записанных в одном или нескольких файлах (с расширением .c и, возможно, .h). Важно отметить, что необходимо присутствие функции main, которая служит точкой входа в программу. Сигнатура этой функции может зависеть от компилятора или целевой среды исполнения.

Арифметика указателей

Указатели — это переменные, которые хранят адрес другой переменной. Они позволяют работать с памятью напрямую. Пример объявления и использования указателей на языке C:

```
int a = 10; // Declare and initialize an integer
    variable

int *p = &a; // Declare a pointer to an integer and
    assign it the address of 'a'
```

Listing 1.1 – Объявление указателей

Арифметика указателей — это набор операций, которые можно выполнять над указателями. Указатели могут быть увеличены или уменьшены, что позволяет перемещаться по массивам или другим структурам данных в памяти:

```
int arr[5] = {1, 2, 3, 4, 5}; // Declare and
    initialize an array of integers

int *ptr = arr; // Declare a pointer to an
    integer and assign it the address of the first
     element of the array

for (int i = 0; i < 5; i++)
{
    printf("%d ", *(ptr + i)); // Use pointer
        arithmetic to access array elements
}
```

Listing 1.2 – Арифметика указателей

1.4.1 Массивы

Массивы — это структуры данных, которые хранят набор элементов одного типа. В языке C массивы могут быть одномерными или многомерными, статическими или динамическими. Элементы массива располагаются в памяти последовательно, один за другим. Доступ к элементам массива осуществляется по индексу, которая начинается с 0.

Одномерные массивы могут быть объявлены в стеке или в куче. В стеке массив объявляется с фиксированным размером, т.е. размер этого массива известен на этапе компиляции. Размер задается при объявлении массива и не может быть изменен напрямую во время выполнения программы.

Если размер массива неизвестен на этапе компиляции, можно использовать динамическое выделение памяти с помощью функций malloc, calloc или realloc. Такие массивы создаются в куче, память для них выделяется с помощью функции malloc/calloc.

```
int arr[5] = {1, 2, 3, 4, 5};  // Array in the
    stack

int* dynamicArr = (int*)malloc(5 * sizeof(int));
     // Array in the heap via malloc func

int *dynamicArr_1 = (int *)calloc(5, sizeof(int))
    ; // Array in the heap via calloc func

dynamicArr = (int*)realloc(dynamicArr, 2 * n *
    sizeof(int)); // Increasing array size in 2
    times

free(dynamicArr); // Free memory containing array
```

Listing 1.3 – Одномерный массив

Функция calloc имеет схожий функционал с malloc, но имеет два отличия: автоматическое заполнение выделенной памяти нулями (malloc не инициализирует память) и принимает на

вход количество элементов и размер элемента (вход malloc - общий размер кучи в байтах). Размер динамического массива можно изменить с помощью функции realloc, в результате чего массив-аргумент поместится в начало массива-результата, т.е. указатели на их первые элементы будут совпадать.

Многомерные массивы представляют собой массив массивов (точнее массив указателей на массивы). Они также могут быть статическими или динамическими. Статические многомерные массивы имеют фиксированный размер, динамические создаются с помощью указателей.

```
int matrix[3][3] = {{1, 2, 3},
                    {4, 5, 6},
                    {7, 8, 9}};

int rows = 3, columns = 3; // Matrix sizes

int* dynamicMatrix = (int*)malloc(rows * columns
   * sizeof(int)); // Allocate memory for a 2D
   array (linearized)
```

Listing 1.4 – Многомерный массив

Массивы более высоких размерностей объявляются аналогичным образом с указанием размерностей по осям и выделением общего размера. При этом, ввиду особенностей устройства памяти, многомерные массивы как бы ”разворачиваются” в одномерную последовательность ячеек памяти. Например, двумерный массив хранится как последовательность ”строчек матрицы”, трёхмерный массив — как последовательность ”слоёв” (под слоем понимается двумерный массив, или матрица), и т.д.

1.4.2 Циклы

Циклы используются для повторения определённых действий. В языке C есть 3 основных типа циклов — while, for и do-while.

Цикл for используется, когда заранее известно количество

итераций. Структурно он выглядит так for(инициализация; условие; обновление) // Тело цикла . Например:

```
for (int i = 5; i < 157; i = i + 7) // Initialize
    cycle
{
    printf("i = %d ", i);
}
```

Listing 1.5 – Цикл for

В таких циклах инициализация выполняется единственный раз перед началом цикла. Здесь задается начальное значение переменной-счетчика, перед каждой итерацией проверяется условие (если условие истинно, выполняется тело цикла). Обновление переменной-счетчика выполняется после каждой итерации.

Цикл while используется в ситуациях, когда количество итераций заранее неизвестно, но есть условие, которое должно выполняться для продолжения цикла. Условие цикла проверяется перед каждой итерацией - если оно истинно, происходит очередное выполнение тела цикла:

```
int i = 1;
while (i < 341) // Start cycle, check condition
{
    printf("i = %d\n", i);
    i = i * 3 - 14;
    if (i < -15)
    {
        break;  // aborting cycle in special
            condition
    }
}
```

Listing 1.6 – Цикл while

Цикл do-while по сути своей тот же цикл while, за исключением того, что проверка условия выполняется после каждой итерации. Это означает, что тело цикла выполняется по край-

ней мере один раз, вне зависимости от значения истинности условия:

```
int i = 14;
do // Start cycle
{
    printf("i = %d\n", i);
    i = i * 4 + 1;
}
while (i < 30); // Check condition
```

Listing 1.7 – Цикл do-while

1.4.3 Указатели на функции

Указатели на функции позволяют хранить адрес функции и вызывать её через этот указатель. Синтаксис объявления указателя на функцию выглядит следующим образом: returnType (*pointerName)(parameterTypes); здесь returnType — тип возвращаемого значения функции, pointerName — имя указателя на функцию, а parameterTypes — типы параметров функции:

```
float (*func_ptr)(int, int, float); // Declare
   function pointer

float tmp_func(int a, int b, float c) // Declare
   some function
{
    return (a + b) / c;
}

func_ptr = tmp_func;  // Initialize function
   pointer

float result = func_ptr(347, 51, -71.28); //
   Using function at address
```

Listing 1.8 – Указатель на функцию

Указатели на функцию применяются для реализации механизма обратных вызовов, т.е. в случае одна функция вызывает другую, переданную ей в качестве аргумента. Указатели на

функции можно хранить в массивах или структурах, что позволяет создавать таблицы функций для вызова в зависимости от условий. Также указатели на функции могут быть переданы в другие функции, возвращены из функций и присвоены другим указателям.

1.5 Понятие абстрактного типа данных (АТД)

Тип данных представляет собой совокупность объектов, которые объединены по признаку общности. Эта общность проявляется во внешних характеристиках объектов: их структуре, поведении, свойствах и взаимодействии с другими объектами. Типы данных могут быть как примитивными (целые/дробные числа, строки), так и сложными и включать в себя множество элементов и операций над ними.

Например, при создании класса `Person` сразу фиксируются различные поля: имя, фамилия, отчество и т. д. Иными словами, структура объекта этого типа определяется ещё на этапе его описания, а каждый экземпляр класса обладает одинаковым набором свойств и методов.

Современные языки программирования допускают широкий диапазон синтаксических конструкций, однако лишь небольшая их часть приводит к корректно работающим программам. Столкновение с этим фактом закономерно порождает необходимость в определённой дисциплине программирования. Одним из инструментов такой дисциплины становится концепция абстрактных типов данных, задающая принципы организации и использования структур данных независимо от их конкретной реализации.

Абстрактный тип данных (или АТД) – это тип данных, который описывается своей внешней спецификацией, при этом детали его реализации скрыты от пользователя. Внешняя спецификация определяет, как объект данного типа взаимодействует с внешней средой, какие операции могут быть выполнены

над ним, и какие результаты ожидаются от этих операций. При этом пользователю не нужно знать, как именно эти операции реализованы внутри. Абстрактные типы данных помогают создать более модульный и удобный код.

Внешняя спецификация, в свою очередь, описывает проявление объекта во внешней среде. Она включает в себя:

- Идентификацию – уникальное имя или способ обращения к объекту.
- Атрибуты – характеристики объекта, которые могут быть доступны или изменены.
- Свойства – особенности поведения объекта, которые определяют его взаимодействие с другими объектами.
- Поведение – набор операций, которые могут быть выполнены над объектом, и их ожидаемые результаты.

Внешняя спецификация может быть представлена в двух формах: математической и программной. Математическая спецификация описывает объект в рамках формальной математической системы. Она использует математические модели, аксиомы и теоремы для описания свойств и поведения объекта. Это позволяет строго определить, как объект должен вести себя в различных ситуациях, и проверить корректность его реализации. Программная спецификация описывает конструкции, связанные с объектом, а также синтаксис и семантику. Она включает в себя правила записи в программном коде, правила использования объекта, а также предусловия и постэффекты, которые касаются входных и выходных данных, внутреннего состояния объекта и состояния его окружения. Программная спецификация позволяет разработчикам точно понимать, как использовать объект в коде, и какие ограничения или требования накладываются на его использование.

Внутренняя спецификация (в отличие от внешней) уже описывает внутреннее устройство и процессы, которые происходят внутри объекта. Она включает в себя детали реализации, такие как структуры данных, алгоритмы и методы, используемые для выполнения операций. Однако эти детали скрыты от пользователя, что позволяет изменять внутреннюю реализацию без необходимости изменения кода, который использует данный АТД.

Рассмотрим пример: пусть задана структура `Person`:

```
struct Person {
    char* firstName;
    char* lastName;
    int BirthDate;
}
```

Если требуется получить полное имя, можно добавить функцию `GetFIO`. Еще одним вариантом может быть создание нового поля, но тогда, так как все поля общедоступные, то при изменении имени полное имя не поменяется. Для решения подобных проблем используется принцип инкапсуляции: для C++, например, можно сделать поля приватными, а доступ к ним осуществлять через геттеры и сеттеры, обеспечивающие контроль за изменениями.

Таким образом, абстрактный тип данных позволяет чётко разделить внешнюю спецификацию и внутреннюю реализацию. Это разделение обеспечивает защиту данных и гибкость при изменении кода.

1.6 Понятие абстрактного типа данных (АТД)

Тип данных представляет собой совокупность объектов, которые объединены по признаку общности. Эта общность проявляется во внешних характеристиках объектов: их структуре, поведении, свойствах и взаимодействии с другими объектами. Типы данных могут быть как примитивными (целые/дробные

числа, строки), так и сложными и включать в себя множество элементов и операций над ними.

Например, при создании класса `Person` сразу фиксируются различные поля: имя, фамилия, отчество и т. д. Иными словами, структура объекта этого типа определяется ещё на этапе его описания, а каждый экземпляр класса обладает одинаковым набором свойств и методов.

Современные языки программирования допускают широкий диапазон синтаксических конструкций, однако лишь небольшая их часть приводит к корректно работающим программам. Столкновение с этим фактом закономерно порождает необходимость в определённой дисциплине программирования. Одним из инструментов такой дисциплины становится концепция абстрактных типов данных, задающая принципы организации и использования структур данных независимо от их конкретной реализации.

Абстрактный тип данных (или АТД) – это тип данных, который описывается своей внешней спецификацией, при этом детали его реализации скрыты от пользователя. Внешняя спецификация определяет, как объект данного типа взаимодействует с внешней средой, какие операции могут быть выполнены над ним, и какие результаты ожидаются от этих операций. При этом пользователю не нужно знать, как именно эти операции реализованы внутри. Абстрактные типы данных помогают создать более модульный и удобный код.

Внешняя спецификация, в свою очередь, описывает проявление объекта во внешней среде. Она включает в себя:

– Идентификацию – уникальное имя или способ обращения к объекту.

– Атрибуты – характеристики объекта, которые могут быть доступны или изменены.

– Свойства – особенности поведения объекта, которые определяют его взаимодействие с другими объектами.

– Поведение – набор операций, которые могут быть выполнены над объектом, и их ожидаемые результаты.

Внешняя спецификация может быть представлена в двух формах: математической и программной. Математическая спецификация описывает объект в рамках формальной математической системы. Она использует математические модели, аксиомы и теоремы для описания свойств и поведения объекта. Это позволяет строго определить, как объект должен вести себя в различных ситуациях, и проверить корректность его реализации. Программная спецификация описывает конструкции, связанные с объектом, а также синтаксис и семантику. Она включает в себя правила записи в программном коде, правила использования объекта, а также предусловия и постэффекты, которые касаются входных и выходных данных, внутреннего состояния объекта и состояния его окружения. Программная спецификация позволяет разработчикам точно понимать, как использовать объект в коде, и какие ограничения или требования накладываются на его использование.

Внутренняя спецификация (в отличие от внешней) уже описывает внутреннее устройство и процессы, которые происходят внутри объекта. Она включает в себя детали реализации, такие как структуры данных, алгоритмы и методы, используемые для выполнения операций. Однако эти детали скрыты от пользователя, что позволяет изменять внутреннюю реализацию без необходимости изменения кода, который использует данный АТД.

Рассмотрим пример: пусть задана структура `Person`:

```
struct Person {
    char* firstName;
    char* lastName;
    int BirthDate;
}
```

Если требуется получить полное имя, можно добавить функцию `GetFIO`. Еще одним вариантом может быть создание но-

вого поля, но тогда, так как все поля общедоступные, то при изменении имени полное имя не поменяется. Для решения подобных проблем используется принцип инкапсуляции: для C++, например, можно сделать поля приватными, а доступ к ним осуществлять через геттеры и сеттеры, обеспечивающие контроль за изменениями.

Таким образом, абстрактный тип данных позволяет чётко разделить внешнюю спецификацию и внутреннюю реализацию. Это разделение обеспечивает защиту данных и гибкость при изменении кода.

1.7 Программный интерфейс (API)

Application Programming Interface (API) – это часть внешней программной спецификации, описывающей возможности взаимодействия с объектом – как правило, в форме описания доступных функций или методов. API определяет, как разработчики могут использовать предоставляемые функции или методы для выполнения определённых задач, не вникая в детали внутренней реализации. Это позволяет абстрагироваться от сложности системы и сосредоточиться на её функциональности. Говоря в какой-то степени проще, API – интерфейс взаимодействия между программными компонентами.

Важно, чтобы интерфейсы были полными и удобными. Под удобством в данном случае понимается наличие как базовых средств для решения типовых задач, так и расширенных — тех, что упрощают выполнение часто встречающихся действий. Расширенные средства, как правило, представляют собой функции, которые можно было бы реализовать последовательностью элементарных операций, но ради удобства и читаемости кода оформляются в виде отдельных, единичных вызовов.

Понятие полноты можно проиллюстрировать на примере языка Ruby, в котором существует три или четыре способа

реализовать замыкание (не вдаваясь сейчас в детали самого понятия). Однако ни один из них не обеспечивает полноценного решения задачи. Иными словами, ни один механизм не является функционально полным. Это свидетельствует о недостатках проектирования языка: во-первых, из-за отсутствия полноты — невозможности выразить всё, что требуется в рамках выбранной модели; во-вторых, из-за недостаточной консистентности — несогласованности между различными средствами языка.

Как правило, грамотно спроектированный API включает четыре базовых типа операций: создание объекта, декомпозицию (здесь это некоторый обощающий термин, включающий атрибуты, если они придусмотрены, и структуру), операции над объектом и его уничтожение.

API включает в себя описание доступных функций или методов, которые можно вызывать для взаимодействия с объектом. Это описание охватывает два ключевых аспекта: синтаксический и семантический.

1.7.1 Синтаксический аспект

Синтаксический аспект API описывает сигнатуру функции или метода. Она определяет, как функция должна быть вызвана с точки зрения синтаксиса языка программирования. Сигнатура включает в себя:

- Имя функции/метода – уникальный идентификатор, по которому можно обратиться к функции.
- Возвращаемый тип – тип данных, который функция возвращает в результате своей работы.
- Порядок и типы аргументов – список параметров, которые необходимо передать функции, их типы и порядок следования.
- Атрибуты аргументов – дополнительные характеристики, такие как признак обязательности аргумента, значе-

ния по умолчанию или модификаторы (например, ключевые слова const, ref и т.д.).

1.7.2 Семантический аспект

Семантический аспект API описывает условия, правила использования функции или метода и ожидаемый результат. В большинстве языков программирования семантика API описывается неформально, например, с помощью комментариев или документации. Этот аспект включает в себя:

- Предусловия
- Постэффекты
- Ожидаемый результат
- Исключения и ошибки

К ключевым свойствам API можно отнести инкапсуляцию, т.е. скрытие деталей реализации от пользователя, а также разделение логики и данных. В объектно-ориентированном программировании реализация API достигается через классы и методы.

2. Полиморфизм

2.1 Понятие полиморфизма

Полиморфизм (от греч. ”многоформность”) означает способность функции, метода или объекта работать с разными типами данных, обеспечивая единый интерфейс. Существует несколько видов полиморфизма, разницу между которыми необходимо четко разделять, особенно в контексте языка C, в котором нет встроенной поддержки ООП. В языке C полиморфизм достигается в основном с помощью указателей, макросов и void* - это требует строгого контроля типов. Основные виды полиморфизма:

– Ad-hoc полиморфизм - перегрузка функций и операторов. Это разновидность полиморфизма, при которой одна и та же операция может быть применена к различным типам данных, но каждый раз требует отдельной реализации. Например, в C++ можно перегружать функции для разных типов аргументов, а в C это можно реализовать через макросы или Generic.

– Параметрический полиморфизм - шаблоны. Этот вид полиморфизма позволяет определять универсальные функции или структуры данных, которые могут работать с разными типами без необходимости их явного перечисления. В C++ это реализуется с помощью шаблонов (template), а в C его можно частично имитировать с помощью void*.

– Полиморфизм подтипов - наследование и виртуальные методы. Это вид полиморфизма, при котором можно использовать объекты производных классов вместо объектов базового класса. Это реализуется через указатели

на базовый класс и использование виртуальных методов (например, в C++).

Практически полиморфизм может понадобиться для расширяемости и масштабируемости кода, при работе с различными типами данных через общий интерфейс. Яркий пример его востребованности:

```
struct VectorN_float // define float-vectors type
{
    int dimension;
    float* coordinates;
};

struct VectorN_int // define integer-vectors type
{
    int dimension;
    int* coordinates;
};

struct VectorN_float* sumN_float(struct
   VectorN_float* v1, struct VectorN_float* v2)
   // define float-vectors sum
{
    struct VectorN_float* v3 = (struct
       VectorN_float*)malloc(sizeof(struct
       VectorN_float));
    v3->dimension = v1->dimension;
    v3->coordinates = (float*)malloc(sizeof(float
       ) * v3->dimension);
    for (int i = 0; i < v3->dimension; i++)
    {
        v3->coordinates[i] = v1->coordinates[i] +
            v2->coordinates[i];
    }
    return v3;
}

struct VectorN_int* sumN_int(struct VectorN_int*
   v1, struct VectorN_int* v2) // define integer-
   vectors sum
```

```
{
    struct VectorN_int* v3 = (struct VectorN_int
       *)malloc(sizeof(struct VectorN_int));
    v3->dimension = v1->dimension;
    v3->coordinates = (int*)malloc(sizeof(int) *
       v3->dimension);
    for (int i = 0; i < v3->dimension; i++)
    {
        v3->coordinates[i] = v1->coordinates[i] +
            v2->coordinates[i];
    }
    return v3;
}
```

Listing 2.1 – Код без использования полиморфизма

В данном коде ключевой недостаток - отсутствие масштабируемости. для каждого типа данных (float и int) созданы отдельные структуры и функции для сложения векторов. Если бы мы хотели добавить поддержку нового типа данных (скажем, double) нам пришлось бы создавать новую структуру (например, VectorN_double), новую функцию sumN_double и т.д. Т.е. каждая функция для сложения векторов по сути своей идентична, за исключением типа данных - таким образом происходит дублирование кода. Это показывает, что без полиморфизма код становится громоздким и трудным в сопровождении.

2.2 Полиморфизм в языке C

Как уже отмечалось ранее в языке C нет встроенной поддержки Объектно-ориентированного программирования. Это означает, что полиморфизм реализуется без классов и объектов, однако возможны способы его имитации.

Ad-hoc полиморфизм

Ad-hoc полиморфизм уже отчасти был показан ранее, но можно представить и более тривиальный пример:

```
int max_int(int a, int b)  // define max_func for
     integer
{
    return (a > b) ? a : b;
}

double max_double(double a, double b) // define
   max_func for double
{
    return (a > b) ? a : b;
}

float max_float(double a, double b) // define
   max_func for float
{
    return (a > b) ? a : b;
}

int main()
{
    printf("Max int: %d\n", max_int(3, 7));
    printf("Max double: %.2f\n", max_double(3.5,
        7.2));
    return 0;
}
```

Listing 2.2 – Ad-hoc полиморфизм

Недостатком этого полиморфизма в данном контексте является явное задание разных имен функций для каждого типа данных. Это связано с тем, что язык C не поддерживает перегрузку функций (т.е. не позволяет объявлять несколько функций с одинаковым именем, в отличие, например, от C++).

2.2.1 Void. Реализация полиморфизма с помощью void

void* в C — это универсальный указатель, который может хранить адрес данных любого типа. Он используется для реализации обобщенного программирования, но требует явного приведения типов при разыменовании. Подчеркнем, что void* дает гибкость, но плата за это есть опасность того, что компи-

лятор не проверяет корректность типов, что может привести к ошибкам во время исполнения программы

В качестве примера безопасного использования void* можно привести функцию qsort(). Это достигается тем, что qsort() принимает указатель на функцию сравнения. Эта функция сортирует массив элементов произвольного типа, используя переданный компаратор для определения порядка элементов. Внутри компаратора программист должен явно приводить void* к нужному типу, что гарантирует отсутствие ошибок на этапе выполнения. Классический пример использования:

```
int compare_ints(void* a, void* b)  // define
   integer comparison-function
{
    return (*(int*)a - *(int*)b);
}

int main()
{
    int arr[] = {42, 7, 93, 15, 86, 23, 58, 1,
       67, 34, 8, 55, 29, 71, 12};  // define
       array
    size_t size = sizeof(arr) / sizeof(arr[0]);
           // calculate array size

    qsort(arr, size, sizeof(int), compare_ints);
          // apply qsort on integer array

    for (size_t i = 0; i < size; i++)
    {
        printf("%d ", arr[i]);        // Out
           sorted array in console
    }
    printf("\n");
    return 0;
}
```

Listing 2.3 – использование void* на примере qsort()

Использование void* в функции сравнения позволяет создать

универсальную функцию для сортировки, которая может работать с любыми типами данных, передаваемыми через указатель. Это уменьшает дублирование кода, так как одна и та же функция может быть использована для массивов разных типов (int, float и др., в том числе собственных с определенной функцией сравнения). Благодаря такому полиморфизму, не нужно писать отдельные функции для каждого типа данных, как в примерах в предыдущих разделах.

2.2.2 Макросы и полиморфизм

Стоит отметить, что в стандарте C11 все же присутствует имитация перегрузки функций с использованием ключевого слова _Generic:

```
int max_int(int a, int b)
{
    return (a > b) ? a : b;
}

double max_double(double a, double b)
{
    return (a > b) ? a : b;
}

float max_float(float a, float b)
{
    return (a > b) ? a : b;
}

#define MAX(x, y) _Generic((x), \
    int: max_int, \
    double: max_double, \
    float: max_float \
)((x), (y))

int main()
{
    printf("Max int: %d\n", MAX(3, 7));
    printf("Max double: %.2f\n", MAX(3.5, 7.2));
```

```
    printf("Max float: %.2f\n", MAX(3.5f, 7.2f));
    return 0;
}
```

Listing 2.4 – Перегрузка функций в стандарте C11 с помощью _Generic

Макросы — это дешевый способ обобщенного программирования, но они не проверяют типы. Таким образом, макросы представляют собой простую, но небезопасную форму полиморфизма. Использование же _Generic представляет собой безопасный способ работы с полиморфизмом в C - в отличие от макросов, _Generic позволяет компилятору проверять соответствие типов на этапе компиляции. Небезопасный пример использования макросов без проверки типов выглядит так:

```
#define MAX(a, b) ((a) > (b) ? (a) : (b))

int main()
{
    printf("Max: %d\n", MAX(3, 7));
    printf("Max: %.2f\n", MAX(3.5f, 7.2f));
    return 0;
}
```

Listing 2.5 – Использование макросов для полиморфизма

Вообще говоря вторая строка может привести к ошибке, поскольку макрос в некоторых версиях компилятора может работать с типом double, при этом он не выполняет автоматическое приведение типов внутри себя (от float к double). Это также может привести к специфичным результатам в случае разных типов входных аргументов (например int и double). Поэтому будет безопаснее использовать пример выше с явным определением типов и использованием ключевого слова _Generic.

3. Классификация структур данных

3.1 Классификация структур данных

Структуры данных представляют собой способы организации и хранения информации в вычислительных системах. Различные структуры данных предназначаются для решения различных задач и обладают собственными характеристиками, такими как сложность и порядок доступа, вставки, удаления или модификации их элементов.

Рисунок 3.1 – Общая классификация структур данных

Важнейшим критерием классификации структур данных является их упорядоченность и форма организации. Также учитываются принципы доступности элементов, изменяемости структуры и особенности реализации в различных языках программирования.

Структуры данных можно условно разделить на:

- Скалярные – представляют собой одиночные значения (числа, символы, булевы значения).
- Нескалярные – состоят из множества элементов и в свою очередь подразделяются на:
  - Линейные – последовательности элементов с фиксированным порядком (списки, массивы, очереди, стеки)
  - Нелинейные – структуры с более нетривиальными связями и порядками между элементами (графы, деревья, кучи и другие)

– Ассоциативные – структуры, основанные на концепции ключ-значение (словарь, хеш-таблица)

Данная классификация носит относительный характер и во многих случаях указанные структуры могут комбинировать несколько принципов организации одновременно. Наглядно она показана на рис.1.

Исторически в теории данных существовало другое деление — на атомарные и неатомарные типы. Понятие атомарности пришло из теории баз данных. Согласно первой нормальной форме, отношение находится в нормальной форме, если значения его атрибутов являются атомарными, то есть неделимыми. В таблицах это означает, что каждая ячейка содержит одно «простое» значение — например, число или строку.

Однако со временем стало ясно, что подобное определение не отражает всей сложности современных данных. Например, объектом значения может быть изображение или строка. Такие объекты нельзя назвать атомарными в строгом смысле, однако в программировании они вполне могут рассматриваться как единичные (скалярные) значения. Здесь является важным, что значением не является другое отношение (так называемые relation-valued attributes).

Например, строки и списки не являются атомарными. Условно типы данных можно разделить на скалярные и нескалярные по принципу наличия или отсутствия вложенных структур. Иными словами, скалярные типы представляют собой неделимые значения, тогда как нескалярные выступают в роли контейнеров — абстракций, применяемых в программировании и обозначающих абстрактный тип данных, предназначенный для хранения элементов, как правило, одного типа, но иногда и нескольких.

Если рассматривать структуры данных на более фундаментальном уровне, то всё их разнообразие можно свести к двум основным принципам организации: однородные линей-

ные структуры (все элементы одного типа) и неоднородные (элементы могут иметь различные типы)

Более того, самой базовой конструкцией, позволяющей породить абсолютно все остальные, является понятие упорядоченной пары. Дальнейшие нюансы определяются тем, что является первым элементом, что вторым, какие принимаются соглашения по типам и другим моментам. Но базовая концепция — это упорядоченная пара.

3.2 Линейные структуры данных

Рисунок 3.2 – Базовые и производные линейные структуры данных

Линейные структуры данных можно разделить на две категории: структуры с последовательным доступом и структуры с произвольным доступом. Принципиальное различие между ними заключается в способе обращения к элементам.

В структурах с последовательным доступом возможность обращения к элементу по индексу возникает только после прохождения всех предыдущих элементов. В противоположность этому, структуры с произвольным доступом позволяют обращаться к любому элементу напрямую по его индексу. Это различие напрямую связано с оценкой алгоритмической сложности операций, которая для разных способов доступа существенно отличается.

Алгоритмическая сложность – это оценка, потребляемых алгоритмом ресурсов в зависимости от размера или размерности задачи/входных данных. Наиболее часто оцениваются два параметра: процессорное время и объём памяти (в основном оперативной, но в ряде случаев и внешней).

Размер и размерность задачи могут совпадать, но не всегда. Например, при вычислении скалярного произведения век-

торов в n-мерном пространстве сложность алгоритма определяется именно размерностью пространства. Однако при разработке матричного калькулятора, поддерживающего операции с матричными полиномами, необходимо учитывать не только размерность самих матриц, но и их количество, а также степень полинома.

При анализе процессорного времени следует учитывать, что измеряемое время выполнения программы может незначительно варьироваться из-за случайных факторов. Тем не менее характер зависимости (логарифмический, степенной и т. п.) остаётся устойчивым, поэтому время удобно выражать в условных единицах — например, в количестве атомарных операций (присваивания, обращения по индексу и т. д.). Ключевое требование — операции должны быть атомарными.

3.2.1 Массивы

Массивы –- это фиксированные структуры данных, хранящие элементы одного типа в непрерывном блоке памяти. Массивы подразделяются, как можно было заметить в предыдущих лекциях, на следующие типы:

– Статические массивы (фиксированной длины)

– Динамические массивы (переменной длины)

– Многомерные массивы

Таблица 3.1 – Внешняя спецификация StaticArray

`struct StaticArray {`
`  int count;`
`  int elementSize;`
`  //bool* check;`
`  void* pData;`
`} ;`
	Структура массива. Содержит в себе количество элементов, емкость буфера, размер элемента в байтах, указатель на буфер.

Создание объекта	

`void*`
`  fillFromArray(`
`  StaticArray* array,`
`  void* data,`
`  int elementSize);`
	Заполнить существующий массив элементами из переданного массива

`StaticArray*`
`  createFromItem(`
`  void* item,`
`  int elementSize);`
	Создание массива из одного элемента

`StaticArray*`
`  createFromArray(`
`  void* data,`
`  int elementSize);`
	Создание нового массива с элементами из существующего массива.

`StaticArray*`
`  copyFromList(`
`  LinkedList* l);`
	Создание нового массива, скопировав все элементы из связного списка

`StaticArray*`
`  copyEmptyFromList(`
`  LinkedList* l);`
	Создании пустого массива с тем же размером элементов, что и в переданном списке.

Декомпозиция	

`void* geti(`
`  StaticArray* l,`
`  int index);`
Может выбрасывать исключения
– IndexOutOfRange (если массив короче, чем заданный индекс)
	Получить элемент по индексу.

`int getLength(` `  StaticArray* l);`	Получить количество элементов в массиве.
Операции	

`void seti(`
`  StaticArray* l,`
`  int i,`
`  void* value);`
Может выбрасывать исключения
– IndexOutOfRange (если индекс отрицательный или больше/равен числу элементов)
	Изменяет значение заданного по индексу элемента в списке.

Уничтожение	
`void delete(` `  StaticArray* list);`	Удаление массива

Таблица 3.2 – Внешняя спецификация DynamicArray

`struct DynamicArray {`
`  int count;`
`  int capacity;`
`  int elementSize;`
`  void* pData;`
`} ;`
	Структура массива. Содержит в себе количество элементов, емкость буфера, размер элемента в байтах, указатель на буфер.

Создание объекта	

`DynamicArray*`
`  createEmpty(`
`  int elementSize);`
	Создание пустого массива

`DynamicArray*`
`  createFromArray(`
`  void* data,`
`  int elementSize);`
	Создание нового массива с элементами из существующего массива.

`DynamicArray*`
`  copyFromList(`
`  LinkedList* l);`
	Создание нового массива, скопировав все элементы из связного списка

`DynamicArray*`
`  copyEmptyFromList(`
`  LinkedList* l);`
	Создании пустого массива с тем же размером элементов, что и в переданном списке.

Декомпозиция	

`void* geti(`
`  DynamicArray* l,`
`  int index);`
Может выбрасывать исключения
– IndexOutOfRange (если массив короче, чем заданный индекс)
	Получить элемент по индексу.

`int getLength(` `  DynamicArray* l);`	Получить количество элементов в массиве.

Операции	

`void seti(`
`  StaticArray* l,`
`  int i,`
`  void* value);`
Может выбрасывать исключения
– IndexOutOfRange (если индекс отрицательный или больше/равен числу элементов)
	Изменяет значение заданного по индексу элемента в списке.

`void resize(`
`  DynamicArray* l,`
`  int newSize);`
	Изменяет длину массива.

Уничтожение	
`void delete(DynamicArray*` `  list);`	Удаление массива

Для динамических массивов, помимо базовых операций получения, добавления и удаления элементов, а также определения длины, ключевой является возможность изменения длины массива.

Рассмотрим операцию изменения длины. Пусть имеется массив длины n, и требуется изменить её.

Если новая длина совпадает с исходной, операция ничего не делает.

Если новая длина меньше исходной, выделяется новая область памяти меньшего размера, и элементы исходного массива копируются в неё до заполнения. Элементы, не вошедшие в новую область, теряются. Возможны разные варианты реализации — например, можно удалять элементы из конца, из начала или из середины массива. Соответственно, поведение функции

должно зависеть от поставленной задачи.

Если новый размер больше прежнего, также выделяется новая область памяти, и элементы исходного массива копируются в неё (обычно — с начала). Оставшиеся ячейки можно заполнить заданным значением по умолчанию.

Поэтому целесообразно разделять операции уменьшения и увеличения длины, чтобы при увеличении можно было передать дефолтное значение для новых элементов.

3.2.2 Связные списки

Связный список –- это такая структура данных, которая состоит из узлов, каждый из которых содержит данные и ссылку на следующий элемент.

Связные списки также подразделяются на:

– Односвязные списки

– Двусвязные списки

– Кольцевые списки

В односвязных списках каждый узел содержит значение и ссылку на следующий элемент. В двусвязных списках дополнительно хранится указатель на предыдущий элемент, что облегчает двусторонний обход.

Каждый узел представляет собой структуру с двумя или тремя полями: одно для хранения данных (обычно указатель), и одно или два для ссылок на соседние узлы. Для представления всего списка создаётся отдельная структура, содержащая указатель на первый элемент (а при необходимости — и на последний), а также, в некоторых реализациях, поле с количеством элементов в списке.

Чтобы получить элемент по индексу в двусвязном списке, необходимо убедиться, что список не пуст, и пройти по узлам до нужного индекса, последовательно переходя от одного узла к следующему. При этом важно проверять, что ссылка на следующий элемент не равна `null` — это означает, что список не

закончился. Если список закончился раньше нужного индекса, следует вызвать исключение, сообщающее о выходе за пределы.

Если структура списка содержит указатель не только на первый, но и на последний элемент, а также число элементов, можно оптимизировать поиск, начиная обход с ближайшего конца (начала или конца списка). В односвязных списках возможен обход только в одном направлении — от начала к концу.

Вставка элементов в двусвязный список возможна как в начало, так и в конец: достаточно изменить указатели соответствующих узлов. В односвязных списках вставка аналогичным образом выполняется в конец.

При вставке в середину списка алгоритм усложняется: необходимо найти место вставки, переопределить указатели у соседних элементов и корректно установить связи у нового узла.

Таблица 3.3 – Внешняя спецификация LinkedList (для односвязного списка)

`struct LinkedList {`
`  int count;`
`  int elementSize;`
`  Node* pFirst;`
`} ;`
	Структура списка. Содержит в себе количество элементов, размер элемента в байтах, указатель на узел с первым элементом.

`struct Node {`
`  void* data;`
`  Node* pNext;`
`} ;`
	Структура узла. Содержит в себе полезные данные и указатель на следующий узел.

Создание объекта	

`LinkedList*`
`  createEmpty(`
`  int elementSize);`
	Создание пустого списка

`LinkedList*`
`  createFromArray(`
`  void* data,`
`  int elementSize);`
	Создание списка с элементами из переданного массива

`LinkedList*`
`  copyFromList(`
`  LinkedList* l);`
	Создание полной копии переданного списка

`LinkedList*`
`  copyEmptyFromList(`
`  LinkedList* l);`
	Создании пустого списка с тем же размером элементов, что и в переданном списке.

Декомпозиция	

`void* geti(`
`  LinkedList* l,`
`  int index);`
Может выбрасывать исключения
– IndexOutOfRange (если список короче, чем заданный индекс)
	Получить элемент по индексу.

`int getLength(` `  LinkedList* l);`	Получить количество элементов в списке.

`void* getFirst(`
`  LinkedList* l);`
Может выбрасывать исключения
– IndexOutOfRange (если список пуст)
	Получить первый элемент в списке (может быть реализовано дополнительно)

`void* getLast(`
`  LinkedList* l);`
Может выбрасывать исключения
– IndexOutOfRange (если список пуст)
	Получить последний элемент в списке (может быть реализовано дополнительно)

Операции	

`void prepend(`
`  LinkedList* l,`
`  void* value);`
	Добавляет элемент в начало списка

`void append(`
`  LinkedList* l,`
`  void* value);`
	Добавляет элемент в конец списка

`void seti(`
`  LinkedList* l,`
`  int i,`
`  void* value);`
Может выбрасывать исключения
– IndexOutOfRange (если индекс отрицательный или больше/равен числу элементов)
	Изменяет значение заданного по индексу элемента в списке.

Уничтожение	
`void delete(` `  LinkedList* list);`	Удаление списка

Кольцевые списки представляют собой разновидность одно- или двусвязных списков, в которых ссылка последнего элемента указывает на первый. Такие структуры были особенно актуальны в ранних системах, где память была крайне

ограничена и требовалась максимальная эффективность. В современных условиях необходимость их использования значительно снизилась, однако они по-прежнему применяются в ряде специализированных задач.

3.2.3 Очереди и стеки

Изначально очереди и стеки были разновидностью связных списков. Их можно считать производными от линейных структур, рассмотренных ранее, так как все операции с этими структурами данных можно реализовать на основе списков или массивов.

Очередь реализует принцип FIFO (First In First Out). У нее имеется указатель на первый и последний элемент, а все операции могут основываться на операциях со связным списком. Если же говорить об очереди как об идеоме, то это абстрактный тип данных, у которого есть первый элемент, последний элемент, при этом добавлять элементы можно только в конец, а извлекать – только из начала. Классические реализации очереди не предполагают возможности узнать, что находится в её середине, а современные, наоборот, обеспечивают некоторую прозрачность и как следствие возможность просмотра элементов внутри очереди.

```
./ch-lec3/pic-queue.png
```

Рисунок 3.3 – Очередь

Таблица 3.4 – Внешняя спецификация Queue

Создание объекта

`Queue* create(` `  int elementSize);`	Создание пустой очереди

`Queue* createFromArray(`
`  void* data,`
`  int elementSize);`
	Создание очереди с элементами из заданного массива

Декомпозиция	

`void* peek(`
`  Queue* s,`
`  int index);`
	Получить элемент очереди по индексу

`int getLength(` `  Queue* s);`	Получить количество элементов в очереди
Операции	

`void enqueue(`
`  Queue* s,`
`  void* value);`
	Положить элемент в конец очереди

`void* dequeue(`
`  Queue* s);`
Может выбрасывать исключения
– IndexOutOfRange (если очередь пуста)
	Извлечь первый элемент

`int isEmpty(` `  Queue* s);`	Проверить пуста ли очередь
Уничтожение	
`void delete(` `  Queue* s);`	Удаление очереди

Стек, в отличие от очереди, реализует прицип LIFO (Last In First Out). Его можно реализовывать на основе связного списка,

у которого есть указатель только на первый элемент. Вставка и извлечение элементов возможны только с одной стороны, при этом данные операции выполняются за константное время. Опционально также можно реализовать возможность просмотра содержимого.

./ch-lec3/pic-stack.png

Рисунок 3.4 – Стек

Таблица 3.5 – Внешняя спецификация Stack

Создание объекта	
`Stack* create(` `  int elementSize);`	Создание пустого стека

`Stack* createFromArray(`
`  void* data,`
`  int elementSize);`
	Создание стека с элементами из заданного массива

Декомпозиция	
`void* top(` `  Stack* s);`	Получить первый элемент стека

`void* peek(`
`  Stack* s,`
`  int index);`
	Получить элемент стека по индексу

`int getLength(` `  Stack* s);`	Получить количество элементов в стеке

Операции	

`void push(`
`  Stack* s,`
`  void* value);`
	Положить элемент в начало стека

`void* pop(`
`  Stack* s);`
Может выбрасывать исключения
– IndexOutOfRange (если стек пуст)
	Извлечь первый элемент

`int isEmpty(` `  Stack* s);`	Проверить пустой ли стек
Уничтожение	
`void delete(` `  Stack* s);`	Удаление стека

Дек является некоторым гибридом между стеком и очередью, так как операции вставки и извлечения доступны с обоих концов.

./ch-lec3/pic-deque.png

Рисунок 3.5 – Дек

Таблица 3.6 – Внешняя спецификация Deque

Создание объекта

`Deque* create(` `  int elementSize);`	Создание пустого дека

`Deque* createFromArray(`
`  void* data,`
`  int elementSize);`
	Создание дека с элементами из заданного массива

Декомпозиция	

`void* peek(`
`  Deque* s,`
`  int index);`
	Получить элемент дека по индексу

`int getLength(` `  Deque* s);`	Получить количество элементов в деке
Операции	

`void prepend(`
`  Deque* s,`
`  void* value);`
	Положить элемент в начало дека

`void append(`
`  Deque* s,`
`  void* value);`
	Положить элемент в конец дека

`void removeFirst(`
`  Deque* s);`
Может выбрасывать исключения
– IndexOutOfRange (если дек пуст)
	Извлечь элемент из начала дека

`void* removeLast(`
`  Deque* s);`
Может выбрасывать исключения
– IndexOutOfRange (если дек пуст)
	Извлечь элемент из конца дека

`int isEmpty(` `  Deque* s);`	Проверить пуст ли дек
Уничтожение	
`void delete(` `  Deque* s);`	Удаление дека

3.2.4 Последовательности

Во всем текущем разделе в первую очередь повествование ведется о коллекциях различных элементов. В математике в принципе есть фундаментальная граница: с одной стороны, для рассмотрения может быть выбран отдельно взятый объект, а с другой стороны – множество объектов. Методологически переход от одного объекта к множеству, осуществляется путем формирования промежуточной консрукции – пары. Именно пара позволяет объединять объекты и служит основой для построения более сложных структур данных.

Пара представляет собой упорядоченную конструкцию, содержащую два элемента. Последовательное объединение объектов в пары позволяет создать линейную структуру — последовательность.

Например, список можно представить в виде системы вложенных упорядоченных пар:

$$[a_1, \ldots, a_n] = [a_1, [a_2, [\ldots, [a_{n-1}, [a_n, []]] \ldots]]]$$

Для завершения структуры используется пустой элемент (пустая пара), обозначающий конец последовательности. Его вве-

дение позволяет унифицировать обработку структуры и избежать специальных случаев, связанных с обработкой последней пары.

Извлечение i-го элемента в таком представлении сводится к последовательному применению второй проекции пары $i-1$ раз и первой — на завершающем шаге.

Таким образом, упорядоченная пара является базовой конструкцией, на основе которой можно построить все линейные коллекции.

Ранее уже рассматривались базовые линейные структуры данных — массивы и списки. Однако на практике нередко возникает необходимость использовать абстракцию, которая объединяет преимущества обоих подходов. В одном случае выгодно хранить элементы в непрерывной области памяти, как в массиве, в другом — разрозненно, как в связном списке. Для решения этой задачи целесообразно ввести абстрактный тип данных **Последовательность**.

Последовательность представляет собой линейную совокупность элементов, у которой есть начало и конец, к последовательности можно добавлять элементы с обеих сторон, а также есть возможность чтения и доступа к элементам в произвольной позиции.

Последовательность можно реализовать как на базе связных списков, так и на базе массивов. При этом каждая из реализаций будет иметь свои преимущества. Поэтому целесообразно создать два конкретных наследника: `ArraySequence` на базе динамического массива и `ListSequence` на базе связного списка.

В программировании существует два основных подхода, как можно реализовать функционал, который похож на уже существующий, – это наследование и инкапсуляция. Наследование подразумевает, что дочерний тип является подтипом родительского, то есть обладает всеми его свойствами и может использоваться везде, где ожидается родительский тип.

Например, тип «Студент» является подтипом типа «Человек». В случае же реализации различных вариантов последовательностей этот подход не всегда удобен, поскольку речь идёт не о разновидностях, а о разных способах внутренней реализации одной и той же абстракции.

Поэтому предпочтительным является подход на основе инкапсуляции. В этом случае функциональность конкретной структуры делегируется внутреннему полю нужного типа.

Например, `ArraySequence` содержит приватное поле типа `DynamicArray`, которому делегируется работа с данными. Аналогично, `ListSequence` содержит поле типа `LinkedList`.

Такой подход позволяет работать с последовательностями, не вдаваясь в детали реализации, и при этом сохраняет возможность выбора наиболее эффективной структуры для конкретной задачи. Например, при реализации алгоритма сортировки можно использовать параметр типа `Sequence`, что позволит быстро сравнить производительность с массиовм и связным списком и определить оптимальный.

Благодаря гибкости абстракции `Sequence` на её основе можно построить широкий набор структур данных: стек, очередь, дек и другие линейные структуры.

3.2.5 Адаптивные последовательности

Адаптиные последовательности `AdaptiveSequence` номинально является разновидностью `ArraySequence`. Как и любая последовательность, она поддерживает операции добавления элементов в начало и в конец. Предполагается, что эти операции выполняются за константное время. Однако на практике константной является только операции добавления элементов в конец последовательности, причем при условии автоматического управления буфером и его увеличения не по одному элементу, а сразу кратно (как правило, в два раза).

Типовая реализация `ArraySequence` использует динамически расширяемый буфер. Изначально резервируется

небольшой объём памяти (иногда совсем маленькй, например, 4, 8 или 10 элементов). Когда буфер заполняется, выполняется реаллокация, при котором размер буфера удваивается. Такой подход значительно снижает количество реаллокаций при большом объёме вставок. Например, если исходный размер буфера равен 4, и требуется разместить 1024 элемента, то произойдёт всего около 6–7 реаллокаций. Общее число реаллокаций растёт медленно, что позволяет добиться практически константного времени вставки в конец. Операция удаления также выполняется эффективно, поскольку сводится к тому, что длина последовательности уменьшается на 1.

Из выше сказанного возникает идея, что можно ввести индекс на первый элемент, что позволит выполнять операции с левого конца также за константное время. То есть возникает структура, имеющая буфер и 2 индекса, обозначающие место, где находится начало и конец полезных данных. Способ расположения элементов внутри буфера называется выравниванием.

Существует 3 основных способа выравнивания:

- Элементы прижимаются к правому краю (тогда операции будут выполняться за константное время с правого края)
- Элементы прижимаются к левому краю (здесь константные операции уже соответственно с левого края)
- Элементы выравниваются по середине (с обоих концов константная асимптотика)

Центральное выравнивание кажется оптимальным, поскольку позволяет эффективно выполнять операции вставки с обеих сторон. Однако при существенном перекосе в сторону вставок с одного конца оно приводит к более частым реаллокациям. Например, если в буфере хранится n элементов полезной информации, а длина буфера при этом $2n$, то и левее, и правее полезных данных есть по $\frac{n}{2}$ пустых ячеек. Если количество вставок в правый конец значительно больше, чем

количество вставок в левый конец, то свободное место справа заканчивается быстро, а значит реаллокации будут происходит значительно чаще. Отсюда можно сделать вывод, что выравнивание по середине является хорошим подходом, но только при оптимистичном предположении, что количество вставок с обоих концов примерно одинаковое. Если это предположение неверно, то асимптотика станет хуже.

Для решения этой проблемы применяется адаптивное выравнивание. Идея заключается в том, чтобы отслеживать соотношение вставок в левый и правый конец и при каждой реаллокации выбирать новое оптимальное расположение данных в буфере. Существуют две стратегии: первая – запоминать соотношение вставок за последнюю реаллокацию, а вторая – использовать соотношение вставок за все время жизни объекта.

Выбор между этими стратегиями зависит от характера процессов, происходящих с последовательностью. Если распределение вставок меняется медленно и плавно (процесс стационарный), имеет смысл использовать только последние значения. Если же наблюдаются частые и резкие изменения, то предпочтительнее учитывать всю историю вставок (сглаживая скачки).

3.3 Нелинейные структуры

3.3.1 Ветвящиеся структуры

К ветвящимся структурам данных относятся структуры, элементы которых не образуют линейной последовательности и не подчиняются явно заданному порядку. Для подобных структур можно ввести некоторый способ упорядочивания, однако он, как правило, носит искусственный характер и не отражает их природы.

Наиболее известными представителями данного класса являются графы и их частный случай — деревья.

Поскольку ветвящиеся структуры данных представляют

особый интерес, их свойства и принципы организации будут подробно рассмотрены в последующих главах.

3.3.2 Неупорядоченные структуры

Более «радикальный» случай – это когда упорядочения нет вовсе. Если в линейные структуры предполагают наличие полного (total), или линейного, порядка, а древовидные и (некоторые) графовые структуры – порождают частичный порядок, есть примеры, когда все элементы попарно не сравнимы, или необходимо обстрагироваться от их размещения в памяти.

Типичный пример – множества, а также ассоциативные структуры. Последние предполагают организацию данных, при которой рассматриваются пары ключ-значение, причем на основании ключа за константное (в том смысле, что независящее от количества элементов) время вычисляется положение второго элемента каждой такой пары. Для используются т.н. хеш-функции, которые позволяют получить для каждого объекта некоторое целочисленное (иногда натуральное) значение, которое затем по какому-либо несложному правилу сопоставляется уже с фактическим расположением искомого объекта.

Такие конструкции на программном уровне называются ассоциативными массивами или словарями, а на аппаратном – ассоциативной памятью.

4. Обработка ошибок

Изучаемый нами «вычислительный» подход к программированию предполагает, что программа строится композиционным образом из блоков, у которых есть вход и выход: мы подаем исходные данные и ожидаем получить результат. Т.е. эти блоки проявляют свойства функций, однако, в отличие от обычного рассмотрения функций, например, в математическом анализе, где область определения и область значения более или менее очевидны, мы акцентриуемся на том, как от входных данных «перейти» к выходным, а точнее, как выходные объекты могут быть сконструированы. При этом границы областей определения и значения функций оказываются не (точно) определенными – в основгном, из-за ограничений системы типов в большинстве языков программирования.

Теоретически, алгоритм (реализующие его вычислительные элементы) должен быть тотальным (т.е. определён и корректно работать для всех входных данных, которые попадают в то, что принимается как область его определения). Это означает, что алгоритм не должен приводить к неопределённым значениям, проявлять неопределенное поведение, приводить к ошибкам на каких-либо входных значениях, считающихся корректными для него. На практике с этим возникат некоторые трудности, в частности потому, что возможностей базовой системы типов обычно недостаточно, чтобы тонко очертить область определения (и область значений), «отфильтровать» из нее нежелательные элементы. Скажем, алгоритм определен (по какому-либо из своих аргументов) на множестве целых чисел $\mathbb{Z}$, за исключением 0; формальным тиипом этого аргумента с логической точки зрения является, очевидно $\mathbb{Z} \setminus \{0\}$, а на «физическом» уровне, т.е. при записи (представлении) этого алгоритма на языке программирования будет в

большинстве случаев какой-либо вариант знакового целого, например, `int`; этот же «физический» тип является способом представления (по крайней мере, одним из) логического типа $\mathbb{Z}$. В результате приходится искусственно расширять область определения алгоритмов и затем каким-то образом обрабатывать «лишние» попадающие значения. В теории возможно построить язык программирования, в котором определение таких «точных» типов возможно, но на практике связано с определенными трудностями. В подавляющем большинстве существующих языков приходится довольствоваться «более грубыми» средствами. C++ в этом отношении составляет некоторое исключение благодаря введенному в него способу задавать ограничения на типах с помощью концептов, но об этом будет отдельный разговор позже.

добавить ссылок на онтологии и концептуальное программирование

Стандартным средством, применяемым в различных языках программирования, чтобы обойти описанные проблемы, являются такие механизмы как исключения, которые замещают собой гарантию тотальности алгоритма. С точки зрения теории вычислений, это порождает ключевую проблему: как следует интерпретировать значение выражения, если его подвыражение не поддается вычислению? Каким образом организовать обработку ошибок так, чтобы вычисления оставались непрерывными и композиционными, а программы – корректными, полными и завершаемыми? Существуют формальные ответы на эти вопросы, изучаемые в рамках теории вычислений, пока обсудим круг идей на относительно неформальном уровне.

Существуют разные подходы, от передачи специальных кодов ошибок до использования структурированных типов данных, инкапсулирующих как успешный результат, так и сведения об ошибке. Рассмотрим основные варианты решения этой проблемы с практической точки зрения.

4.1 Возврат специального значения/кода ошибки

Наиболее простой в реализации подход: при возникновении ошибки возвращать некоторое значение, которое, как достоверно известно, не может быть получено в качестве результата вычисления. Примером таких значений может быть физически невозможная величина -300 К для температуры, NULL для указателя и т.п.

Пример использования этого метода:

```
struct Vector* div_scalar(struct Vector* v, void*
    scalar) {
     struct Vector* v2 = ...;
     if (!is_zero(scalar))
         for (int i = 0; i < length(v); ++i)
             set_i(i, v2, div(get_i(i, v), scalar));
     else
         return NULL;
     return v2;
}
```

Listing 4.1 – Локализация ошибки возвратом специального значения

Сложности, связанные с использованием данного подхода:

1. Такое «запрещённое» значение далеко не всегда можно найти;
2. Затруднена диагностика ошибок и их дальнейшая обработка, особенно в случаях, когда возвращается одно и то же значение при любых нештатных вариантах работы;
3. В каждом вычислительном блоке необходима явная предварительная проверка результата на ошибочность.

4.2 Использование глобального флага ошибки

Можно использовать глобальную переменную (в том числе, переменную, передаваемую по указателю), которая хранит

в себе информацию об ошибке. Глобальная переменная не обязательно должна быть общедоступной; можно реализовать её через функцию, возвращающую код результата выполнения последней операции. Каждая реализованная операция меняет значение глобальной переменной: код ошибки, сообщение об ошибке и т.д. Благодаря коду, происхождение ошибок явно различимо.

Пусть переменная ERROR_DIV_BY_ZERO типа int задана и имеет некоторое оговоренное значение, а struct Vector также определена:

```
struct Vector* div_scalar(struct Vector* v, void*
    scalar, int* errorCode) {
     struct Vector* v2 = ...;
     if (!is_zero(scalar))
         for (int i = 0; i < length(v); ++i)
             set_i(i, v2, div(get_i(i, v), scalar));
     else
     {
         *errorCode = ERROR_DIV_BY_ZERO;
         return NULL;
     }
     return v2;
}

// Примерыиспользования
{
     int errorCode = ERROR_OK;
     struct Vector* v2 = div_scalar(v, scalar, &
        errorCode);
     if (errorCode)
         // обработкаошибки
     else
        // нормальноетечениеалгоритма  ...
}
```

Listing 4.2 – Идентификация ошибок их кодированием

Подход решает проблему и отличается относительной простотой, но небезопасен, мешает композируемости вычис-

лений и порождает неявные зависимости. Во-первых, факта получения результата вычислительного процесса недостаточно для продолжения процесса, необходимо проверять некий внешний объект на наличие ошибки. Во-вторых, состояние ошибки контролируется только авторами библиотеки, тогда как пользователи не могут на него влиять (эту проблему можно снять, введя глобальную функцию для задания сообщений об ошибках). Кроме того, вычисления могут выполняться параллельно, а значит, потенциально, в один и тот же момент времени сразу несколько независимых процессов могут обратиться к значению одной и той же глобальной переменной.

4.3 Возврат значения вычисления и структуры с описанием ошибки (ErrorInfo)

Вместе со значением может быть возвращена структура с кодом ошибки и сообщением, описывающим ее, а также, возможно, указателем на вложенную цепочку ошибок. Также, если хранить статус выполнения операции, то эта структура возвращается после выполнения каждой операции, а не только в случае возникновения ошибки.

Возникает необходимость быстрого определения, успешно ли завершилось вычисление. Это можно реализовать через ленивую инициализацию с мемоизацией: сообщение об успешном выполнении существует не более чем в одном экземпляре, как и нулевой вектор, нулевая матрица и т. д., что позволяет экономить память и другие вычислительные ресурсы. Реализация выглядит так: объявляется переменная, содержащая указатель на объект и булевский признак того, что этот объект создан. Когда мы обращаемся к объекту впервые, мы его инициализируем (ленивая инициализация) и для того, чтобы не выполнять это действие дважды, объект запоминается (мемоизация).

```
struct ErrorInfo {
    int code;
    char* message;
```

```
    struct ErrorInfo* innerError;  // опционально
};

struct ErrorInfo SUCCESS_RESULT;
bool is_init = false;

// Ленивая инициализация с мемоизацией
struct ErrorInfo GetSuccResult() {
    if (!is_init) {
        SUCCESS_RESULT.code = 0;
        SUCCESS_RESULT.message = NULL;
        is_init = true;
    }
    return SUCCESS_RESULT;
}

// Генерация объекта  ErrorInfo с заданными параметрами
struct ErrorInfo GetErrorInfo(int code, char* msg) {
    struct ErrorInfo errorInfo;
    errorInfo.code = code;
    errorInfo.message = msg;
    errorInfo.innerError = NULL;
    return errorInfo;
}

struct Vector* div_scalar(struct Vector* v, void*
   scalar, struct Vector* result) {
    for (int i = 0; i < length(v); ++i)
    {
        if (!is_zero(scalar))
            set_i(i, result, div(get_i(i, v), scalar)
                );
        else
        {
            struct ErrorInfo errorInfo = GetErrorInfo
                (10, "division by zero");
            return NULL;
        }
    }
    return result;
}
```

```
// Примерыиспользования
struct Vector* vResult = div_scalar(v1, scalar, &
    result);
if (!vResult) {
    // обработатьошибку
    struct ErrorInfo* error = GetLastError();
    // ...
}
```

Listing 4.3 – Структура данных ErrorInfo

Структура ErrorInfo поддерживает рекурсивное вложение ошибок через поле innerError. Это позволяет одной ошибке содержать информацию о другой, создавая иерархию. Например, если ошибка деления вызвана другой ошибкой в вычислениях, можно передавать цепочку причин. Структура ErrorInfo также содержит код ошибки и сообщение о ней.

Функция div_scalar(), как и раньше, выполняет поэлементное деление вектора на скаляр. Если scalar равен нулю, создается объект ErrorInfo с кодом 10 и сообщением ”division by zero”, при этом div_scalar() возвращает NULL. В противном случае обновляется переданный result. Ошибки можно обрабатывать через функцию GetLastError().

Метод безопасен и информативен, но препятствует композиции, и приходится возвращать два значения вместо одного.

4.4 Объединение результатов вычисления и обработки ошибок (ResultInfo)

Развитием предыдущей идеи является использование структурированного результата выполнения, где хранится признак успеха и, в случае успеха, результат выполнения операции, а в случае неуспеха — вся информация об ошибке.

```
template <typename T>
class ResultInfo {
private:
```

```
    T result;
    exception* ex;
    bool isOk;
public:
    ResultInfo(T value): result (value),
                         isOk(true),
                         ex(nullptr) {}
    ResultInfo(exception* except): ex(except),
                                   isOk(false) {}

    bool IsOk() { return this->isOk; }
    T GetResult() {
        if (this->isOk)
            return this->result;
        else
            throw new exception();
    }
    exception* GetException() {
        if (!this->isOk)
            return this->ex;
        else
            throw new exception();
    }
};
```

Такая структура приближает реализацию к композиционной модели вычислений. Но при использовании ResultInfo каждый вычислительный блок имеет типы аргументов и результатов не T, а ResultInfo<T>. Это означает, что каждая функция должна обрабатывать ResultInfo и генерировать его, причем эти действия стандартны. Средства, которые автоматизируют распаковку и запаковку результатов, будут рассмотрены позже.

На текущий момент при взаимодействии сложных систем данный вариант считается оптимальным. Используя этот подход, средствами языка C++ можно добиться композиционного построения вычислений, избегая необходимости передачи исключений через множество уровней.

4.5 Использование исключений в C++

В C++ есть встроенный механизм обработки ошибок через исключения (try-catch), который позволяет автоматически перехватывать ошибки и управлять потоком выполнения программы. Математически это можно интерпретировать как переход в новое состояние (скажем, в состояние обработки ошибок). Вместо явной проверки возвращаемых значений или кодов ошибок, исключения позволяют ”чище” разделять логику основного алгоритма и обработки ошибок, что удобнее для дальнейшего сопровождения:

```
struct Vector* div_scalar(struct Vector* v, void*
    scalar) {
     if (is_zero(scalar))
         throw std::runtime_error(”Division by zero”);

     struct Vector* v2 = ...;
     for (int i = 0; i < length(v); ++i)
         set_i(i, v2, div(get_i(i, v), scalar));
     return v2;
}

// Примерыиспользования
{
     try {
         struct Vector* result = ...;
         // ...
     }
     catch (const std::runtime_error& e) {
         std::cerr << ”Error: ” << e.what() << std::
             endl;
         // обработкаошибки ...
     }
}
```

Если вычисление локализовано в одном процессе, механизм исключений, используемый в C++, оказывается наиболее подходящим. Вычисления могут быть и не локальными (например, система взаимодействующих веб-сервисов или некоторых дру-

гих параллельных агентов). В этом случае обработка исключений невозможна, поскольку все исключения инкапсулированы в соответствующие вычислительные единицы. Единственным надежным способом обмена данными между компонентами является структура ResultInfo.

Недостатком этого метода выступают дополнительные затраты производительности, поскольку выброс и перехват исключения выполняются медленнее, чем простая проверка условия. Обработка исключений требует значительных ресурсов. В высокочастотных вычислениях (например, миллионы итераций в секунду) даже небольшая вероятность (0.1-1%) исключений замедлит выполнение цикла на порядок. Это связано с несколькими причинами: во-первых, как и у предыдущего метода, когда происходит ошибка (программа выбрасывает исключение) создается объект исключения, который также содержит информацию об ошибке. Программа выходит из текущей области выполнения (разрушая объекты в стеке), ищет обработчик ошибки, восстанавливается и продолжает работу.

Некоторые другие особенности, вызываемые использованием механизма исключений try-catch:

1. Усложнение на уровне восприятия. При обработке исключений с помощью try-catch значительная часть кода оказывается обернутой в такие конструкции. Код становится более раздутым и нагроможденным, появляются новые уровни вложенности.

2. Нарушение линейного потока исполнения. Программы выполняются последовательно, даже с учетом ветвления: выполнится либо блок if, либо блок else, а дальше снова продолжится последовательное выполнение. Если где-то возникает исключение, естественная последовательность нарушается, возникают проблемы с генерацией значения на выход из такого вычислительного блока;

такое значение-заглушка, если может быть создано, протаскивается через все оставшиеся блоки и должно быть явно в них обработано.

3. Места возникновения и обработки исключений часто разнесены, из-за чего выявить связь между ними бывает непросто. А композиционность, среди прочего, предполагает некоторую степень локализованности и изолированности.

Следует также обратить внимание, что конструкция try-catch не является фундаментальной концепцией идентификации и локализации ошибок. Её основная идея в том, чтобы изолировать код, который может вызвать ошибку в процессе выполнения программы. Цель этого – обработка ошибок без прерывания работы программы.

5. Общие операции над коллекциями (map-reduce)

Обработка коллекций данных – неотъемлемая часть практически любой вычислительной задачи. Однако прямой императивный с использованием циклов и промежуточных состояний часто приводит к громоздкому, сложному для анализа и модификации коду. Функциональные операции над коллекциями предлагают альтернативу: они позволяют выражать сложные преобразования через композицию простых, детерминированных операций. Это не только повышает выразительность и читаемость кода, но и закладывает основу для его оптимизации и параллелизации.

5.1 Отображение: операция map

Операция map применяет функцию преобразования к каждому элементу коллекции, в результате чего порождается новая коллекция. Использование map позволяет абстрагироваться от низкоуровневых итераций, сосредоточившись на сути преобразования, и исключает возможность случайного изменения исходных данных.

Формально операцию отображения можно представить следующим образом:

$$map\, f\, [a_1, a_2, a_3, \ldots, a_n] \mapsto [f(a_1), f(a_2), f(a_3), \ldots, f(a_n)],$$

где $f : T_1 \to T_2$ – функция преобразования, $[a_1, \ldots, a_n]$ – исходная коллекция, $[f(a_1), \ldots, f(a_n)]$ – результирующая коллекция. Если рассматривать данную операцию как метод класса, то ей может быть приписан тип $map : (Seq\langle T_1\rangle \times (T_1 \to T_2)) \to Seq\langle T_2\rangle$, а в качестве самостоятельной функции – $map : ((T_1 \to T_2) \times Seq\langle T_1\rangle) \to Seq\langle T_2\rangle$.

Мы реализуем операции над коллекциями как методы. Проблема: стандарт C++ не гарантирует, что нестатический метод можно типизировать и рассматривать как функцию с явным первым параметром this, из-за чего невозможны некоторые приёмы передачи его как указателя на функцию; поддержка таких конструкций зависит от компилятора.

Пример реализации функции map на языке C++ может выглядеть так:

```
template<typename T1, typename T2>
Sequence<T2>* Map(Sequence<T1>* s, T2 (*f)(T1)) {
    Sequence<T2>* s2 = new ArraySequence<T2>();
    for (int ind = 0; ind < s->GetCount(); ind++)
        s2->Append(f(s->Get(ind)));
    return s2;
}
```

Listing 5.1 – Реализация функции map на языке C++

В коде создается новая последовательность, в которую циклически добавляются результаты применения функции f к каждому элементу исходной последовательности.

Несмотря на кажущуюся простоту, именно эта операция лежит в основе таких концепций, как параллельные вычисления и ленивые последовательности.

5.2 Свертка: операции reduce и fold

Операция $reduce$ (свертка) используется для сведения коллекции к единственному значению. Механизм ее работы состоит в том, что к элементу-аккумулятору и к каждому элементу коллекции последовательно применяется некоторая бинарная функция, образуя элемент-результат:

$$reduce : Seq\langle T1\rangle \times (T2 \times T1 \to T2) \to T2$$

$$reduce\, f\, [a_1, a_2, ..., a_n]\, acc \mapsto f(...f(f(acc, a_1), a_2)..., a_n)$$

В функции $reduce$ важно правильно выбрать начальное значение аккумулятора acc: неверное значение может исказить результат операции. Кроме того, если бинарная операция f не является ассоциативной, результат будет зависеть от порядка вычислений.

Операция $fold$ является аналогичной $reduce$ и часто упоминается в том же контексте, но в качестве аккумулятора в ней используется первый элемент коллекции, что делает ее однородной:

$$fold : Seq\langle T\rangle \times (T \times T \to T) \to T$$

$$fold\, f\, [a_1, a_2, a_3, ..., a_n] \mapsto f(...f(f(a_1, a_2), a_3)..., a_n)$$

Пример реализации функции $reduce$ в C++:

```
template<typename T1, typename T2>
T2 Reduce(Sequence<T1>* s, T2 init, T2 (*f)(T2, T1))
{
    T2 acc = init;
    for (int i = 0; i < s->GetCount(); i++)
        acc = f(acc, s->Get(i));
    return acc;
}
```

Listing 5.2 – Реализация функции reduce на языке C++

В данной реализации аккумулятор инициализируется внешним значением $init$, что обеспечивает корректную обработку пустых коллекций.

5.3 Операция flatMap

Операция $flatMap$ представляет собой синтез map и $reduce$: каждому элементу в соответствии с заданной функцией сопоставляется коллекция элементов, после чего все коллекции сцепляются в одну.

$$flatMap : Seq\langle T_1\rangle \times (T_1 \to Seq\langle T_2\rangle) \to Seq\langle T_2\rangle$$

Например, с помощью $flatMap$ можно получить все делители набора чисел: необходимо задать функцию, возвращающую коллекцию всех делителей одного числа, а $flatMap$ соберет все такие коллекции в результирующую.

Операция map является частным случаем $flatMap$, когда каждому элементу сопоставляется коллекция ровно из одного элемента.

5.4 Фильтрация: операция where

Операция $where$ (также встречается как $filter$, $find$) применяет к каждому элементу коллекции предикат $p(a)$, который имеет булево значение. Если предикат возвращает $true$, то элемент включается в результирующую коллекцию. Таким образом, $where$ не меняет сами элементы, но изменяет состав коллекции, оставляя только элементы, удовлетворяющие заданному условию.

$$where\ p\ [a_1, \ldots, a_n] \mapsto [a'_1, \ldots, a'_k]$$
$$\forall i \in \overline{1, k}\ a'_i \in [a_1, \ldots, a_n],\ p(a'_i) = true,\ k \leq n$$

С помощью операции $where$ выполняется отбор данных по определенным критериям, обработка граничных значений. Операция позволяет сократить объем информации, необходимой для дальнейших преобразований, чем повышает общую эффективность вычислений.

5.5 Сцепление и распаковка: операции zip, unzip

Операция zip объединяет элементы двух коллекций в одну коллекцию пар. Длина результирующей коллекции определяется минимальной длиной исходных коллекций, лишние элементы отбрасываются.

$$zip : Seq\langle T_1 \rangle \times Seq\langle T_2 \rangle \to Seq\langle T_1 \times T_2 \rangle$$

$$zip\,[a_1,\ldots,a_n]\,[b_1,\ldots,b_m] \mapsto [(a_1,b_1),\ldots,(a_k,b_k)]; k = min(n,m)$$

Обратная операция $unzip$ выполняет разделение коллекции пар на две независимые:

$$unzip : Seq\langle T_1 \times T_2\rangle \to Seq\langle T_1\rangle \times Seq\langle T_2\rangle$$

$$unzip\,[(a_1,b_1),\ldots,(a_n,b_n)] \mapsto [a_1,\ldots,a_n],[b_1,\ldots,b_n]$$

Эти операции необходимы при попарных операциях, объединении и совместной обработке данных из различных источников.

5.6 Агрегирующие функции

Агрегирующие функции представляют собой частные случаи $reduce$, используемые для вычисления различных, преимущественно статистических, характеристик коллекции.

Некоторые примеры таких функций:

– Подсчет элементов ($count$):

$$count\,[a_1,\ldots,a_n] = fold\,(\lambda x.x+1)\,[a_1,\ldots,a_n]$$

– Среднее аррифметическое значение (avg):

$$avg\,[a_1,\ldots,a_n] = \frac{fold\,(+)\,[a_1,\ldots,a_n]}{count\,[a_1,\ldots,a_n]}$$

– Максимум (max):

$$max\,[a_1,\ldots,a_n] = fold\,(\lambda xy.if\;x \geq y\;then\;x\;else\;y)\,[a_1,\ldots,a_n]$$

Агрегирующие функции (SUM, AVG, COUNT, MAX, MIN) активно используются в SQL-запросах для обработки больших объемов данных. В СУБД агрегирующие функции могут выполняться в параллельном режиме, что ускоряет обработку больших объемов данных.

5.7 Функциональные списки

Функциональные списки — это структуры данных, которые поддерживают неизменяемость и ленивые вычисления. Эти списки имеют ключевое отличаются неизменяемостью в процессе работы программы, вместо этого создаются новые списки, основанные на старых (с учетом неизменности их элементов).

В функциональных языках программирования обычно используют такой набор операций для работы с функциональными списками:

- Конструктор списка :: — операция добавления элемента в начало списка. Так в выражении $1 :: 2 :: 3 :: 4 ::[]$ означает создание списка $[1, 2, 3, 4]$, где 1 — это голова списка, а оставшаяся часть — хвост.

- **hd** — функция для извлечения первого элемента списка (головы):
$$\mathbf{hd}(1 :: 2 :: 3 :: 4 ::[]) = 1$$

- **tl** — функция для извлечения хвоста списка - всех элементов списка, кроме головы:

$$\mathbf{tl}(1 :: 2 :: 3 :: 4 ::[]) = 2 :: 3 :: 4 ::[]$$

Такой подход эффективен при работате со списками. Здесь элементы списка могут быть вычислены по мере необходимости, что может быть полезно при работе с большими объемами данных. Функциональные списки используются для реализации более сложных структур данных (очереди, стеки, деревья и т.п.). В функциональных языках программирования, таких как Haskell, их использование - обычная практика, т.к. списки одна из базовых структур данных, поддерживающих функциональный стиль программирования

5.8 Лямбда-функции в C++

Лямбда-функции (анонимные функции) позволяют инкапсулировать короткие блоки логики непосредственно в месте их применения. Они делают использование операций вроде map и $reduce$ удобным и практичным.

Синтаксис лямбда-выражения в C++ имеет следующий вид:

$$[capture](parameters) \rightarrow return_type\{body\}$$

где:

- $capture$ – переменные, копируемые или захватывающиеся по ссылке из внешней области видимости;
- $parameters$ – параметры функции;
- $return_type$ – тип возвращаемого значения (может быть выведен автоматически);
- $body$ – тело функции, реализующее требуемую логику.

Механизм захвата переменных может быть задан поразному:

- Захват по значению: $[x, y]$ – создает копии переменных;
- Захват по ссылке: $[\&x, \&y]$ – переменные доступны по ссылке, изменения внутри лямбда-функции влияют на внешние переменные;
- Захват всех переменных по значению: $[=]$;
- Захват всех переменных по ссылке: $[\&]$.

Пример лямбда-функции, складывающей два числа:

```
auto add = [](int a, int b) -> int { return a + b; };
```

Тип возвращаемого значения может быть опущен, и тогда лямбда-функция может быть записана компактнее:

```
auto add = [](int a, int b) { return a + b; };
```

Лямбда-выражения в C++ позволяют удобно и безопасно определять небольшие функциональные объекты, сохраняя при этом доступ к окружающему контексту, что значительно улучшает выразительность и лаконичность кода.

5.9 Применение операций для матричной и векторной арифметики

Рассмотрим применение операций над коллекциями для реализации некоторых преобразований линейной алгебры.

Сложение векторов

Сложение векторов можно реализовать через композицию zip и map: соответствующие элементы векторов попарно объединяются, затем к каждой паре применяется операция сложения.

```
template <typename E, size_t n>
Vector<E, n> operator+(const Vector<E, n>& v) const {
    return Vector<E, n>(
        this->elems->zip(v.elems)
        .map([](Pair<E, E> t)
            -> E { return t.Item1 + t.Item2; }
        )
    );
}
```

Listing 5.3 – Сложение векторов через zip и map

Здесь $Item1$, $Item2$ – первая и вторая проекции пары. Для типа должен быть определен оператор сложения.

Скалярное произведение векторов

Скалярное произведение реализуется аналогично сложению, но после map с попарным умножением применяется

$reduce$ или $fold$ с операцией сложения для суммирования полученных произведений.

```
template <typename E, size_t n>
E operator*(const Vector<E, n>& v) {
    return this->elems->zip(v.elems)
        .map([](Pair<E, E> t)
            -> E { return t.Item1 * t.Item2; }
        ).fold([](E acc, E item)
            -> E { return acc + item; }
        );
}
```

Listing 5.4 – Скалярное произведение векторов

Произведение матриц

Для умножения матриц введем операцию декартова произведения двух коллекций - метод $product$:

```
template <typename T1, T2>
Sequence<Tuple<T1, T2>> product(Sequence<T2>* s) {
    return this->flatMap([s](T1 el1) {
        return s->map([el1](T2 el2) {
            return Tuple<T1, T2>(el1, el2);
        });
    });
}
```

Listing 5.5 – Декартово произведение коллекций

Операция $flatMap$ применяется к каждому элементу $el1$ первой коллекции. С помощью операции map создается пара ($el1$, $el2$) для каждого элемента $el2$ из второй коллекции, после чего все списки пар объединяются в единую коллекцию.

Тогда итоговое вычисление произведения матриц сводится к формированию декартова произведения столбцов одной матрицы на строки другой, вычислению скалярного произведения для каждой из этих пар и формированию из результатов новой матрицы ($split(n)$ делит полученный одномерный список на n подсписков-строк).

```
template <typename E, size_t n>
Matrix<E, n> operator*(const Matrix<E, n>& m2) {
    return Matrix<E, n>(
        this->GetCols().product(m2.GetRows())
        .map([](Tuple<Vector<E, n>, Vector<E, n>> t)
            {
             return t.Item1 * t.Item2;
        })
        .split(n)
    );
}
```

Listing 5.6 – Умножение матриц

Таким образом, операции, относящиеся к классу map-$reduce$, предоставляют практичный и емкий инструментарий для реализации сложных вычислительных алгоритмов, позволяя описывать, что именно нужно сделать с данными, а не как это должно быть реализовано.

5.10 Монады

5.10.1 Понятие идиомы в программировании

Идиома – это концепция, приём, общая идея, которая может реализовываться в конкретных языках и системах разными способами.

Пример: указатель как идиома. Это переменная, значение которой – адрес объекта определённого типа. Обычно тип объекта явно указан, но не всегда – например, $void*$ в C. В абстрактном виде указатель – это нечто, поддерживающее:

- Возможность указывать на элемент в некоторой известной области данных.
- Возможность обратиться к этому элементу.
- Арифметику указателей: перемещение по области данных.

Реализация указателя как типа данных – лишь один из вариантов реализации этой идиомы. Другими примерами являются итератор и индекс в массиве, поскольку они тоже используются как инструмент для доступа к элементу в известной области и перемещения по ней.

5.10.2 Монада как идиома

Так же, как у идиомы «указатель» есть разные реализации, у нас есть идиома «монада». Формально, монада $M\langle T\rangle$ – это параметризованный тип данных, для которого определены операции:

- $from : T \to M\langle T\rangle$ – «единицу»: функцию, погружающую значение типа T в алгебру $M\langle T\rangle$, т.е. порождение монады из экземпляра;
- $flatMap : M\langle T_1\rangle \times (T_1 \to M\langle T_2\rangle) \to M\langle T_2\rangle$ – отображение, которое распаковывет значение внутри исходной

монады, применяет преобразование и возвращает новые значения в другой монаде.

Математически $from$– мономорфизм: различным T соответствуют различные M<T>, но обратного взаимно-однозначного соответствия нет, так как, например, в монаду могут быть инкапсулированы ошибки.

Мы получаем стандартизованный интерфейс и набор операций, применяемых ко всем коллекциям. Вне зависимости от особенностей конкретной коллекции, мы можем проделать над ней некоторые операции, используя этот набор. Правда, не всегда эти операции имеют одинаковую алгоритмическую сложность для разных коллекций.

5.10.3 Реализация монады для обработки ошибок: ResultInfo

При использовании традиционного подхода с исключениями ($try - catch$) каждый вычислительный блок может прервать выполнение программы, что приводит к нелинейности кода и необходимости ручной обработки исключений на каждом уровне.

Мы пришли к выводу, что для обработки ошибок в C++ удобнее всего использовать $ResultInfo\langle T\rangle$, в котором инкапсулировано значение, если его удалось получить, и сообщение об ошибке в ином случае.

Дополним реализацию класса $ResultInfo$, приведенную ранее (см. листинг 4.4). Введем методы $Value$, $Exception$ – реализации «единицы», функции $from$. Они дублируют конструктор и нужны для семантической ясности.

```
template <typename T>
class ResultInfo {
public:
    static ResultInfo<T> Value(T value) {
        return ResultInfo(value);
    }
    static ResultInfo<T> Exception(std::exception* ex
        ) {
```

```
        return ResultInfo(ex);
    }
// ...
};
```

Listing 5.7 – Класс ResultInfo: методы Value и Exception

Базовый вариант реализации функции Map:

```
template <typename T, typename T2>
ResultInfo<T2> ResultInfo<T>::Map(T2 (*f)(T)) {
    if (this->isOk) {
        try {
            T2 r = f(this->result);
            return ResultInfo<T2>::Value(r);
        }
        catch (exception* e)
            return ResultInfo<T2>::Exception(e);
    }
    else
        return ResultInfo<T2>::Exception(this->ex);
}
```

Listing 5.8 – Реализация Map для ResultInfo

Логика метода: если значение успешно – применить функцию и запаковать результат, иначе – протащить ошибку дальше. Если при применении функции возникает исключение, оно перехватывается и помещается внутрь $ResultInfo$ как состояние ошибки.

Формально, в $ResultInfo$ должен быть реализована операция $flatMap$, но рассматриваемая задача не предполагает возвращения списка значений: мы всегда ожидаем список из одного элемента, поэтому здесь достаточно метода map. Причем он реализован с учетом того, что и объект-аргумент – всегда список из одного значения. В данном контексте мы понимаем эту операцию как принимающую объект, инкапсулированный в монаду, которая должна достать из монады значение, применить к нему переданную функцию и результат выполнения снова инкапсулировать в монаду.

Теперь процесс вычислений может быть выражен как линейная цепочка:

```
ResultInfo<Tn>::Value(a).Map(gn). ... .Map(g1)
```

Преимущества – не нужно вручную ставить try-catch вокруг каждого шага и протаскивать «заглушки» вместо нормальных значений; не возникает побочных эффектов; код сохраняет линейную, композиционную структуру.

Если на каком-то шаге произошла ошибка, то вычисление не останавливается. Вместо этого информация об ошибке инкапсулируется в $ResultInfo$ и этот объект продолжает участвовать в цепочке.

5.10.4 Расширения DSL

Для вычислительной полноты можно расширить DSL, добавив условное ветвление и другие операции.

Можно переименовать Map в $AndThen$ для большей семантической ясности:

```
ResultInfo<T>::Value(a).AndThen(gn). ... .AndThen(g1)
```

Такая запись делает линейность выполнения программы еще более очевидной. Развивая идею, можно реализовать методы $If(...)$, $then()$, $Else()$, $end()$, тогда ветвления представимы в виде:

```
.AndThen(gi)
    .If([](x) => ...)
        .then()
        .AndThen(...)
        .end()
    .Else()
        .AndThen(...)
        .AndThen(...)
        .AndThen(...)
        .end()
.AndThen(...)
```

Так может быть получен язык программирования внутри языка программирования, который позволяет писать почти «скриптово», сохраняя читаемость. Стиль и оформление играют важную роль для понимания подобных конструкций: без правильно подобранных имен методов и отступов восприятие было бы сильно затруднено.

Для полной реализации такого стиля нужно создать параметры-генераторы:

1. подтип $IfResultInfo$ с методом $.then()$, возвращающим подтип $IfThenBodyResultInfo$;
2. в $IfThenBodyResultInfo$:
   - метод $.AndThen(...)$, возвращающий $IfThenBodyResultInfo$;
   - метод $.end()$, возвращающий $ElseResultInfo$.
3. в $ElseResultInfo$ - метод $Else()$, возвращающий $ElseBodyResultInfo$;
4. в $ElseBodyResultInfo$:
   - метод $.AndThen(...)$, возвращающий $ElseBodyResultInfo$;
   - метод $.end()$, возвращающий $ResultInfo$

Реализация требует введения большого количества вспомогательных классов, но каждый из них имеет небольшую сложность. Результат – выстраиваем безопасную и удобную композицию вычислений, поддерживая обработку ошибок, условные и другие управляющие конструкции, внутри монадообразного типа ($ResultInfo$), без нагромождения $try - catch$ и с чистой, читаемой логикой.

Текущий DSL можно расшить следующими операциями:

– switch/case-подобные структуры (множественный выбор);

– обработчики конкретных типов ошибок: $Catch<TypeError>(...)$, $.Finally(...)$, аналогичные конструкциям try-catch-finally в C++;

– параллельную композицию, то есть независимые ветки вычислений с последующим объединением результатов (логическую, не обязательно реальное многопоточное исполнение).

Итак, получена иллюстрация сразу трех концепций:

1. $ResultInfo$ как структура, инкапсулирующая результаты вычислений, успешные и неуспешные. Она позволяет абстрагироваться до определенного уровня возникновения нештатных ситуаций в процессе вычислений. Мы избавляемся от необходимости взаимодействия с try-catch.

2. Использование монад. Поддерживается композиционный стиль построения вычислений.

3. Вычисление описывается на предметно-ориентированном языке (DSL), встроенном в нашу вычислительную систему. Мы унифицируем API для любых коллекций и структур данных и достаточно лаконично, понятно и безопасно описываем шаги обработки данных.

6. Синтаксический подход к объектно-ориентированному программированию

6.1 Предпосылки

Программирование на языке C++ строится на концепции объектно-ориентированного подхода, который сочетает в себе возможности процедурного и объектного программирования. Основные предпосылки создания языка включают в себя ряд причин математического и технического толка.

Одна из причин - техническая и логическая разобщенность структур данных и алгоритмов. Техническая заключается в том, что в ранних языках программирования (в том числе C) структуры данных были определены отдельно от операций, которые над ними выполнялись. Например, для работы с массивами или списками необходимо создавать функции, не связанные непосредственно с самими структурами данных. Это означает, что изменение структуры данных могло потребовать изменения множества функций для работы с ними, что усложняло поддержание и сопровождение программ. Логическая разобщенность заключалась в том, что на относительно высоком уровне логики приходится передавать дополнительные данные для корректной работы операций. В C часто приходится использовать указатели и структуры для передачи параметров функции - это, опять же, создает сложности при проектировании и масштабировании программ. В случае, например, с map: T1 -> T2 передача типов данных требует ряд ”обходных путей” для обеспечения корректного функционирования.

Когда в рамках программы использовались различные функции или переменные с одинаковыми именами, это приво-

дило к коллизиям, особенно в достаточно больших проектах. Для примера - две библиотеки могут использовать одинаковые имена для функций, при компиляции программы это вызывает ошибки. Для решения этого приходится прибегать к префиксы и суффиксы, что не всегда были удобно и эффективно, иногда даже некорректно.

Также в С нет поддержки полиморфизма - это ограничивает возможность работы с объектами разных типов. Это требует, как обсуждалось ранее, написания отдельных функций для каждого типа данных. Для реализации обработки различных типов данных в одной функции (параметрический полимфорфизм) приходится использовать небезопасный подход с void*. Так или иначе, все же есть возможность реализовать Ad-hoc полизморфизм, однако он имеет свои неудобства и ограничения.

В ранних языках программирования не существовало стандартных механизмов для обработки ошибок (например, тех же исключений). Ошибки должны были обрабатываться вручную в каждом случае с использованием классических подходов, описанных в 4 лекции.

В 70-е годы (как раз во времена разработки языка С++) исследовались новые способы представления и обработки знаний (фреймы и семантические сети, например), соответственно возникла потребность в описании объектов и их взаимосвязей, т.е. в работе с данными и операции над ними на более высоком уровне абстракций. Разработчики языка учитывали эти тенденции и внедряли концепции объекто-ориентированного подхода.

Основной идеей, лежащей в основе С++, было создание инструмента, в котором возможно моделировать логику и структуру заданной предметной области через объекты, их свойства и методы.

6.2 Программирование, ориентированное на объекты

Объектно-ориентированное программирование (ООП) позволяет организовывать код в виде объектов, которые взаимодействуют друг с другом. Ключевые принципы ООП:

– Абстракция - выделение исключительно значимых характеристики объекта, при этом детали его реализации скрываются, упрощая взаимодействие с ним.

– Инкапсуляция - ограничение доступа к данным объекта и объединение их с методами, которые управляют ими, чтобы предотвратить нежелательные изменения (концепция ”Черного ящика”).

– Наследование - позволяет одному классу (дочернему) получать свойства и методы другого класса (родительского). Один класс может иметь один или несколько классов-родителей и дочерних классов

– Полиморфизм - способность объектов разных классов использовать один и тот же интерфейс, при этом каждая реализация ведет себя по-своему.

Здесь важно разделять понятия класса и структуры, т.к. структура по умолчанию имеет публичные (видимые для пользователей) члены. Для сокрытия содержимого в классах (в т.ч. соблюдения принципа инкапсуляции) используют модификаторы доступа $private$ и $protected$. Причем private оставляет доступ только внутри самого класса, protected дает доступ внутри класса и его наследникам. Типы наследований и изменения модификаторов доступа будет рассмотрено далее.

В завершение этой вводной части порекомдуем некоторые источники для дополнительного чтения и углубленного изучения предмета. В первую очередь, разумеется, это стандартная (во смыслах этого слова) книга Бьёрна Страуструпа [1]. А также настоятельно рекомендуем изучить книгу [2].

6.3 Класс. Поле. Метод

Класс – это шаблон для создания объектов. Поля представляют собой переменные внутри класса, а методы – функции, оперирующие данными класса. Классы можно соотнести с декартовыми произведениями, точнее – с их внутренним состояние. Компоненты состояния объектов хранятся в полях – переменных, область действия которых ограничена экземпляром класса. Методы при этом обеспечивают единственный способ воздействия на объект, безопасный доступ к закрытым данным и позволяют структурировать логику программы.

```
class Person
{
private:
    string fName;
    string lName;
    static string GetFullName(string fName,
        string lName);
public:
    Person(string firstName, string lastName) //
        constructor
    {
        fName = firstName;
        lName = lastName;
    }

    ~Person() // Destructor
    {
        // Free memory and smth like that
    }

    string GetFName(); // Getter (give value of
        lName)
    string GetLName()  // Getter (give value of
        lName)
    {
        return this->lName;
    }
};
```

Listing 6.1 – Пример объявления класса

Статические static (или классовые) переменные и методы принадлежат самому классу, но не объектам этого класса. Статические поля разделяются между всеми экземплярами и используются для хранения общих данных (простеший пример - счётчики объектов). Статические методы вызываются без создания объекта, также они не могут обращаться к нестатическим членам. Static используется, когда необходимо реализовать общее ”состояние” для всех экземпляров класса или методы, не зависящие от конкретного объекта.

Для взаимодействия с полями класса также используются геттеры (getters) и сеттеры (setters) - методы, которые позволяют получать значения закрытых (private) переменных, или изменять их, проверяя корректность данных, соответственно. Использование геттеров и сеттеров помогает инкапсулировать внутреннее состояние объекта, предотвращая некорректные или нежелательные изменения полей. Так GetLName() является простейшим примером геттера.

Для инициализации и удалении объекта определеяются конструкторы и деструкторы. Конструкторы автоматически вызываются при создании объекта и инициализируют его поля, т.е. задают начальное состояние. Деструкторы же вызываются при удалении объекта и освобождают занятые ресурсы (память, файловые дескрипторы). Конструкторы могут быть перегружены для поддержки разных вариантов инициализации, при этом деструктор всегда один и не принимает параметров.

Понятие класс следует разделять с понятием множества объектов (имеющих общие свойсвта и т.п.) – строго говоря, это разные понятия. Класс определяет совокупность объектов, но сам по себе не обладает стандартными операциями (в отличие от множества, где заданы стандартные операции, общие для всех множеств). Множество при этом само по себе является

совокупностью обектов. Отличаются они и способом задания: множества задаются распознающей/порождающей функцией или явным перечислением, а классы описываются структурой, внутренним состоянием и поведением. Таким образом, класс - метод определения множества.

6.4 Наследование

Наследование представляет собой механизм, который позволяет одному классу (дочернему) заимствовать свойства и поведение другого класса (родительского). Это форма определения подтипа, где новый класс расширяет или изменяет функциональность родительского класса. В результате, в подклассе могут быть добавлены новые компоненты внутреннего состояния и новые элементы поведения. Более того, возможно изменение или замена существующих элементов поведения родительского класса (переопределение).

Особенностью наследования в C++ является возможность множественного наследования, т.е. класс может наследовать функциональность от нескольких классов одновременно. Это позволяет комбинировать разные типы поведения и состояния в одном подклассе. Для разрешения ситуаций, в которых у разных родительских классов присутствуют одинаковые методы используются такие механизмы как явное указание на родительский класс с помощью операторов разрешения области видимости.

В C++ есть три вида наследования - public, protected и private, такие же, как и модификаторы доступа. Каждый из них обладает своими свойствами, которые определяют, как члены базового класса становятся доступными в производном классе и за его пределами. Ниже предствалена таблица, описывающая различные типы наследования:

Public-наследование используется, когда дочерний класс ”является” базовым (имеется в виду, что производный класс

Доступ в базовом классе	public-наследование	protected-наследов
public	public	protected
protected	protected	protected
private	недоступен	недоступен

Таблица 6.1 – Таблица видимости членов класса при различных типах наследования в C++

логически представляет собой подтип базового класса). Protected, когда класс должен унаследовать функциональность, но она не должна быть доступна извне. Private-наследование же используется когда базовый класс нужен только как реализация, но не как интерфейс (дочерний класс в этом случае не может быть использован как базовый для дальнейшего наследования с сохранением оригинального доступа). Говоря проще, public-наследование синоним ”является” (is-a), private-наследование – ”не является”, лишь использует базовый класс как часть реализации.

Обратим внимание, что в C++ для структур (struct) по умолчанию используется public наследование, для классов (class) же - private.

6.5 Полиморфизм и Подтипизация

Полиморфизм — это способность объектов разных классов использовать один и тот же интерфейс, при этом каждая реализация ведет себя по-своему. Это ключевой механизм ООП, позволяющий писать гибкий и расширяемый код. Существует несколько видов полиморфизма, которые по-разному реализуются в C++.

6.5.1 Ad-Hoc полиморфизм (Перегрузка)

Ad-Hoc полиморфизм, или **перегрузка**, заключается в определении нескольких функций или методов с одинаковым именем, но разными наборами параметров (сигнатурами), отличающихся количеством или типами. Выбор конкретной функции происходит статически, во время компиляции.

```
class Example
{
    public:
        void Print(int a);
        void Print(double a);
};
```

Listing 6.2 – Пример перегрузки методов

При перегрузке функции важно избегать неоднозначностей, особенно если аргументы могут быть приведены к нескольким подходящим типам.

Сигнатура функции и разрешение перегрузки

Сигнатура функции — это конструкция, включающая имя функции, а также количество, типы и порядок следования её аргументов. Тип возвращаемого значения в сигнатуру для разрешения перегрузки не входит.

Разрешение перегрузки выполняется компилятором, который для данного набора аргументов выбирает наиболее подходящую функцию. Если точного совпадения нет, компилятор пытается выполнить неявные преобразования типов. Если подходящих вариантов несколько, возникает ошибка компиляции из-за неоднозначности.

6.5.2 Параметрический полиморфизм (Шаблоны)

Параметрический полиморфизм позволяет создавать функции и классы, которые могут работать с любыми типами данных. В C++ этот механизм реализован через **шаблоны**. Код пишется один раз, а компилятор автоматически генерирует его конкретные реализации для каждого используемого типа.

```
template<typename T>
class Example
{
    T value;
};
```

Listing 6.3 – Создание шаблонного класса

Шаблоны позволяют писать обобщённый код, но могут увеличивать время компиляции и усложнять сообщения об ошибках.

6.5.3 Полиморфизм подтипов (виртуальные методы)

Этот вид полиморфизма тесно связан с наследованием и позволяет работать с объектами производных классов через указатели или ссылки на базовый класс. Поведение определяется фактическим типом объекта во время выполнения программы (**динамическое связывание**). В C++ это реализуется с помощью **виртуальных методов**.

```
class Base
    {
        public:
            virtual void Show();
    };
class Derived : public Base
    {
        public:
            void Show() override;
    };
```

Listing 6.4 – Шаблон использования виртуальных методов

Виртуальные функции должны использоваться там, где ожидается различное поведение у потомков, и обязательно сопровождаться виртуальными деструкторами в базовом классе для корректного освобождения ресурсов.

6.5.4 Подтипизация

Подтипизация — это отношение между типами, при котором один тип (подтип) может использоваться везде, где ожида-

ется другой тип (надтип). В C++ наследование является основным способом **декларирования отношения подтипизации**.

Если класс ‘Derived‘ публично наследуется от ‘Base‘, то ‘Derived‘ является подтипом ‘Base‘. Это означает, что всё, что справедливо для ‘Base‘, справедливо и для ‘Derived‘, но не наоборот.

Отношение подтипизации является рефлексивным, транзитивным и антисимметричным, то есть образует частичный порядок на множестве типов.

6.5.5 Зависимости Типов

Концепция лямбда-куба (Barendregt’s Lambda Cube) классифицирует системы типизации по восьми классам, образующим частичный порядок. Классификация основана на четырёх видах зависимостей:

- **Терм от терма**: Зависимость значения от значения (обычные функции).
- **Тип от терма**: Зависимость типа от значения (например, шаблоны C++ с целочисленными параметрами: ‘template <int N> class Array‘).
- **Терм от типа**: Зависимость значения от типа (параметрический полиморфизм, например, шаблонные функции: ‘template <typename T> void func(T arg)‘).
- **Тип от типа**: Зависимость типа от типа (например, шаблонные классы: ‘template <typename T> class MyContainer‘).

Взаимодействие подтипизации с зависимостями от типов описывается понятиями ковариантности и контравариантности:

- **Ковариантность**: Если ‘T1‘ является подтипом ‘T2‘, и ‘C<T1>‘ является подтипом ‘C<T2>‘, то зависимость ‘C‘

от ‘T‘ ковариантна. Направление подтипизации сохраняется.

– **Контравариантность**: Если ‘T1‘ является подтипом ‘T2‘, но ‘C<T2>‘ является подтипом ‘C<T1>‘, то зависимость ‘C‘ от ‘T‘ контравариантна. Направление подтипизации инвертируется.

Эти правила дополняют правила подтипизации и разрешения перегрузок.

6.6 Объектная модель и абстрактные типы данных

Основой для проектирования в ООП является **объектная модель** — представление рассматриваемой системы в терминах объектов и отношений между ними. Этот подход является отправной точкой для структурирования сложных систем.

Предметная область — это та часть реального или воображаемого мира, которую мы моделируем. В её основе лежит гипотеза о том, что любую систему можно описать через:

– Отдельные, различимые предметы (индивиды).

– Отношения между этими предметами.

В программе мы оперируем не самими предметами, а их именами, которые составляют **мир речи** (universe of discourse) — образ предметной области в языке.

Эта идея лежит в основе как языков программирования, так и моделей представления знаний, и нашла широкое применение в разработке информационных систем.

Переход к ООП синтаксически упростил работу с объектной моделью. Если раньше для доступа к атрибуту объекта требовался вызов отдельной функции, то языки ООП ввели прямой синтаксис (например, ‘объект.атрибут‘), что создало прочную ассоциацию между объектом и его членами (атрибутами и методами).

При написании классов мы отталкиваемся от объектной модели: каждому типу объектов предметной области сопоставляется класс, а хранимым атрибутам — приватные поля с доступом через публичные методы (геттеры).

Абстрактный тип данных (АТД) — это тип, определённый своей внешней спецификацией (интерфейсом), в то время как детали его реализации скрыты. В C++ эта концепция реализуется с помощью классов, которые инкапсулируют данные и методы их обработки.

В C++ основным механизмом для достижения абстракции является регулирование области видимости членов класса с помощью модификаторов доступа:

- **public**: Члены доступны из любой части программы.
- **protected**: Члены доступны внутри самого класса и в классах-наследниках.
- **private**: Члены доступны только внутри самого класса.

Понятие ”тип” в языках программирования отличается от его математического аналога.

- В математике тип — это множество объектов, удовлетворяющих условиям.
- В программировании тип — это, как правило, именованное множество объектов с заданной структурой.

Равенство типов в C++ является **декларативным** (по имени), а не **структурным** (по строению). Два класса с идентичной структурой, но разными именами, будут считаться разными типами.

Абстрактный класс — это класс, содержащий хотя бы один чисто виртуальный метод. Такой класс нельзя инстанцировать, он предназначен для использования в качестве базового.

Интерфейс — это частный случай абстрактного класса, который содержит только чисто виртуальные методы. Он используется для определения ”контракта” — набора методов, которые должен реализовать любой класс-наследник.

В базовых классах с виртуальными функциями обязательно должен быть **виртуальный деструктор**, чтобы гарантировать корректное освобождение памяти при удалении объекта производного класса через указатель на базовый класс.

6.7 Перегрузка операторов

C++ позволяет переопределять операторы для пользовательских типов (классов и структур), чтобы обеспечить интуитивное и естественное поведение при использовании стандартных операций, заданных на множестве объектов (например, +, -, ==, [], () и др.) Это делает пользовательские типы похожими на встроенные по поведению и, если угодно, эксплуатации.

```
class Vector
{
private:
    double x, y;
public:
    Vector(double x, double y) : x(x), y(y) {}

    Vector operator+(const Vector& other)
    {
        return Vector(this->x + other.x, this->y
            + other.y);
    }

    bool operator==(const Vector& other) const
    {
        return x == other.x && y == other.y;
    }
};
```

Listing 6.5 – Пример перегрузки операторов

При этом важно, чтобы оператор отражал семантику операции: предположим, если + используется, то результат не должен изменять операнды. Также лучше не перегружать операторы, поведение которых может ввести в заблуждение (например, && или ,). В особенности перегрузка такого оператора как operator= требует аккуратного подхода, с акцентом на работу с указателями (не забывать про самоприсваивание). Важно помнить про симметрию операций (если то предусматривает алгебра): если a + b работает, то желательно, чтобы b + a также имел смысл — для этого стоит писать внешние (неклассовые) перегрузки с friend или шаблонные.

Некоторые операторы язычк не позволяет перегружать: например, . (точка), ::, sizeof, typeid

6.8 Глобальные константы

Глобальные константы используются для определения неизменяемых значений:

```
const double PI = 3.14159;
```

Listing 6.6 – Объявление численной константы

Несмотря, на первый взгляд, их полезность, константы в ряде случаев могут быть вредны и опасны в некоторых ситуациях. Для начала, глобальная константа всегда существует вне класса, т.е. она никогда не принадлежит объекту, а также доступна в ЛЮБОМ месте кода, вне зависимости от контекста и доступа (напомним, что в ООП концепция противоположная - данные должны быть инкапсулированы внутри объектов, чтобы их изменение или замена не затрагивала остальную систему).

Простейший случай их вреда - когда класс обращается к некоторой глобальной контанте (const double PI), то невозможно перенести этот класс в другойхъпроект или решение, где значение этой константы может быть другим или вообще отсутствовать. Из-за этого класс становится менее гибким.

Глобальные константы могут конфликтовать по именам, в особенности если они определены в заголовках и подключаются во множестве программных модулях. Особенно болезненно в больших проектах и библиотеках. Кроме того, для const объектов сложных или пользовательских типов в глобальной области видимости порядок инициализации между модулями никогда не гарантирован. Это потенциально приводит к ошибке ”static initialization order fiasco”.

6.9 Обработка ошибок и еонцепция ResultInfo

Структура, инкапсулирующая результат вычислений (часто называемая ‘Result Info‘), позволяет достичь компромисса между явной обработкой ошибок и сохранением установленного порядка вычислений.

При композиции вычислительных блоков возникновение исключения в одном из них может привести к неопределённому результату всей конструкции. Стандартный механизм обработки исключений (‘try-catch‘) не всегда удобен, так как передаёт ответственность за обработку вызывающему контексту.

Подход с использованием ‘ResultInfo‘ решает эти проблемы, делая обработку ошибок явной и локализуемой. Объектно-ориентированные языки предоставляют элегантные средства для реализации этого подхода.

7. Семантический, или осмысленный, подход к объектно-ориентированному программированию

Одрой из частых ошибок, совершаемых новичками в программировании, и иногда и их более опытными коллегами, состоит в том, что программирование на объектно-ориентированных языках рассматривается как продвинутая версия модульного программирония: классы – это модули, методы в них – это функции; наследование – способ повторного использования кода, и т.д. Этим обстоятельством и обусловлено название предыдущей главы. Как показывает опыт, такой подход имеет целый ряд недостатков, некоторые из которых неопытному программисту далеко не сразу становятся заметны и понятны. В частности, написанный программный код трудно соотносить с требованиями к программному обеспечению и с моделями, построенными аналитиками (а следовательно и сложнее проверить соответствие кода этим требованям и моделям). Кроме того, код в целом при этом обычно оказыается организован так, что тего не слишком удобно поддерживать и развивать. У этих проблем имеется множество различных проявлений, и некоторых из них мы обсудим далее в этом разделе.

В настоящей же главе мы займемся обсуждением более осмысленого, содержательного и даже, как иногда говорят *семантического* подхода к ООП. По ходу дела мы не только немного по-другому посмотрим на уже обсуждавшиеся концепции и конструкции, но и отчасти пересмотрим восприятие ООП как такового.

7.1 Предпосылки

Начать обсуждение вопроса нужно с того, отметим, что исторически сложилось так, что языки программирования разивались параллельно с такими областями, как базы данных и искусственных интеллект, и конкретно та часть этой области исследования, которая связана с представлением знаний и моделированием рассуждений человека. Обусловлена эта связь тем, что в 1960-х – 1970-х гг происходило формирование того, что мы сегодня называем разработкой информационных систем, а в сущерсии эта задача сводится к тому, чтобы некоторым образом смоделировать и представить в виде совокупности программ и данных определенный набор знаний и предметной области. Не будем сейчас вдаваться в обсуждение подробностей, связанных с самими терминами: «данные», «знания» и др. Сейчас важно отметить, что отправной точкой для всего последующего является принятие т.н. гипотезы о предметной области. Но сначала напомним определение.

Сама же **гипотеза о предметной обалсти** формулируется так.

Гипотеза 1. *Предметная область состоит из отдельных, различимых предметов (называемых также* индивидами*), наделыемых тем, что называется атрибутами, а также между этими индивидами наблюдаются (устанавливаются) т.н. связи – выделенные конфигурации индивидов.*

Определение 7.1.1. ***Атрибут*** *– ситуация, когда некоторому предмету (индивиду) приписывается (еще говорят – с предметом соотносится) какое-либо значение, называемое значением атрибута.*

Определение 7.1.2. ***Связь*** *– ситуация, когда для данного набора предметов устанавливается (обнаруживается, фиксируется) определенное отношение, в котором эти предметы состоят. Эти предметы называются участниками связи, а у самих связей иногда обнаруживаются атрибуты.*

В литературе можно встретить и другую терминологию: атрибуты называют характеристиками, значения атрибутов – значениями характеристик.

Сама же гипотеза о предметной области, особенно в ее полном варианте, требует достаточно подробного обсуждения в рамках целой серии курсов; это обсуждение отложим на некоторое время. Различные аспекты этого обсуждения можно найти в ряде работ [ВЭВ, РВВ, ...] Мы привели несколько измененную, упрощенную формулировку, которая лучше подходит для целей дальнейшего изложения.

7.2 Объекты данных и процессы

Ранее мы уже говорили о том, что по одной из классификаций структуры данных подразделяются на две категории: изменяемые (называемые нередко мутабельными, от англ. *mutable*) и неизменяемые (называемые также немутабельными, или иммутабельными, от англ. *immutable*). В таких терминах об объекта обычно говорят с точки зрения определенных на них операциях, а когда хотят подчеркнуть особенности функционирования объектов во времени, часто прибегают к другой терминологии.

Объекты, хранящие состояние, – это объекты (структуры), в которые инкапсулированы какие-либо данные, причем выполнение методов (результаты их вызова), во-первых, зависит от этого инкапсулированного состояния, а во-вторых, может эти данные менять. Совокупность инкапсулированных данных в этом контексте и называется *(внутренним) состоянием*.

Результатом выполнения функции (метода) является не только возвращаемое значение, но и, что немаловажно, побочные эффекты, в данном случае это в первую очередь как раз изменения внутреннего состояния объектов. Отчасти, вопросы, связанные с побочными эффектами и взаимодействием вычислений со средой, обсуждаются в некоторых других разделах,

например, в связи с понятием замыкания. Однако серьезное изучения вопроса с привлечением необходимых формализмов приходится отложить до другого случая.

При этом часто считается, что объекты, хранящие состояние, не должны реализовывать никакой логики; их ответственность – хранить данные, а не выполнять какие-либо вычисления. В предельном случае, у объектов, хранящих состояние, нет методов (кроме, возможно, геттеров и сеттеров). Двойственным же является понятие *объектов, реализующих поведение*, – т.е. таких объектов, которые не имеют собственного состояния, и выполнение методов (результаты их выполнения) зависит только фактических значений входных аргументов.

Эти два понтия иногда воспринимают максималистки, требуя, чтобы объекты, хранящие состояние, буквально не содержали никаких методов (кроме, быть может, геттеров и сеттеров), в то время как объекты, реализующие поведение, буквально не должны содержать никакого внутреннего состояния. И хотя в целом эти два понятия приводят к весьма продуктивным идеям и подходам к разработке, буквальное следование этим требованиям приводит к различным сложностям.

Мы будем следовать другому подходу, опирающемуся на содержание, смысловую нагрузку, подразумеваемый смысл, или семантику, создаваемых объектов. Для этого мы введем два других понятия.

Объект данных – объект или структура хранящая данные (описывающая или представляющая информацию) о каком-либо объекте, обычно, в терминах атрибутов, приписываемых этому объекту. Эта концепция напрямую вытекает из принятия обсуждавшейся ранее гипотезы о предметной области. Введение этой концепции во многом связано с развитием некоторых идей из искусственного интеллекта, в первую очередь, в части моделей представления знаний, а также из теории и практики баз (реляционных) данных.

Идея заключается в том, что каждый объект воспринима-

ется через призму набора характеризующих его атрибутов, абстрагируясь, в базовом варианте, от природы и структуры рассматриваемого объекта. Если два объекта обладают одинаковым набором атрибутов (а атрибут обычно характеризуется (а) названием, причем с названием обычно связывается некоторая смысловая нагрузка, хоть обычно и на неформальном уровне, и (б) типом допустимых значений), то такие объекты считаются однотипными; а если у объектов еще и при этом совпадают значения атрибутов, то они становятся неотличимы друг от друга. При этом следует выделить атрибуты: (а) хранимые и (б) вычислимые.

С концептуальной точки зрения, *хранимые атрибуты* – те, значения которых приписываются объекту и никак не вытекают из его природы или структуры. Например, имя, которое дается человеку при рождении; или, скажем, фамилия; или, например, в конексте инвентаризации – инвентарный номер. С физической, или, если угодно, с точки зрения реализации, хранимый атрибут – это совокупность, включающая:

1. поле, хранящее значение (*backing field*);

2. геттер (*getter*) – метод, возвращающий значение этого поля, обычно, без дополнительных манипуляций;

3. [опционально] сеттер (*setter*) – метод, позволяющий изменить значение поля, хранящего значение, причем обычно это новое значение задается непосредственно, и оно же непосредственно заменяет преженее.

Если конкретный атрибут (или тем более объект в целом) не предполагает изменений, сеттер может отсутствовать. Впрочем, нередко сеттер все же добавляется, но принимаются соглашения, что используется он только на этапе создания объекта и иногда в некоторых других случаях. Современная практика разработки предполагает, что объекты данных являются неизменяемыми – по крайней мере, везде, где это возможно. Пред-

почтение отдается неизменяемым объектам, но есть и ситуации когда накопление изменений является неизбежным, а в некоторым случаях – и вовсе основным предназначением объекта данных, так что в целом объекты данных могут быть как изменяемыми, так и нет.

Вычисляемый атрибут – это атрибут, значение которого вычисляется исходя из либо внутренней структуры или природы объекта, или на основании значений других атрибутов. Атрибуты первой категории можно условно назвать *исходными*, в то время атрибуты второй – *производными*. Вычисляемые атрибуты почти никогда не бывают изменяемыми, поскольку их значение зависит от других элементов, а создание неочевидных побочных эффектов, каковым и является изменение других атрибутов при попытке изменить значение данного, считается крайне нежелательным и плохим тоном.

Объекты данных в целом не предполагают, что они реализуют какую-то сложную логику, или поведение, тем более, затрагивающее другие (связанные) объекты. Например, логика, связанная с переводом студента из одной учебной группы в другую, всегда будет реализована за пределами как объекта, описывающего студентов, так и объекта, представляющего учебные группы.

Объекты, реализующие процессы (или просто *объекты-процессы*) – это объекты или структуры данных, которые представляют какую-либо логику (часто говорят – логику предметной области) или вычисления. Обычно каждому процессу или вычислению из предметной области сопоставляется свой объект, а декомпозиция этих процессов или вычислений соотносится с декомпозицией объектов. Обычно подразумевается, что методы, реализованные в таких объектах – это различные варианты вызова соответствующего процесса/-вычисления (например, при различных доступных наборах исходных данных).

Существует большой пласт обсуждений, связанный с се-

мантикой выполнения процессов, который мы тут вынуждены по большей части пропустить. Речь идет о взаимодействии объектов друг с другом и с тем, что называется средой. Это включает и обсуждение вопросов, связанных с созданием побочных эффектов. На данном этапе мы ограничимся следующим *предписанием*. Процессы не должны создавать неожиданных побочных эффектов, не подразумеваемых непосредственно решаемой ими задачей. Если же процесс создает какой-то побочный эффект, он обязательно должен быть документирован и на него должны быть указания в интерфейсе объекта, например, в именах методов или самого объекта.

7.3 Объекты как глобальные константы

В первом приближении понятие константы в языках программирования – несложная конструкция. Суть сводится к тому, константа – это идентификатор, объявленный в определенной области видимости, для которого компилятор гарантирует невозможность каким-либо образом изменить значение.

С логической точки зрения константа – это какое-либо значение, не меняющееся в зависимости от контекста вычисления, в т.ч. во времени. Основная сложность в реализации этой концепции средствами того или иного императивного языка программирования состоит в том, что использование ключевого слова `const` не позволяет решить задачу, т.к. оно требует, чтобы справа от знака = стояло выражение, значение которого может быть вычислено на этапе компиляции, а это означает, что создание объекта должно быть не только атомарным (этого можно добиться, реализовав подходящий конструктор или/и порождающий статический метод), но и не всякая конструкция, относящая к категории выражений (даже константных), является допустимой (если конкретно, это должно быть `const` `r`-value).

С логической точки зрения многие объекты, во-первых,

сложно устроены, а во-вторых, из-за этого создаются поэтапно, в несколько приемов. Скажем, векторы и матрицы: сначала требуется выделить память для буфера, в котором будут храниться их элементы, а затем уже задать значение каждого элемента (в лучшем случае – значение каждого ненулевого элемента). Из-за этого с формальной точки зрения такие объекты не всегда удается сделать строго неизменяемыми. Но любая матрица и любой вектор (как многие другие объекты, например, списки и в целом различные последовательности) – с логической точки зрения, это константы, хоть и сложно устроенные.

В идеале, каждому вектору должен бы соответствовать единственных экземпляр соответствующего класса; добиться этого в принципе возможно и даже не слишком сложно. Проблема состоит в том, что временные затраты на проверку того, что нужный экземпляр уже создан, сопоставимы с затратами на порождение этого объектаю, причем эти затраты нередко приходится нести «поверх» вычисления элементов[1], а единственный плюс – в потенциальной экономии памяти, которую занимает такой объект, и во многих случаях эта экономия не оправдывает затраты.

Поэтому на практике интерес представляет хранить не все элементы, а только некоторые, имеющие особое значение, и часто используемые. В таком случае возникает два вопроса:

– в какой момент материализовать, т.е. инстанцировать, эти объекты?

– как обеспечить, чтобы эти объекты не создавались повторно?

[1]В качестве примера можно рассмотреть реализацию операции сложения произвольных матриц: для того, чтобы понять, существует ли уже такая матрица, сначала необходимо вычислить все ее элементы, а затем этот набор элементов передать в функцию поиска/сравнения.

В качестве решения возьмем идиому, которую назовем «объект как глобальная константа».

Идея заключается в том, чтобы с помщью статического порождающего метода создавать экземпляр в момент, когда он впервые запрашивается, а значение запоминать в статическом поле. Это называется ленивая инициализация с мемоизацией.

```
class Vector3 {
private:
    static Vector3* m_zero;
public:
    static const Vector3& Zero() {
        if (!Vector3::m_zero)
            Vector3::m_zero = new Vector3(0.0f, 0.0f,
                0.0f);
        return *Vector3::m_zero;
    }
private:
    float x;
    float y;
    float z;
public:

    Vector3(float x = 0.0f, float y = 0.0f, float z =
        0.0f)
        : x(x), y(y), z(z) {}

    Vector3 operator+(const Vector3& rhs) const {
        return Vector3(x + rhs.x, y + rhs.y, z + rhs.
            z);
    }

    Vector3 operator-(const Vector3& rhs) const {
        return Vector3(x - rhs.x, y - rhs.y, z - rhs.
            z);
    }

// .....
};
```

Для начала, разберемся, почему `m_zero` является указате-

лем. Причина довольно проста, хотя и состоит из нескольких частей:

– Нужен какой-то признак, по которому можно было бы понять, инстанцирован ли уже объект или нет. При использовании указателя таким признаком служит значение этого указателя `nullptr`; если же типов `m_zero` будет `Vector3`, без указателя, то потребуется дополнительная (булева) переменная.

– Объявление `static Vector3 m_zero;` компилятор не примет, поскольку, упрощенно говоря, получается рекурсивное, бесконечное определение класса. В случае с указателем проблем не возникает, т.к. указатель – это всегда завершенное определение, размер объекта (в памяти) всегда понятен.

– Даже если бы вариант без указателя был допустим, объект пришлось бы как-то инициализировать (нарпимер, вызовом конструктора по умолчанию, которого может и не быть, но это отдельная проблема). А после этого – создавать новое, уже требуемое значение в методе `Zero()`, что в итоге приводит к лишним затратам.

В самом методе `Zero()` условием `!Vector3::m_zero` проверяем, что поле `m_zero` не инициализировано, и, если это так, инициализируем. Наличие этого условия, которое будет проверять каждый раз при вызове метода `Zero()`, можно считать достаточно небольшой платой за общую эффективность решения.

Если же требуется реализовать структуру, представляющую вектор некоторой произвольной размерности, то если эта размерность передается параметром в конструктор, тогда применение описанной концепции осложняется, поскольку нужно для каждой размерности хранить свой нулевой вектор, а значит потребуется вспомогательная структура – словарь

(размерность будет выступать в роли ключа). Однако если размерность вынести как параметр типа (по терминологии C++, сделать параметром шаблона класса), то эта проблема исчезает.

```
template<typename T, size_t N>
class VectorN {
private:
    static VectorN<T,N>* m_zero;
public:
    static const VectorN<T,N>& Zero() {
        if (!Vector3::m_zero)
            Vector3::m_zero = new Vector3(
                DynamicArray<T>::From(0.0f, N));
        return *Vector3::m_zero;
    }
private:
    DynamicArray<T> items;
public:

    Vector3(DynamicArray<T> items)
        : items(items) {}

// .....
};
```

7.4 Отсутствующие значения: альтернатива NULL

7.5 Неоднородные коллекции: дизъюнктивная сумма

Описать с помощью either: $\mathbb{Z} \setminus \{0\}$. Очевидно, придется написать свой тип, в котором будет инкасулировано $\mathrm{Either}\langle \mathrm{Minus}(\mathrm{Unit}), \mathrm{Plus}(\mathrm{Unit})\rangle$, где $\mathrm{Minus}(\mathrm{Unit})$ представляет отрицательную часть целочисленного ряда, а $\mathrm{Plus}(\mathrm{Unit})$ – положительную.

7.6 Подтипизация и дизъюнктивная сумма

7.7 Косвенные связи между объектами

7.8 Снова об обработке ошибок: композиция вычислений

7.9 Концепты как средство реализации алгебры типов

рассказать немного про алгебру типов в идеале

описать пример, как с помощью концептов можно описать тип – неотрицательное целое, ненулевое целое, и как это будет работать при композиции функций

8. Замыкания и семантика передачи параметров

8.1 Передача параметров по ссылке и по значению

Для начала объясним, что такое ссылка. Допустим, есть переменная `v1` типа `T1`, а переменная `v2` имеет тип `T1&` и является ссылкой на `v1` (`T1 &v2 = v1`). Это означает, что с переменной `v2` мы можем взаимодействовать так, как будто работаем с объектом `v1`, не копируя его содержимое (например, чтобы обратиться к `v1` из другого метода).

Передача параметра по ссылке — это способ, при котором функция получает ссылку на исходную переменную, а не её копию, в отличие от случая, когда мы передаём параметр по значению.

Продемонстрируем разницу между передачей параметров по ссылке и по значению. Объявим две функции `f1` и `f2` и применим их к переменной. `f1` будет принимать аргумент по значению, а `f2` — по ссылке.

```
int f1(int x)
{
    return ++x;
}

int f2(int &x)
{
    return ++x;
}

int main()
{
    int y = 4;

    std::cout << f1(y) << ", ";
    std::cout << y << ", ";
```

img/pointer-and-ref.drawio.png

Рисунок 8.1 – Указатель и ссылка в памяти

```
    std::cout << f2(y) << ", ";
    std::cout << y << std::endl;
}
```

Listing 8.1 – Демонстрация передачи параметров по ссылке и по значению

Если запустить этот код, в консоль будет выведено «5, 4, 5, 5». Обе функции вернули одно и то же значение, однако `f2` произвела побочный эффект: изменила значение переменной `y` вне функции. Это произошло потому, что в `f1` оператор инкремента применялся к копии объекта `y`, а в `f2` — к самому объекту, так как переменная `x` являлась ссылкой на `y`.

8.2 Различия между указателем и ссылкой

Не стоит путать понятия указателя и ссылки. Несмотря на то, что они оба указывают на какой-то объект, они обладают разной семантикой (см. рис. 8.1). Указатель — это численное значение, которое обозначает ячейку памяти, с которой начинается объект.

При этом для указателя определены операции сложения и вычитания. Например, мы можем прибавить целое число n к указателю, и тогда он будет указывать на ячейку, адрес которой равен выражению:

img/pointer-and-ref-assign.drawio.png

Рисунок 8.2 – Присваивание значения ссылке и указателю (см. разницу с рис. 8.1)

текущее значение + sizeof(тип, на который указывает указатель) * n.

Ссылку же скорее можно рассматривать как псевдоним для другого объекта. При этом объект, на который ссылается ссылка, должен быть определён в момент её создания и не может меняться позже. То есть мы можем поменять то, на что ссылается указатель, однако при попытке присвоить другое значение ссылке мы скопируем его по адресу, где раньше лежало значение, обозначаемое ссылкой (см. рис. 8.2).

8.3 Семантика копирования

Ранее мы сказали, что при передаче параметра по значению функция получает копию переменной, но как эта копия создаётся? Для составных типов вроде структур и классов это происходит при помощи особого вида конструктора — конструктора копирования (в отличие от базовых типов, для которых просто

копируются байты из одной области памяти в другую). Конструктор копирования принимает один параметр — объект, который нужно скопировать, и на его основе строит новый объект. Этот новый объект и получает функция при передаче параметров по значению. В C++ конструктор копирования объявляется следующим образом:

```
T(const T& object) { /* тело конструктора */ },
```

где `T` — тип (класс), а `object` — объект, копию которого мы создаём. По сути своей это обычный конструктор, который автоматически вызывается в момент, когда необходимо создать копию объекта.

Если конструктор копирования не указан программистом явно, то создаётся конструктор копирования по умолчанию. Этот конструктор вызывает конструктор копирования для каждого своего поля. Однако возникают сложности при копировании объектов классов, которые содержат в себе указатели. Формально для компилятора указатель — это просто номер ячейки, в которой находятся данные. Следовательно, при копировании указателя происходит копирование только этого номера, и указатель продолжает ссылаться на те же данные, что и до этого. Приведём пример, когда это может создать проблемы. Пусть есть класс `DynamicArray`, который содержит указатель на массив.

```
template <typename T>
class DynamicArray
{
    private:
        T *storage;
    public:
        void Add(T* storage);
};
```

Listing 8.2 – Класс DynamicArray для примера с конструктором копирования

Если скопировать этот класс, используя конструктор копирования по умолчанию, копия всё ещё будет использовать ту же область памяти, что и оригинальный объект, и при добавлении нового элемента он появится и в исходном массиве. Такое поведение может быть нежелательным. Вместо указателя мы могли бы использовать тип, для которого уже определён конструктор копирования, однако далеко не всегда стоит создавать глубокую копию объекта (например, ради оптимизации).

8.4 Семантика перемещения

Конструктор перемещения (`move constructor`) используется для передачи ресурсов от одного объекта к другому без их копирования. В отличие от конструктора копирования, который создаёт новую копию объекта, конструктор перемещения забирает владение ресурсами у временного объекта. Это позволяет избежать лишнего копирования данных, особенно когда объект содержит динамическую память или другие ресурсы.

В C++ конструктор перемещения объявляется так:

```
T(T&& other) { /* тело конструктора */ }
```

где `T&&` — это тип, обозначающий rvalue-ссылку на объект типа `T`. Такой конструктор вызывается, когда создаётся новый объект из временного (`rvalue`), например при передаче временного объекта в функцию или при использовании `std::move`. Функция `std::move` не перемещает данные сама, а лишь преобразует объект в временный, позволяя компилятору вызвать конструктор перемещения.

Рассмотрим следующую композицию функций:

```
f1(f2(x)).
```

`f2(x)` возвращает промежуточный объект, который не будет никак использован, кроме как в качестве аргумента функции `f1`. Поэтому имеет смысл вызвать конструктор перемеще-

img/func-compose.drawio.png

Рисунок 8.3 – Диаграмма композиции функций для двух случаев

ния вместо копирования. Возвращаясь к предыдущему примеру с `DynamicArray`, это как раз тот случай, когда мы не хотим выполнять глубокую копию объекта.

Эту композицию функций можно представить следующей диаграммой (см. рис. 8.3).

Функции `f1` и `f2` можно рассматривать как изолированные от внешней среды блоки, но с односторонней границей: из блоков есть доступ к внешней среде, но не наоборот. Видно, что сформированный в `f2` объект должен пройти через две границы — вернуться из `f2` и передаться в `f1`. Единственный способ преодолеть эти барьеры — скопировать (или переместить) объект. Таким образом, объект нужно будет скопировать дважды — по одному разу на каждый барьер. Однако если передавать объект в `f2` не по значению, а по ссылке, второй барьер становится «прозрачным».

У такого подхода есть большой недостаток: если возвращаемый `f2` объект был создан внутри функции в стеке, то при возврате он будет уничтожен. Поэтому вместо того чтобы возвра-

щать ссылку, необходимо использовать указатель на объект в куче.

8.5 Замыкание

Изолированные блоки вычислений, обсуждаемые в предыдущем разделе, в контексте языка C++ имеют более точное значение, а именно значение области видимости (scope). Область видимости обозначается отдельной синтаксической конструкцией — фигурными скобками.

```
int x = 10;
{
    int y = 15;
    //Область видимости переменной y
}
x++;
y++;
```

Listing 8.3 – Область видимости переменной

Код в примере выше не скомпилируется, так как переменная `y` входит в текст программы за пределами её области видимости.

Под вхождением переменной (идентификатора) понимается любое упоминание переменной (идентификатора) в тексте программы в месте, отличном от её декларации. Другое важное понятие — связывание значения с переменной. В коде выше можно рассматривать объявление `int x = 10` как связывание значения 10 с переменной `x`, а вхождение этой переменной в `x++` — как вхождение, связанное при помощи этого объявления.

Связывание и вхождение — термины, которые применяются и за пределами текстов программ. Они могут использоваться в формальных системах, например в логике, где нет явного определения переменных. В логические формулы переменные входят свободным образом. Единственное, что близко в них к

декларации переменных, — это конструкции вида $f(x, y, z) = \ldots$. Здесь переменные x, y, z считаются связанными, так как являются аргументами функции.

Дополнительно стоит рассмотреть вопрос идентичности переменных.

```
{
    int y = 0;
    //Тут область видимости задаётся явно
}
int x = 10;
foo(x);
//Тут заканчивается область видимости переменной x
   со значением 10
int x = 20;
bar(x);
```

Listing 8.4 – Переменные с одним именем, но разной идентичностью

Область видимости может быть заданна не только явно, но и неявно. Каждая декларация задаёт идентичность и неявную область видимости конкретного идентификатора, пример выше демонстрирует это. Тут функции `foo` и `bar` применяются к переменным с одним именем, но разными идентичностями, т.е. по сути к разным переменным.

8.6 Умные указатели и предпосылки к ссылочным типам

Напомним, что ранее мы обсуждали вычислительные блоки. По аналогии с композицией `f1(f2(x))`, где значение рождается в одной функции, а используется в другой, можно поставить задачу с двумя блоками внутри одной функции: один создаёт объект, второй его потребляет. В связи с этим возникает вопрос, какие должны протекать процессы для успешной передачи этого объекта. Вместо передачи значения структуры, объём которой может быть большим, можно использовать указатель. Но при работе с указателями есть ряд проблем связанных

с управлением паматью. Объект может создаться в одном блоке, использоваться в нескольких других, и при этом в какой-то момент он должен быть уничтожен. Подход используемый в C заключается в том, что объект уничтожается в месте, в котором был создан. При этом этот объект передаётся во все необходимые функции по указателю.

Такой подход порождает код с множеством побочных эффектов, что затрудняет композицию блоков кода и его понимание. Получается, что нужно найти способ управлять жизненным циклом объекта автоматически. Этой задачи служат умные указатели.

Идея умных указателей заключается в следующем: умный указатель хранит в себе объект, на который указывает и понимает, когда это значение больше не нужно. Всё что для этого надо это подсчёт ссылок на хранимый объект.

Приведём возможную имплементациую класса для такого умного указателя.

```
template <class T>
class Ptr
{
public:
    static Ptr<T> GetNull();
private:
    T* pointer;
    int *counter;
public:
    Ptr(T&& item)
    {
        pointer = malloc(sizeof(T));
        *pointer = std::move(item);
        counter = new int(1);
    }

    Ptr(Ptr<T>& const ptr)
    {
        this->pointer = ptr.pointrer;
        this->counter = ptr.counter;
```

```
        *(this->counter)++;
    }

    ~Ptr()
    {
        *(this->counter)--;
        if (*(this->counter) == 0)
        {
            delete this->pointer;
            delete this->counter;
        }
    }

    T* operator->() { return this->pointer; }
    bool operator==(Ptr<T> other);
    bool operator!=(Ptr<T> other);
    bool IsSame(Ptr<T> other);
    bool IsNull();
};
```

Listing 8.5 – Возможная имплементация умного указателя

Обратим внимание на то, что вместо вызова new T() мы выделяем нужное количество байт и копируем объект в выделенную область, чтобы позволить пользователю самостоятельно сконструировать объект с нужными аргументами (ещё конструктора по умолчанию может попросту не существовать). Чтобы это работало мы должны допустить, что объект может быть скопирован.

На принципе умных указателей основано управление памятью в более высокоуровневых языках программирования: C#, Java, JavaScript и т.д.

Для удобства использования умных указателей мы бы хотели, чтобы везеде где ожидается тип `T` мы могли бы использовать тип `Ptr<T>` (Учитывая, что методы принято вызывать в виде `object->method()`, то мы хотим использовать умный укзатаель там где ожидается `T*`, но по смыслу это скорее `T`). Первая проблема, которая у нас возникает - это создание объек-

та. Наш умный указатель должен поддерживать все те же конструкторы, что и тип `T`. Эта проблема обходится конструкцией вида `Ptr(T(...))` (Ранее мы сказали, что объект должен быть копируемым). Вторая, более серьёзная проблема заключается в циклических указателях. Если в какой-то момент образуется циклическая структура, в которой объекты указывают друг ну друга через умные указатели, то счётчик указателей никогда не станет равным нулю. Если бы у умных указателей было бы указанное выше свойство, и отсутствовали бы перечисленные проблемы, то умные указатели по сути были бы эквиваленты ссылочным типам из высокоуровневых языков.

Несмотря на огромное преимущество ссылочных типов в виде автоматической очистки памяти, ими всё ещё стоит обращаться с осторожностью. При передаче объекта ссылочного типа в функцию просиходит передача объекта по ссылке и он может быть изменён, чего программист может не ожидать.

Умные указатели, которые мы описали, в стандартной библиотеке C++ представлены типом `shared_ptr`. Помимо него есть ещё два типа указателей.

1. `weak_ptr` — ведёт себя аналогично `shared_ptr`, но не увеличивает счётчик ссылок. Нужен для решения проблемы циклических зависимостей;

2. `unique_ptr` — указатель, который единолично владеет объектом. Используется, когда нужно передать объект по значению, а не по ссылке.

Из-за проблем с циклическими ссылками (причем на практике длина цикла может превышать 2, т.е. «обратная» связь между объектами может реализовываться через несколько объектов-посредников), а также из-за необходимости порождать для каждого класса хоть и однотипный, но все же несколько различающийся код (конструкторы, деструктор), умные указатели не позволяют реализовать непосредственно

реализовать концепцию ссылочных типов. Наиболее значимая проблема состоит в необходимости т.н. сборщика мусора, который будет отслеживать возникновые «самозамкнутых» конгломератов объектов, на которые больше не ссылок извне. Использование `weak_ptr` – это инструмент, позволяющий обойти описанную проблему. Т.к. счетчик ссылок не увеличивается, можно добиться того, что как только пропадает последняя ссылка на «корневой» объект этого конгломерата, все входящие в него объекты будут поэтапно уничнтожены. Однако этот подход хорошо работает только в иерархических структурах; в графовых его применять сложнее ввиду равноценности дуг между узлами. В языках с автоматическим управлением памятью задача сборщика мусора как раз и состоит в нахождении таких изолированных наборов и инициации их уничтожения.

9. Ленивые последовательности

9.1 АТД LazySequence

АТД LazySequence реализует концепцию ленивого списка (или, если использовать введенную ранее терминологию, ленивой последовательности), т.е. ситуации, когда существует какое-то порождающее правило, по которому любой i-й элемент можно вычислить, возможно, на основе вычисленных ранее, но при этом последовательность в целом бесконечна в интуиционистском смысле . Типичные примеры: последовательность факториалов $\{n!|n = \overline{(1,\infty)}\}$, последовательности Фибоначчи.

С точки зрения классификации структур данных, ленивый список (будем использовать это обозначение как более удобный синоним для термина «ленивая последовательность») – это линейная структура с последовательным доступом. Поскольку в общем случае вычисление значения i-го элемента требует вычисления значений всех предшествующих элементов. Однако если известно, что за время работы программы потребуется не более, чем n первых элементов последовательности, то среднее время получения элемента по индексу будет асимптотически стремиться к $O(1)$ за счет мемоизации уже вычисленных значений. Заметим, что в рассматриваемом контексте (ленивые) списки могут быть как конечными, так и бесконечными структурами. Простейший пример, когда ленивая последовательность является конечной – это ситуация, когда элементов в ней много, а хранятся они, например, в файле на диске или получаются по сети – т.е. материализация последовательности целиком является либо невозможной, либо нежелательной, по каким-либо причинам.

Особенность ленивых списков по сравнению с обычными линейными структурами состоит в деталях реализации ряда

операций, таких как все виды вставки (в начало – prepend, в конец – append, в середину – insert), удаление и некоторые другие. На первый взгляд может показаться, что эти операции вообще не должны быть определены на ленивых последовательностях, и тем более контринтуитивным может показаться, что эти операции работают в целом почти так же, как и для обычных последовательностей, особенно если учесть, что ленивая последовательность не обязана быть бесконечной. Отличие заключается в деталях реализации, в первую очередь, бесконечных ленивых списков. В общих чертах, идея состоит в том, что результирующая последовательность не материализуется как набора элементов; вместо этого (а) сохраняется ссылка (в широком смысле этого слова) на исходную последовательность, (б) в отдельном списке (а лучше, в очереди с приоритетами) запоминается, в какой позиции нужно внести изменение, и какое это изменение – удаление одного или нескольких элементов, добавление элементов или (бесконечного) списка, и т.п.

Ниже приводится описание примерной внешней спецификации ленивой последовательности, при этом можно заметить, что интерфейс LazySequence мало отличается от Sequence. Этого можно добиться, правильно выделив ответственности, декомпозировав объект и введя необходимые абстракции. Вопросы, которые послужат отправной точкой для дальнейших рассуждений, следующие:

1. Как добавлять элементы в ленивую последовательность? В частности, что значит вставить элемент в конец бесконечной ленивой последовательности? А также, что значит вставить элемент (или целую последовательность, в т.ч., возможно, ленивую), в произвольную позицию, заданную индексом?

2. Как обращаться к элементам, в т.ч. как их перечислить. Это также включает ответ на вопрос, каким образом и в

какой момент материализуется очередной элемент последовательности и какой конкретно компонент за это отвечает.

На первый из этих вопросов ответ уже частично дан выше: мы не порождаем результирующую последовательность, а как бы запоминаем, что нужно сделать с исходной, чтобы получить желаемых результат. А ответ на второй вопрос подсказывает способ упростить решение обеих проблем: выделить ответственность «порождение очередного элемента последовательности, заданной процедурно», а объект, реализующий эту ответственность, назовем просто –- генератором. Этот объект инкапсулирует правило порождения очередного элемента, которое может быть как рекуррентным, так и нет. Основной метод у такого объекта – GetNext() – возвращает очередной элемент. Генератор в плане интерфейса напоминает итератор, но в отличие от последнего (а) существует строго в единственном экземпляре для каждого списка, (б) не доступен извне списка, (в) может менять объект, которому принадлежит.

Следует обратить внимание, что:

(а) с одной стороны, ленивая последовательность является неизменяемым объектом в том плане, что в интуиционистском смысле это константа, т.е. сама последовательность в принципе изменена быть не может, просто означивается (материализуется) «не сразу», а «постепенно», а ее бесконечная природа проявляется в том, что процесс перечисления элементов может продолжаться потенциально бесконечно;

(б) с другой стороны, это не обычный неизменяемый объект, какие мы прежде рассматривали, в частности, тут практически нет константных методов, т.к., например, получения элемента по индексу потенциально требует выполнение некоторых вычислений и их мемоизацию, а значит

и изменение того, что в мире императивного программирование принято называть «внутренним состоянием объекта».

Еще раз напомним, что произвольно взятая ленивая последовательность не обязательно бесконечна; однако концепция ленивых структур данных возникла именно в связи с обсуждением возможностей представления бесконечных объектов.

Таблица 9.1 – Внешняя спецификация LazySequence

Создание объекта	
`LazySequence();`	Создать пустой список

`LazySequence(`
`  T* items,`
`  int count);`
`LazySequence(`
`  Sequence<T>* seq);`
	Копировать элементы из переданного массива

`LazySequence(`
`  T(*)(Sequence<T>*),`
`  Sequence<T>)`
`LazySequence(`
`  std::function<T>(`
`  Sequence<T>*)`
`  , Sequence<T>*)`
	Задание (рекуррентного) порождающего правила, использующего k предшествующих элементов для порождения $(n+1)$-го, где k≤n, а также списка первых k элементов

`LazySequence (`
`  LazySequence<T> & list`
`  const)`
	Копирующий конструктор

Декомпозиция	
`T GetFirst();` Может выбрасывать исключения – IndexOutOfRange (если последовательность пуста)	Получить первый элемент в списке
`T GetLast();` Может выбрасывать исключения – IndexOutOfRange (если последовательность пуста) Вычисление может не завершиться ввиду бесконечности последовательности	Получить последний элемент в списке.
`T Get(size_t index);` Может выбрасывать исключения – IndexOutOfRange (если индекс отрицательный или больше/равен числу элементов) Вычисление может не завершиться ввиду бесконечности последовательности	Получить элемент по индексу. На данном этапе рассматривается упрощенная версия, в которой индекс задается неотрицательным целым. Более подробно вопрос индексации элементов обсуждается далее.

`LazySequence <T>* GetSubsequence( int startIndex, int endIndex);` Может выбрасывать исключения – IndexOutOfRange (если хотя бы один из индексов отрицательный или больше/равен числу элементов) Получить список из всех элементов, начиная с startIndex и заканчивая endIndex.	Получить элемент по индексу. На данном этапе рассматривается упрощенная версия, в которой индекс задается неотрицательным целым. Более подробно вопрос индексации элементов обсуждается далее.
`Cardinal GetLength() const ;`	Получить длину списка. Тип Cardinal предназначен для представления множества, включающего обычные натуральные числа и первое трансфинитное число.

`size_t GetMaterializedCount() const;`	Получить количество материализованных элементов. Эта характеристика носит служебный, вспомогательный характер и не является частью логического определения типа. Более того, с логической точки зрения как раз не должно быть такой характеристики – хотя бы потому, что это нарушает принцип абстрагирования внешнего мира от внутреннего содержания и деталей реализации АТД.
Операции	
`Sequence<T>* Append( T item);`	Добавляет элемент в конец списка
`Sequence<T>* Prepend( T item);`	Добавляет элемент в начало списка

`LazySequence <T>*InsertAt(`
`  T item,`
`  int index);`
`LazySequence <T>*InsertAt(`
`  size_t index,`
`  Sequence <T>* items);`
`LazySequence <T>*InsertAt(`
`  size_t index,`
`  LazySequence <T>*`
`  items);`
Может выбрасывать исключения
– IndexOutOfRange (если индекс отрицательный или больше/равен числу элементов)
	Вставляет (возможно, бесконечную) последовательность в заданную позицию.

`LazySequence <T>*Skip(`
`  int index);`
`LazySequence <T>*Skip(`
`  size_t start,`
`  size_t end);`
Может выбрасывать исключения
– IndexOutOfRange (если индекс отрицательный или больше/равен числу элементов)
	«Удаляет», а точнее «пропускает» подпоследовательность элементов. Следует подчеркнуть, что фактического удаления не происходит, отсюда и использование кавычек. Подобробнее об этом будет сказано далее.

`LazySequence <T>*Replace(`
`  int index,`
`  T item);`
`LazySequence <T>*Replace(`
`  size_t start,`
`  size_t end,`
`  Sequence <T>* seq);`
`LazySequence <T>*Replace(`
`  size_t start,`
`  size_t end,`
`  LazySequence <T>* seq)`
`;`
Может выбрасывать исключения
– IndexOutOfRange (если индекс отрицательный или больше/равен числу элементов)
	«Удаляет», а точнее «пропускает» подпоследовательность элементов. Следует подчеркнуть, что фактического удаления не происходит, отсюда и использование кавычек. Подобробнее об этом будет сказано далее.

`LazySequence <T>* Concat(`
`  LazySequence <T> *list`
`);`
Может выбрасывать исключения
– IndexOutOfRange (если индекс отрицательный или больше/равен числу элементов)
	Сцепляет два списка. Очевидно, если первый список бесконечный, то перечисление (например, итератор) дойдет до элементов второго списка только в бесконечности.

`LazySequence<T2>* Map(`
`  T2 (*)(T));`
`T2 Reduce(`
`  T2 (*)(T2,T));`
`LazySequence<T>* Where(`
`  bool (*)(T));`
`LazySequence<T>* Zip(`
`  Sequence<T>* seq);`
`\dots`
`template<typename T1,`
`  \dots,`
`  typename Tn`
`LazySequence<Tuple<T1,\`
`  dots,Tn>> Zip(`
`  LazySequence<T1>* seq1`
`  ,`
`  \dots,`
`  LazySequence<Tn>* seqn`
`  );`
Может выбрасывать исключения
– IndexOutOfRange (если индекс отрицательный или больше/равен числу элементов)
	Реализация многих операций из группы map-reduce ничем принципиально не отличается от реализации этих операций для обычных последовательностей, поскольку использует либо получение элемента по индексу, либо перечисление элементов (т.е. итерирование), что и позволяет абстрагироваться от деталей, связанных с ленивостью и мемоизацией.

Теперь, когда установлены некоторые базовые идеи, необходимо углубиться в детали, в частности, в то, как обходиться с потенциально бесконечными последовательностями и их комбинациями (в т.ч. сцеплением). Это, в свою очередь, требует пересмотреть вопрос об индекчации, т.к., очевидно, обычные числа не способны справится с этой задачей. Поэтому, прежде, чем продолжить разговор о последовательностям, поговрим немного о числах, а точнее об ординалах.

9.2 Ординалы. Арифметика ординалов

Попытки сформулировать принципы, по которым можно было бы корректно обращаться с бесконечными конструкциями, – это и есть то, что способствовало и во многом определило становление классической логики и оснований математики. Откладывая в сторону исторический и аналитический обзоры проблематики, обратимся сразу к интересующей нас конструкции: ординалам, или ординальным числам.

Формально ординалы вводятся как характеристика множеств: это число, связываемое с порядковым типом того или иного (линейно упорядоченного) множества. Пусть M – некоторый линейный порядок, тогда **порядковым типом** M называется совокупность всех изоморфных ему линейных порядков. Можно показать, что эта совокупность формирует некоторый класс эквивалентности, что дает возможность рассматривать лишь один (любой) представитель этого класса. Например, можно использовать числовые последовательности[1] для иллюстрации изложения: вместо различных (в данном случае, для удобства и определенности, конечных) последовательностей из n элементов $\{a_1, \ldots, a_n\}$, $\{b_1, \ldots, b_n\}$,...,$\{c_1, \ldots, c_n\}$ досточно рассматривать последовательность $\{0, 1, \ldots, n-1\}$.

Тогда удобно говорить о последовательности, которую образуют такие классы: элементами этой последовательности будут т.н начальные отрезки ряда натуральных чисел. **Начальным отрезком** некоторой (конечной или бесконечной) последовательности $\{a_1, \ldots, a_{k-1}, a_k, \ldots\}$, **отсекаемым элементом** a_k, называется последовательность $\{a_1, \ldots, a_{k-1}\}$. В терминах теории частичных порядков: начальным отрезком линейного порядка $\langle A, \leq_A \rangle$, отсекаемым элементом $a \in A$, называется линейный порядок $\{x | x \in A \wedge a \leq_A a\}$.

Такие начальные отрезки вполне удобно обозначать числа-

[1] в этом разделе часто будет использовать слово «последовательность» как синоним термину «линейный порядок»

ми, следующими непосредственно за последним элементом:

$$\begin{array}{rcl} \emptyset & \leftrightarrow & 0 \\ \{0\} & \leftrightarrow & 1 \\ \{0,1\} & \leftrightarrow & 2 \\ & \dots & \\ \{0,2,\dots,n-1\} & \leftrightarrow & n \end{array}$$

Для конечных последовательностей затруднений не возникает, вопросы начинаются, как только исчерпываются все конечные последовательности. Первая бесконечная последовательность, с которой мы сталкиваемся – это весь ряд натуральных чисел $\mathbb{N}$. Для конечных последовательностей использовалось число, которое фактически является мощностью последовательности как множества. Для $\mathbb{N}$ это первое трансфинитное число $\aleph_0$, и, значит, для соответствующего порядкового типа мы также можем использовать это трансфинитное число, однако принято в этом контексте его обозначать ω.

Теперь разберемся, что будет дальше. В первую очередь, убедимся, что существуют последовательности, более длинные, чем ряд натуральных чисел. Сначала возьмем ряд четных натуральных: $\{0,2,4,\dots\}$. Его мощность, как мы знаем из вводного курса по логике, – $\aleph_0$, а значит и соответствующий порядковый тип – ω. В конец этой последовательности, т.е. после того, как перечислены все натуральные четные, запишем первое нечетное число: $\{0,2,4,\dots,1\}$. А затем так же можно дописывать и другие нечетные:

$$\begin{array}{c} \{0,2,4,\dots,1\} \\ \{0,2,4,\dots,1,2\} \\ \{0,2,4,\dots,1,2,3\} \\ \dots \\ \{0,2,4,\dots,1,2,3,\dots,n\} \end{array}$$

Несмотря на то что эти последовательности равномощны как множества, это разные последовательности и они

представляют разные порядковые типы, причем всякий раз «прибавляется 1» к предшествующему элементу (применяется инкремент). Если для $\{0, 2, 4, \dots\}$ использовалось число ω, то для $\{0, 2, 4, \dots, 1\}$ можно использовать $\omega + 1$, для $\{0, 2, 4, \dots, 1, 2\}$ – $\omega + 2$, и т.д. Неформально, получается, что после того, как мы «закончили» перечислять бесконечный набор элементов, мы продолжаем перечисление следующими элементами $(1, 2, 3, \dots)$. Если же сцепить две бесконечные последовательности, то получится порядковый тип $\omega + \omega = \omega 2$ (но не 2ω, т.к. $2\omega = \omega$).

Наконец, раз мы можем обращаться с бесконечными последовательностями, то ничто не мешает обращаться и с бесконечными последовательностями бесконечных последовательностей (точнее, речь идет о сцеплении бесконечного количества бесконечных последовательностей). Порядковый тип для такой конструкции, очевидно, будет $\underbrace{\omega + \omega + \dots + \omega}_{\omega \text{ раз}} = \omega^2$.

Теперь мы можем завершить нашу таблицу обозначений, начатую выше:

$$
\begin{array}{rcl}
 & \dots & \\
\{0, 2, \dots, n-1\} & \leftrightarrow & n \\
 & \dots & \\
\{0, 2, \dots\} & \leftrightarrow & \omega \\
\{0, 2, \dots\ 1\} & \leftrightarrow & \omega + 1 \\
\{0, 2, \dots\ 1, 3\} & \leftrightarrow & \omega + 2 \\
 & \dots & \\
\{0, 2, \dots\ 1, 3, \dots, n-1\} & \leftrightarrow & \omega + n \\
 & \dots & \\
\{0, 2, \dots\ 1, 3, \dots\} & \leftrightarrow & \omega + \omega \\
\{0, 2, \dots\ 1, 3, \dots, \dots, p, \dots\} & \leftrightarrow & \omega + \dots + \omega \quad = \quad \omega^2
\end{array}
$$

Этот процесс может продолжаться и дальше, порождая все новые ординальные числа: ω^3, ω^4, …, ω^ω, …, ω^{ω^ω}, …Но для на-

ших достаточно и того, что уже рассмотрено. За подробностями можно обратиться, например, к [3].

Порядковые типы еще называют **ординальными числами**, или просто **ординалами**, и они образуют алгебру. Ее особенность в сравнении с обычной алгеброй чисел, как мы уже видели – в некоммутативности сложения и умножения, поскольку $2 + \omega = \omega \neq \omega + 2$ и $2 \cdot \omega = \omega \neq \omega \cdot 2$.

С точки зрения программной реализации, если исходить из того, что для наших целей достаточно рассматривать наиболее ординальное число – ω^2, то можно для представления таких ординальных чисел использовать просто упорядоченные пары: положить, что (p, k) представляет ординал $\omega \cdot p + k$. В частности, ординалы меньше ω представляются парами $(0, k)$. Ниже приводится примерное определение класса на C++.

```
    class Ordinal
{
public:
    static Ordinal Zero();
    static Ordinal Number(size_t number); // \omega *
     0 + number
    static Ordinal Omega(size_t omega); // \omega *
   omega + 0
    static Ordinal From(size_t omega, size_t number);
    static Ordinal OmegaSquare();
private:
    bool isOmegaSquare;
    size_t omega;
    size_t number;
public:
    bool IsOmegaSquare();
public:
    Ordinal operator+(size_t number);
    Ordinal operator-(size_t number);
    Ordinal operator+(Ordinal ordinal);
    Ordinal operator-(Ordinal ordinal);
public:
    bool operator==(Ordinal ordinal);
    bool operator==(size_t number);
```

```
    bool operator<(Ordinal ordinal);
    bool operator<=(Ordinal ordinal);
    bool operator<(size_t number);
    bool operator<=(size_t number);
};
```

Listing 9.18 – Примерное объявление класса, реализующего ординалы

Нетрудно заметить, что реализация некоторых операций при таком подходе затруднительна, в частности, это касается умножения произвольных нумералов друг на друга. Преодолеть это ограничение не слишком трудно, хотя реализация станет значительно более громоздкой. Идея состоит в том, чтобы представлять нумералы по аналогии с полиномами – коэффициентами (списком коэффициентов, если точнее, причем располагаемых в порядке возрастания) при степенях ω. Так нумерал $\omega^3 + \omega * 2 + 5$ представляется списком: $[5, 0, 2, 1]$. Но представление еще больших ординалов потребует дополнительных ухищрений. Например, содержать в списке не только коэффициент, но и степень, а степень, тоже представлять набором, например: $\omega^{\omega^3} \cdot 4$ представить как $(4, 2, 3)$.

9.3 Арифметика бесконечных ленивых последовательностей

Совместное применение концепций ординалов, ленивых вычислений и мемоизации позволяет построить позволяет построить довольно неожиданные конструкции, например – конкатенацию бесконечного количества бесконечных (ленивых) списков, причем с возможностью обращения к любому элементу по индексу (хотя и не за константное время).

Упрощенно, идея решения состоит в том, чтобы построить список последовательностей, входящих в цепочку конкатенаций, и использовать в качестве индекса ординалы вида $\omega p + q$, где p – номер (индекс) последовательности в цепочке, q – ин-

декс элемента в q-й последовательности[2]. При этом индекс ω ссылается на (виртуальный) элемент, «расположенный в конце» бесконечной последовательности, наподобие тому, как $+\infty$ располагается на расширенной числовой прямой после всех положительных вещественных чисел. Аналогично, $\omega 2$ обозначает виртуальный конец 2-й последовательности, $\omega 3$ – третьей, и т.д., а потому индекс первого следующей последовательности будет, соответственно, $\omega + 1$, $\omega 2 + 1$, $\omega 3 + 1$ и т.д.

$$\begin{array}{ccccccccccccc} a_1 & , & \dots & , & b_1 & , & \dots & , & \dots & , & c_1 & , & \dots & , & \dots \\ 0 & & & & \omega & & \omega+1 & & & & \omega\cdot n & & \omega\cdot n+1 & & \omega^2 \end{array}$$

Сама по себе операция конкатенации не вызывает особых сложностей в плане реализации; единственное, о чем следует позаботиться отдельно – это чтобы пустая последовательность[3] действительно была нейтральным элементом для этой операции, как справа, так и слева.

Но кроме конкатенации, над последовательностями можно выполнять и другие операции, которые в целом сводятся к добавлению и удалению элементов. Для того чтобы такая арифметика последовательностей работала корректно с учетом отложенного характера большей части вычислений, требуется принять ряд специальных мер.

1. Ограниченный набор базовых операций. Мы исключаем такие операции, как удаление заданного элемента, а так-

[2]В действительности, q будет индексом только при $p = 0$, в остальных случаях это номер, а значит индекс будет равен $q - 1$.

[3]Нужно напомнить, что хотя мы в основном употребляем словосочетание «бесконечная последовательность», на самом деле имеется ввиду «*потенциально* бесконечная последовательность». Главное свойство тут – это ленивость последовательности, т.е. то, что ее элементы изначально не материализованы, а окажется ли сама последовательность конечной или бесконечной, или даже пустой, изначально может и не быть известно, а проявляется в процессе выполнения программы.

же произвольно объединение последовательностей. Рассматриваются следующие базовые операции:

1.1. конкатенация слева,

1.2. конкатенация справа,

1.3. удаление подпоследовательности,

1.4. вставка последовательности в определенную позицию.

2. Каждая операция – унарна или бинарна в базовой реализации, а конструкции, позволяющие за один вызов метода, например, составить конкатенацию сразу целого набора последовательностей – лишь надстройка, обращающая к базову функционалу, «синтаксический сахар».

3. Каждая последовательность, *с логической точки зрения* должна быть неизменяемой. Фактически, это изменяемая структура, т.к. изначально большая часть элементов не материализованы, они порождаются по требованию и запоминаются. Однако применение любой операции к последовательности всегда приводит порождению нового объекта, ссылающегося на исходный(е). Это приводит к своеобразному следствию: одна и та же (в математическом смысле) последовательность может существовать в нескольких экземплярах, которые отличаются лишь тем, какая часть элементов оказалась матеариализована.

Исходя из обозначенных посылок, приступим к обсуждению того, как такую конструкцию реализовать. Как обычно, будем руководствоваться принципами разделения ответственностей, локализации и абстрагирования зависимостей.

Первая такая наиболее очевидная ответственность – это порождение элементов (по какому-либо заданному закону). Вторая – хранение уже материализованных элементов в каком-то

ch-lazy-list/pic-lazylist-structure-basic.pn

Рисунок 9.1 – Структурная схема реализации ленивой последовательности

буфере, для чего, в первом приближении, представляется целесообразным использовать какую-либо последовательность (а не множество, т.к. порядок элементов для нас важен). Схематически это выглядит так, как показано на рис. 9.1.

В первом приближении, мы, выделив ответственность – хранение вычисленных значений – зафиксировали компонент, который будет эту ответственность реализовывать – буфер (Buffer). Далее, т.к. ясно, что храниться данные должны именно последовательно, в линейой структуре, мы сразу делегировали соответствующие полномочия уже существующему элементу – АТД последовательность (Sequence). Последующее, более детальное рассмотрение вопроса показывает, что буфер не должен реализовывать никакого дополнительного функционала по сравнению с Sequence. Кроме того, т.к. буфер будет инкапсулирован в LazySequence, к нему доступа извне, а значит и нет необходимости дополнительно экранировать Sequence от внешнему мира, скрывая «лишние» возможности. Поэтому функцию хранения вычисленных элементов можно возложить непосредственно на Sequence.

Мы столь детально обсуждаем принимаемые решения потому, что, по сути, то, чем мы сейчас занимаемся – это и есть процесс проектирования (в данном случае – АТД ленивая по-

следовательность), а качество проектирования во многом зависит от того, удастся ли достичь правильного гранулирования, правильной степени декомпозиции создаваемого объекта на его составляющие. Чрезмерная детализация плоха тем, что приводит к избыточным затратам как на этапе реализации, так и на этапе последующего использования (избыточные траты по памяти и процессорному времени); недостаточная степень детализации ведет к негибкости реализации и, в конечном счете, к тем же избыточным тратам как при реализации, так и при использовании создаваемого программного изделия.

Теперь займемся другим вопросом: порождение элементов. Оно может происходить одним из нескольких заранее определенных способов: с помощью порождающей функции (возможно, рекурсивной, либо реализующей рекуррентное соотношение), из заранее определенного (конечного) набора элементов, в результате применения операции (какой-либо из перечисленных выше). Возможно, в тех или иных обстоятельствах окажется целесообразным предусмотреть и другие варианты, но пока что ограничимся тем, что уже названо. В результате получается схема, изображенная на рис 9.2.

На схеме изображена функциональная зависимость объекта LazySequence от генератора (представленного интерфейсом IGenerator) и последовательности Sequence, используемой для мемоизации. Хотя это и не отмечено на схеме, предполагается, что обе перечисленные сущности инкапсулируются в LazySequence. Шесть указанных наследников соответствуют основным методам порождения элементов: два – в исходных последовательностях, задаваемых явным перечислением (случай SeqGenerator) или порождающей функцией (случай FunctionGenerator), и также по одному – на каждую из основных операций (всего 4).

Предполагается, что выполнение какой-либо операции приводит к созданию нового экземпляра LazySequence, в котором используется генератор, соответствующий выполненной опе-

ch-lazy-list/pic-lazylist-structure.png

Рисунок 9.2 – Структурная схема реализации ленивой последовательности

рации. Его задача – обращаться к «родительским» последовательностям и, исходя из того, какая операция выполнена, решить, из какой из исходных последовательностей нужно брать очередной элемент. Например, если была выполнена операция Skip (т.е. пропустить элементы с i-го по j-й), то как только генератор дойдет до i-го элемента, он должен сразу «перескочить» к $(j+1)$-му – если, разумеется, таковой есть. Если же выполнена операция, например, Insert, т.е. вставка другой последовательности в позицию i, то по достижении индекса i генератор должен «переключиться» на эту вторую последовательность и возвращать из нее элементы до тех пор, пока она не будет исчерпана.

В качестве отдельного замечания, отметим, что на схеме используется отношение подтипизации между компонентами-генераторами (а точнее, то, что эти генераторы объединены общим интерфейсом). В большинстве случаев, фактическим способом реализовать это в языке программирование будет –

определить интерфейс IGenerator, от которого унаследовать необходимое количество классов. Однако в C++ возможен и другой вариант, когда вместо супертипа определяется концепт, служащий ограничением на типах-параметрах. Главный недостаток такого решения в данном случае – то, что возникает необходимость дополнительно (и при том явно) параметризовать LazySequence типом генератора, что, кроме менее лаконичного кода, приводит еще и к потере абстрагирования пользователя от деталей реализации.

Описанная схема предполагает, что необходимо хранить позицию, до которой исходная последовательность (одна или две) была уже просмотрена – для того, чтобы гарантировать корректную нумерацию элементов в результирующей последовательности, особенно, когда она, в свою очередь, будет использовать как аргумент для тех или иных операций. А значит, для этих целей необходим дополнительный компонент, который по смыслу и реализации мало чем отличается от итератора (назовем его, например, LazySequenceIterator). Семантически, это все еще означает, что генератор ссылается на одну или две «родительские» последовательности, но теперь ссылка не прямая уже, а косвенная, посредством итератора. При желании, это можно отразить на структурной схеме, см. 9.3.

Тут все еще не учитывается использование довольно типичного приема: публичный, всем доступный интерфейс итератора, реализация которого приватная, спрятанная внутри объекта, для итерирования которого этот итератор и предназначен.

Теперь приведем, фрагментарно, набросок реализации LazySequence, учитывающей перечисленные особенности. Во-первых, объявление класса выглядить примерно так, как показано на листинге

```
template<typename T>
class LazySequence
{
```

ch-lazy-list/pic-lazylist-structure-ext.png

Рисунок 9.3 – Доработанная структурная схема реализации ленивой последовательности, с учетом использования итераторов

```
private:
    class IGenerator;
    class DefaultGenerator;
    class SkipGenerator;
protected:
    unique_ptr<Sequence<T>> items;//materializedItems
    IGenerator* generator;
public:
    LazySequence(unique_ptr<Sequence<T>> startItems,
   T(*generator)(T))// std::function<>
    {
        //
        //this->generator = UnaryFunctionGenerator(1,
    generator);
    }
    LazySequence(unique_ptr<Sequence<T>> startItems,
   T(*generator)(T,T))// std::function<>
    {
        //
        //this->generator = UnaryFunctionGenerator(2,
    generator);
    }
    LazySequence(unique_ptr<Sequence<T>> startItems,
   size_t generatorArity, T (*generator)(Sequence<T
   >*))// std::function<>
    {
        this->items = move(startItems);
        this->generator = (IGenerator*)new
   FunctionGenerator(shared_ptr<LazySequence<T>>(this
   ), generatorArity, generator);
    }
private:
    LazySequence(size_t startSkip, size_t endSkip,
   shared_ptr<LazySequence<T>> parentSeq)
    {
        this->items = make_unique<Sequence<T>>(new
   ArraySequence<T>());
        this->generator = (IGenerator*)new
   SkipGenerator(shared_ptr<LazySequence<T>>(this),
   startSkip, endSkip, parentSeq);
    }
```

```
public:
    int GetLength() const
    {
        return this->length;
    }
    bool IsEmpty()
    {
        return this->length == 0;
    }
public:
    virtual T Get(size_t index)
    {
        if (index >= this->items->GetLength())
        {
            for (auto i = this->items->GetLength() -
   1; i <= index; i++;)
            {
                this->items->Append(this->generator->
   GetNext());
            }
        }
        return this->items->Get(index);
    };
    virtual T& operator[](size_t index) = 0;
    virtual T GetFirst() const = 0;
    virtual T GetLast() const = 0;
    virtual Sequence<T>* GetFirst(size_t count) const
    = 0;
    virtual Sequence<T>* GetLast(size_t count) const
   = 0;
    virtual LazySequence<T>* GetSubsequence(size_t
   startIndex, size_t endIndex) const = 0;
    virtual LazySequence<T>* Append(LazySequence<T>*
   seq) = 0;
    virtual LazySequence<T>* Prepend(LazySequence<T>*
    seq) = 0;
    virtual LazySequence<T>* InsertAt(size_t index,
   LazySequence<T>*) = 0;
    virtual LazySequence<T>* SkipFirst(size_t count)
   const = 0;
    virtual LazySequence<T>* Skip(size_t start,
```

```
    size_t end)
     {
         return new LazySequence<T>(start, end,
    shared_ptr<LazySequence<T>>(this));
     }
     virtual LazySequence<T>* Skip(shared_ptr<
    LazySequence<size_t>> indexes) = 0;
public:
     ILazySequenceIterator<T>* GetIterator();
public:
     virtual ~LazySequence() = default;
private:
     template<typename T>
     class LazySequenceIterator :
    ILazySequenceIterator<T>;
private:
     class IGenerator
     {
     public:
         virtual T GetNext() = 0;
         virtual bool HasNext() const = 0;
         virtual Optional<T> TryGetNext() = 0;
     public:
         virtual shared_ptr<IGenerator> Append(
    shared_ptr<LazySequence<T>> seq) = 0;
         virtual shared_ptr<IGenerator> Skip(
    shared_ptr<LazySequence<T>> seq) = 0;
         virtual shared_ptr<IGenerator> Insert(
    shared_ptr<LazySequence<T>> seq) = 0;

     };

     class FunctionGenerator : public IGenerator;
     class SkipGenerator : IGenerator;
};
```

Listing 9.19 – Объявление класса LazySequence

Отметим, что обозначены некоторые варианты кострукто-ра, которые прежде не обсуждались: а именно, как передавать функцию, порождающую элементы по рекуррентной схеме.

Тут возможны два, не исключающих друг друга, подхода. Первый из них состоит в том, чтобы функция формально принимала список элементов. Это хорошее решение, позволяющее унифицировать интерфейсы, но требуется осторожность в использовании, т.к. нужно в конструктор согласовано передавать два параметра: собственно указательно на порождающую функцию, а также число, обозначающее количество предшествующих элементов, используемых в рекуррентной схеме. Другой вариант состоит в том, чтобы реализовать отдельный конструктор для каждого варианта арности этой рекуррентной схемы, вплоть до некоторого хоть и фиксированного, но разумно выбранного предела, например, 4 или 5.

Также отметим, что в этом варианте кода еще отсутствуют элементы, необходимые для полноценного использования ординалов в качестве индексов, а также кардинальность (GetLength) тут целое число, а не специально определенный тип, включающий как натуральные значения, так и (первое) трансфинитное число. В реальности, необходимы обе эти доработки. Намечены реализации двух операций – Skip и Insert, а также создание последовательности с помощью порождающей функции, для чего используется FunctionGenerator.

Отдельно приведем реализацию DefaultGenerator (листинг 9.20) и FunctionGenerator (листинг 9.21).

```
class FunctionGenerator : public IGenerator
{
private:
    weak_ptr<LazySequence<T>> owner;
    size_t generatorArity;
    T(*generator)(Sequence<T>*);
public:
    DefaultGenerator(shared_ptr<LazySequence> owner,
   size_t generatorArity, T(*generator)(Sequence<T>*)
   )
    {
        this->owner = owner;
        this->generator = generator;
```

```
    }
public:
    virtual T GetNext() override
    {
        Sequence<T>* prevItems = this->owner->items->
    GetLast(this->owner->generatorArity);
        return generator(prevItems);
    }
    virtual bool HasNext() const override;
    virtual Optional<T> TryGetNext() override;
public:
    virtual shared_ptr<IGenerator> Append(shared_ptr<
    LazySequence<T>> seq) override;
    virtual shared_ptr<IGenerator> Skip(shared_ptr<
    LazySequence<T>> seq) override;
    virtual shared_ptr<IGenerator> Insert(shared_ptr<
    LazySequence<T>> seq) override;
};
```

Listing 9.20 – Определение класса FunctionGenerator

```
class SkipGenerator : IGenerator
{
private:
    shared_ptr<LazySequence> owner;
    size_t startSkip;
    size_t endSkip;
    shared_ptr<LazySequence> parentSeq;
    ILazySequenceIterator<T>* last;
public:
    SkipGenerator(shared_ptr<LazySequence<T>> owner,
    size_t startSkip, size_t endSkip, shared_ptr<
    LazySequence<T>> parentSeq)
    {
        this->owner = owner;
        this->startSkip = startSkip;
        this->endSkip = endSkip;
        this->parentSeq = parentSeq;
        this->last = this->parentSeq->GetIterator();
    }
public:
```

```
    virtual T GetNext() override
    {
        if (this->startSkip <= this->last->
   GetCurrentIndex() && this->last->GetCurrentIndex()
    <= this->endSkip)
            for (auto i = this->startSkip; i < this->
   endSkip; i++)
                this->last->TryGetNext();
        return this->last->GetNext();
    }
    virtual bool HasNext() const override;
    virtual Optional<T> TryGetNext() override;
public:
    virtual shared_ptr<IGenerator> Append(shared_ptr<
   LazySequence<T>> seq) override;
    virtual shared_ptr<IGenerator> Skip(shared_ptr<
   LazySequence<T>> seq) override;
    virtual shared_ptr<IGenerator> Insert(shared_ptr<
   LazySequence<T>> seq) override;
};
```

Listing 9.21 – Определение класса SkipGenerator

Мы говорили о ленивых последовательностях как о потенциально бесконечных объектах; помимо чисто математического стремления к общности и стройности, есть и вполне прикладные, практические соображения, почему даже в конечных структурах может требоваться ленивость и почему требует именно потенциальная бесконечность.

Дело в том, что фактически данные могут размещаться в различных источниках и в различной форме: например, в файлах различной структуры, или удаленно в сети. Получение каждого очередного объекта бывает затратной процедурой, не все объекты оказываются востребованы, а бывает и так, что не всегда заранее известно, где заканчивается тот или иной массив данных. Например, это касается получения данных по сети, или при чтении данных из файла, в который параллельно происходит запись (скажем, только в конец файла).

Ленивые последовательности предоставляют инструмент, позволяющий абстрагироваться от многих таких деталей; в известном смысле, можно сказать, что это идеализированное решение по сравнению с хорошо известной концепцией – потоками, о которых речь пойдет далее.

9.4 Потоки (stream)

Структуры с последовательным доступом иногда рассматривают через призму т.н. потоков, в которых чтение, как правило, рассматривается, логически или физически, как удаление элемента из последовательности (хотя бывают и исключения). Потоки во многом похожи на описанные выше ленивые последовательности, однако отличаются тем, что для потоков не определена алгебра (нет таких операций, как конкатенация, например). Как у всякой линейной структуры, у потока есть начало (начальный или первый элемент) и конец (последний или конечный элемент). Потоки как правило конечны, но нередко ведут себя как (конечные) ленивые последовательности (например, получение данных через сокеты нередко рассматривается как чтение из потока). Потоки бывают:

– только для чтения (получение начального элемента, который сразу же удаляется);

– только для записи (добавление элемента в конец);

– смешанный вариант.

Если в ленивых списках элементы задаются, по сути, процедурно, то в потоках они берутся из тех или иных источников – последовательностей различного типа, хранящихся в памяти, из файлов, из других потоков (например, потоков ввода-вывода).

Ниже приводится примерная внешняя спецификация потока только для чтения.

Таблица 9.2 – Внешняя спецификация ReadOnlyStream

Создание объекта	

`ReadOnlyStream (Sequence<T>*);`
`ReadOnlyStream (LazySequence<T>*);`
`ReadOnlyStream (file, Deserializer<T>);`
`ReadOnlyStream (string, Deserializer<T>);`
`ReadOnlyStream (Stream);`
	Различные варианты создать поток: из какой-либо последовательности элементов, из файла или строки (причем необходимо указать также и способ десериализации объектов типа T из строки), из другого потока.

Декомпозиция	
`bool IsEndOfStream () const;`	Возвращает признак того, что достигнут конец потока и дальнейшее чтение невозможно.
`T Read();` Может выбрасывать исключения – EndOfStream (достигнут конец потока)	Считывает текущий элемент потока и переходит на следующую позицию.
`size_t GetPosition() const;`	Возвращает текущую позицию в потоке (т.е. по сути – сколько элементов было считано на данный момент).

`bool IsCanSeek() const;`	Возвращает, возможно ли перемещение вдоль по потоку без чтения. Это возможно, например, если данные расположены в каком-либо sequence, иногда, – если в файле; невозможно – например, если данные читаются из сокета.
`size_t Seek(size_t index);` Может выбрасывать исключения – EndOfStream (достигнут конец потока)	Пробует перейти на заданную позицию, если возможно. Если позиция по какой-либо причине недостижима, может выбросить исключение; иногда – переходит на ближайшую возможную позицию и возвращает соответствующий индекс. Можно рассматривать как аналог операции Skip на списках.
`bool IsCanGoBack() const;`	Возвращает, возможно ли вернуться в ранее прочитанную позицию. При чтении файлов, в зависимости от того, в каком режиме открыт файл, это бывает возможным. Операция почти всегда имеет линейную сложность.

Операции	
`void Open();` `void Close();`	После создания объекта могут требоваться дополнительные действия, прежде чем станет возможным непосредственно чтение данных. Эти действия инкапсулируются в операцию Open. Аналогично, когда работа с потоком завершена, могут требоваться финализирующие действия, например, освобождение дескрипторов файлов. Для этого используется метод Close.

Аналогично, приведем внешнюю спецификацию потока только для записи.

Таблица 9.3 – Внешняя спецификация ReadOnlyStream

Создание объекта	

`ReadOnlyStream (Sequence<T>*);`
`ReadOnlyStream (LazySequence<T>*);`
`ReadOnlyStream (file, Deserializer<T>);`
`ReadOnlyStream (string, Deserializer<T>);`
`ReadOnlyStream (Stream);`
	Различные варианты создать поток: из какой-либо последовательности элементов, из файла или строки (причем необходимо указать также и способ десериализации объектов типа T из строки), из другого потока.

Декомпозиция	
`bool IsEndOfStream () const;`	Возвращает признак того, что достигнут конец потока и дальнейшее чтение невозможно.
`T Read();` Может выбрасывать исключения – EndOfStream (достигнут конец потока)	Считывает текущий элемент потока и переходит на следующую позицию.
`size_t GetPosition() const ;`	Возвращает текущую позицию в потоке (т.е. по сути – сколько элементов было считано на данный момент).
`bool IsCanSeek() const;`	Возвращает, возможно ли перемещение вдоль по потоку без чтения. Это возможно, например, если данные расположены в каком-либо sequence, иногда, – если в файле; невозможно – например, если данные читаются из сокета.

`size_t Seek(size_t index);` Может выбрасывать исключения – EndOfStream (достигнут конец потока)	Пробует перейти на заданную позицию, если возможно. Если позиция по какой-либо причине недостижима, может выбросить исключение; иногда – переходит на ближайшую возможную позицию и возвращает соответствующий индекс. Можно рассматривать как аналог операции Skip на списках.
`bool IsCanGoBack() const;`	Возвращает, возможно ли вернуться в ранее прочитанную позицию. При чтении файлов, в зависимости от того, в каком режиме открыт файл, это бывает возможным. Операция почти всегда имеет линейную сложность.

Операции	
`void Open();` `void Close();`	После создания объекта могут требоваться дополнительные действия, прежде чем станет возможным непосредственно чтение данных. Эти действия инкапсулируются в операцию Open. Аналогично, когда работа с потоком завершена, могут требоваться финализирующие действия, например, освобождение дескрипторов файлов. Для этого используется метод Close.

На практике встречаются и потоки, совмещающие в себе возможности чтения и записи, в первую очередь, это используется при работае с файлами. Их возможности обычно являются некоторой комбинацией потоков только чтения и только для записи. С точки зрения использования, наиболее значимой значимой является возможность в некоторых случаях начать запись в произвольном месте потока, поверх любых данных, которые были до этого. При работе с файлами это может (и чаще всего так и происходит) испортить формат файла, сделать нечитаемыми, по крайней мере, часть данных. Если поток структурированный, т.е. отвечает за то, что объект записывается целиком, и от пользователя абстрагируются такие детали, как сколько физического места требуется для хранения того или иного объекта, то таких проблем, очевидно, не возникает, но и создание таких потоков сложное и зачастую не удается обеспечить надлежащий уровень производительности. Впрочем, это одна из наиболее частных дилем: выбор между безопасностью/на-

дежностью с одной стороны или быстродействием – с другой.

обсудить получение данных из потока, когда поток еще не завершен, но и очередного объекта в нем еще нет. Для этого возвращать аналог Option, только с 3мя состояниями: Some(data) | NotYet | Eof

9.5 Обработка событий и принятий решений

Прежде чем приступить к обсуждению вопроса, сделаем оговорку: с термином «принятие решений» связана целая обширная область исследований со своими методами, моделями и подходами. Хорошее введение в проблематику можно найти в [4; 5]. В данном разделе, да и в целом в данном пособии, мы используем этот термин в неформальном смысле, хотя определенные параллели и будут проводиться. Однако наша текущая задача – изучение идей, в т.ч. формальных, связанных с проектированием и реализацией программного обеспечения, а не с задачей создания конкретно систем принятия решений.

Рассмотрим такой умозрительный пример: имеется некоторый летательный аппарат, с множеством установленных на нем двигателей, обеспечивающих как вертикальную, так и горизонтальную тягу. В начальный момент времени известно положение аппарата (по крайней мере, относительно земли), вектор скорости и другие параметры, в т.ч. связанные с работой двигателей. Требуется создать управляющий компонент (автопилот), который обеспечит подходящий режим посадки (точка посадки, предположим, задана), см. схематическое изображение на рис.

Отметим, что мы рассматриваем задачу лишь с принципиальной точки зрения, абстрагируясь от многочисленных проблем, осложняющих создание и использование подобных систем на практике.

Движение аппарата в целом описывается известной формулой из кинематики, только движение тут неравноускоренное,

ch-lazy-list/pic-highfleet.png

Рисунок 9.4 – Схематическое изображение летательного аппарата и зона посадки

для чего требуется ввести дополнительный член:

$$\vec{s}(t+\Delta t) = \vec{s}(t) + \vec{(v)}(t)\Delta t + \frac{\vec{a}(t)\Delta t^2}{2} + \frac{\Delta\vec{a}\Delta t^3}{6}$$

Задача управляющего компонента состоит в том, чтобы так задать параметры работы двигателей, чтобы создаваемый ими суммарный вектор ускорения удерживал аппарат на желаемой траектории, включая коррекцию скорости снижения. В такой постановке, по сути, имеем классическую задачу кибернетики (см. рис. 9.5); в роли обратной связи выступают различные датчики – высоты, расстояния до других объектов, наклон корпуса и другие измеряемые параметры; в роли управляющего воздействия – команды или целевые (на данной итерации) значения параметров работы двигателей.

Посмотрим на это с необычной точки зрения. Во-первых, предположим, что управляющий компонент работает с некоторой (вероятно, очень высокой) частотой, с которой опрашивает датчики, получая их показания, обрабатывает и формирует выходное воздействие на непосредственно управляемые элементы. Во-вторых, будем считать, что и опрашиваемые датчики, и

ch-lazy-list/pic-cybernetics.png

Рисунок 9.5 – Классическая схема управления с обратной связью

управляющие воздействия представлены ленивымии последовательностями.

Необходимо отметить, что все это время, пока мы говорили о ленивых последовательностях, мы игнорировали один существенный фактор: в ряде случаев в момент, когда мы хотим обратиться к очередному элементу, *его еще может не существовать*. Это еще один фактор, способноствующий использованию ленивых структур. Такое может быть, например, если в момент обращения датчик, который является самостоятельным устройством со своим циклом работы, регламентом и частотой съема показаний – датчик мог еще не завершить очередный цикл и не сформировать значение, которое можно было бы передать системе управления. Это приводит к естесственной идее, что необходимо рассматривать всю систему как работающую в параллельном и асинхронном режиме; однако это предмет отдельного рассмотрения и обсуждение этого круга вопросов мы пока отложим.

Тогда пусть $p : T_1 \times \cdots \times T_n \to T'_1 \times \cdots \times T'_m$ – собственно функция, вычисляющая каким-либо образом на основании показаний датчиков $(a_1, \ldots, a_n) \in T_1 \times \cdots \times T_n$ результирующее воздействие:

$$(b_1, \ldots, b_m) = p(a_1, \ldots, a_n) \in T'_1 \times \cdots \times T'_m$$

Такой подход в целом хорошо согласуется с классическими методами управления и приниятия решений. Пусть далее $l_i = [a_i^{(1)};\ \ldots; a_i^{(p_i)};\ \ldots]$ – ленивая последовательность, содержащая показания i-го датчика (всего таких, очевидно, n штук, т.е. $i = \overline{1, n}$). В таком случае работу всей системы можно представить так:

$$unzip_m \circ ((x : T_1 \times \cdots \times T_n) \mapsto map(p, x)) \circ zip_n$$

Или, с учетом конвейеризации вычислений:

$$\begin{aligned}
&([a_1^{(1)}; \ldots; a_1^{(p_1)}; \ldots], \ldots, [a_n^{(1)}; \ldots; a_n^{(p_n)}; \ldots]) \\
&\gg zip_n \\
&\gg ((x : T_1 \times \cdots \times T_n) \mapsto map(p, x)) \\
&\gg unzip_m \\
&\mapsto ([b_1^{(1)}; \ldots; b_1^{(q_1)}; \ldots], \ldots, [b_m^{(1)}; \ldots; b_m^{(q_m)}; \ldots])
\end{aligned}$$

Эта схема соответствуте ситуации, когда для выработки подходящего управляющего воздействия достаточно знать лишь текущие значения показателей. На практие, впрочем, чаще требуется еще и некоторое количество предшествующих значений (например, чтобы определить относительную скорость движения). Мемоизация уже вычисленных значений в ленивых последовательностях в этом случае оказывается очень полезной, т.к. эти значения всегда «под рукой».

10. Древовидные структуры данных

10.1 Деревья и абстрактные синтаксические деревья (AST)

Дерево определяется рекурсивно как структура данных, которая может быть либо пустой, либо состоять из корня и одного или нескольких поддеревьев. Эти поддеревья, в свою очередь, могут быть пустыми или непустыми, что создает рекурсивное определение. Количество поддеревьев (арность узла) может быть фиксированным (например, бинарные деревья) или варьироваться в заданном диапазоне. Узлы, не имеющие дочерних элементов, называются листовыми (терминальными).

Абстрактное синтаксическое дерево (Abstract Syntax Tree, AST) — это представление структурной информации в коде, где каждый узел соответствует определенной конструкции. AST является важным инструментом для объяснения алгоритмов работы с формулами и используется в формальных языках и языках программирования. Формальные языки часто строятся индукцией по построению, что означает наличие:

- **Базиса индукции**: определяет атомарные элементы конструкции (например, пропозициональные переменные и константы в логике высказываний).

- **Шага индукции**: описывает, как из более простых конструкций построить более сложные (например, логические связки, такие как конъюнкция, дизъюнкция, импликация).

Добавить про нисходящий анализ и паттерн-матчинг

Процедура построения AST называется **нисходящим анализом**: нечто целое декомпозируется на составляющие,

каждая из которых анализируется отдельно. Инструментом для этого является **паттерн-матчинг** (сопоставление с образцом), при котором выражение сопоставляется с предопределенными шаблонами (например, $F_1 \wedge F_2$, $F_1 \vee F_2$, $F_1 \to F_2$).

AST может быть *однородным* (все узлы одного типа) или *неоднородным*. Формально, AST логики высказываний является неоднородным, так как листовые узлы (пропозициональные символы) и нелистовые узлы (логические связки) относятся к разным синтаксическим категориям. Арность узлов в AST, как правило, не фиксирована и определяется количеством операндов соответствующей языковой конструкции (например, в `switch-case` дереве может быть $N + 1$ потомок).

10.2 Неоднородные структуры и универсальные деревья

Для хранения элементов нескольких различных типов в однородной коллекции (например, ‘sequence‘) используется тип ‘Aid‘ (алгебраический тип данных). Если заранее известны все возможные типы элементов ($T_1, T_2, \ldots, T_k$), то коллекция объявляется как ‘sequence<Aid<$T_1, T_2, \ldots, T_k$»‘. При обработке таких коллекций используется **паттерн-матчинг** (например, с помощью ‘switch‘), который позволяет разбирать каждый случай по типу элемента и применять соответствующую логику. Таким образом, гарантируется, что будет отработана только одна ветка кода, соответствующая конкретному типу.

Неоднородность дерева может проявляться по-разному. Например, она может быть *ярусной* (элементы одного типа на одном уровне), но чаще всего это означает, что тип дочерних узлов строго фиксирован для каждого родительского узла. Один из подходов к построению таких деревьев — определение специфических типов для каждого узла с указанием типов его дочерних элементов:

```
class Node1 {
    Node2 child1, child2;
    Node3 child3;
```

```
};
class Node3 {
    Node3 child1;
    Node1 child2;
    Node4 child3;
};
```

Этот подход позволяет создавать гибкие и разнообразные структуры, приписывая каждому узлу свой тип. Такие структуры называются **иерархическими** и похожи на иерархические базы данных. Однако, основной недостаток этого подхода — невозможность написать общий алгоритм навигации по дереву, так как для каждого типа узла необходимо прописывать свои алгоритмы обхода.

Для решения этой проблемы и создания универсального алгоритма навигации используется **абстрактный базовый класс** (например, ‘TExpression‘):

```
class TExpression {
public:
    virtual list<TExpression*> getChildren() const =
    0;
    virtual bool isTerminal() const = 0;
    // ... другие чисто виртуальные методы ...
};
```

От этого базового класса наследуются конкретные типы узлов, например, ‘TConjunction‘ (для логических связок) и ‘TPropSymbol‘ (для пропозициональных символов). Каждый наследник реализует виртуальные методы, возвращая список своих дочерних элементов или определяя, является ли он терминальным. Это позволяет написать общий алгоритм обхода дерева, который работает с базовым типом ‘TExpression‘, а затем, при необходимости, приводить к более частным типам (даункастинг) с помощью ‘static_cast‘ и проверки типа (возможно, с использованием концептов C++).

10.3 Пути в деревьях, обходы и рекурсия

Путь в дереве или графе является абстрактным типом данных и представляет собой последовательность *дуг*, а не вершин. Это важно, поскольку в графах между двумя вершинами может быть несколько дуг, и пара вершин недостаточна для их однозначной идентификации. На множестве дуг определена операция *композиции*, но она не является тотальной: два пути можно скомбинировать только если конец первого совпадает с началом второго. Это отличает пути от обычных последовательностей, которые всегда можно конкатенировать.

В контексте деревьев путь является способом однозначной идентификации вершины. Также существует концепция **относительных путей**, хорошо иллюстрируемая файловой системой, которая по своей сути является деревом с абсолютными и относительными путями.

Большую роль играют **бинарные деревья** — структуры, у которых не более двух дочерних элементов. Применительно к древовидным структурам часто используется понятие **инварианта** — условия, которое обязательно выполняется для каждого узла дерева. Примеры инвариантов:

– **Для AST**: если узел листовой, он содержит пропозициональный символ; если нелистовой — логическую связку.

– **Для бинарного дерева поиска (BST)**: если левое поддерево не пусто, все ключи в нём не превосходят (или строго меньше) ключа корня; если правое поддерево не пусто, все ключи в нём больше (или больше либо равны) ключа корня. Предпочтительнее использовать строгие неравенства и собирать совпадающие ключи в один узел для упрощения алгоритмов.

– **Для бинарной кучи (Heap)**: для каждого узла ключи всех его поддеревьев либо меньше, либо больше ключа корня, в зависимости от типа кучи (min-heap или

max-heap).

Обход дерева — это правило, определяющее порядок посещения узлов дерева для выполнения над ними каких-либо операций (например, печать содержимого). Существуют несколько классических обходов, основанных на трех условных обозначениях: *Корень*, *Лево* и *Право*. Отсюда возникают различные варианты (например, прямой, обратный, симметричный). Возможно реализовать **универсальную процедуру обхода**, которая на вход принимает порядок обхода и желаемое действие над узлами. Процедуры обхода могут быть *рекурсивными* или *нерекурсивными* (реализуются с помощью стека для сохранения состояния).

Рекурсия традиционно изучается перед нерекурсивными алгоритмами, так как многие конструкции (например, заданные индукцией по построению) естественно описываются рекурсивно. Рекурсия является инструментом, который автоматически создает **замыкание** (среду) для вычисления, делая это управление неявным. Отказ от рекурсии (переход к итеративной реализации) означает ручное управление стеком и состоянием. Основная проблема рекурсии — ограниченный размер стека вызовов. В функциональных языках часто используется **хвостовая рекурсия**, которая оптимизируется компилятором, превращая рекурсивные вызовы в итеративные, что позволяет избежать переполнения стека.

10.4 Деревья поиска

Дерево поиска — это конструкция, которая упорядочивает элементы по определённому критерию. Для этого требуется некое множество ‘T‘, проекция (мономорфизм) этого множества в множество ‘K‘, оснащённое тотальным (линейным) порядком. Инъективное отображение гарантирует, что разным элементам ‘T‘ соответствуют разные элементы ‘K‘. Тотальный порядок означает, что все элементы попарно сравнимы.

Дерево поиска позволяет гарантировать, что случайный поток элементов будет отсортирован за счёт их размещения в структуре дерева. Фактически, это ”чёрный ящик”, в который добавляются элементы, и который позволяет за логарифмическое время (или за константное, если поддерживать указатели на крайние элементы) получить наименьший и наибольший элемент. Это свойство используется, например, для реализации отсортированной очереди или очереди с приоритетами.

Значение, сопоставляемое каждому элементу из ‘T‘ (элементу из ‘K‘), называется ключом. Благодаря инъективному отображению, по ключу можно найти соответствующий элемент в дереве поиска. Даже если с одним ключом связано несколько элементов, их можно получить все и работать с ними дальше.

Эта идея приводит к концепции словаря, который также называется ассоциативным массивом или коллекцией ”ключ-значение”. В литературе также встречается термин ”ассоциативная память”, который относится к аппаратной реализации этого принципа.

10.5 Обходы дерева и поиск

В абстрактном дереве, где определены только дочерне-родительские отношения, единственный способ понять, входит ли элемент в дерево, — это перебрать все узлы. Рекомендуется упорядоченный обход:

1. **Обход в ширину (BFS)**: дерево обходится по ярусам. Для реализации используется очередь.

2. **Обход в глубину (DFS)**: сначала доходим до конца по одному пути (например, по самой левой траектории), затем поднимаемся и идем вправо. Для реализации используется стек.

10.6 Бинарные деревья поиска (BST)

Бинарное дерево поиска — это дерево, для которого определен инвариант: в левом поддереве все ключи меньше ключа текущего узла, а в правом поддереве — больше. Предпочтительно использовать строгие неравенства.

Если возможно повторение ключей, словарь можно объявлять как `ключ → множество значений`. Однако это увеличивает накладные расходы. Существуют модификации алгоритмов вставки и поиска, обрабатывающие нестрогие отношения сравнения и редкие совпадения ключей.

Дерево, по сути, является формой словаря. Преимущество дерева — логарифмическая сложность операций вставки, удаления и поиска, что делает его хорошим выбором для часто меняющихся данных.

10.7 Улучшение работы деревьев поиска

10.7.1 Балансировка деревьев

Если элементы добавляются в дерево постоянно по возрастанию, дерево превращается в линейный список, что негативно влияет на асимптотику всех операций. Для этого используется балансировка.

Стратегии балансировки: АВЛ-дерево (AVL-tree), красно-чёрное дерево (Red-Black tree) и другие. Дерево называется сбалансированным, если высота его поддеревьев отличается не более чем на единицу.

При вставке или удалении элементов могут выполняться повороты (малый и большой, левый и правый) для восстановления баланса. Повороты изменяют структуру дерева, поднимая или опуская узлы для выравнивания высот поддеревьев. Отслеживание высоты каждого поддерева является линейной операцией, поэтому высота аккуратно пересчитывается при вставке и поворотах.

10.7.2 Прошитые деревья

Прошитые деревья — это способ обхода дерева нерекурсивным образом без использования стека. В стародавние времена это было важно из-за экономии памяти и алгоритмической нагрузки.

Реализация прошивки:

1. **Внутренняя прошивка**: в узел добавляется дополнительный указатель или признак, указывающий на следующий узел в заданном порядке обхода (например, самый левый узел в правом поддереве или вышестоящий родитель). Экономия относительна, так как все равно требуется хранить признак прошивки, а поддержка прошивки при вставке/удалении элементов сложна.

2. **Внешняя прошивка**: прошивка реализуется как внешняя структура (например, связанный список), отдельная от узлов дерева. Это позволяет не модифицировать дерево. Это продуктивный приём: дополнительные данные навешиваются вокруг основной структуры. Внешняя прошивка обеспечивает расширяемость, гибкость и не требует изменения существующего кода.

Любые структуры, оптимизированные для чтения (например, индексы в базах данных), плохо работают при частых обновлениях данных. Обновление индексов в базах данных может значительно ухудшить время для операций вставки и удаления.

10.8 Бинарные кучи (Binary Heaps)

Бинарная куча имеет инвариант: ключи дочерних узлов либо оба меньше, либо оба больше ключа корня, что определяется при создании кучи. Это позволяет иметь наименьший или наибольший элемент в вершине.

Бинарную кучу можно хранить как линейную структуру (массив), где дочерне-родительские отношения вычисляются за константное время по формулам. Вставка элемента происходит за логарифмическое время: элемент сначала помещается в конец, затем ”поднимается” до своего места.

Основная польза бинарной кучи — её использование для реализации очереди с приоритетами, где порядок определяется весом элементов.

11. Неупорядоченные структуры

11.1 Концепция словаря

Ассоциативные массивы, или словари, занимают особое место среди структур данных, поскольку предоставляют механизм доступа к информации, основанный не на позиции элемента в коллекции, а на его смысловой идентификации через ключ. В отличие от массивов и списков, где получение нужного элемента требует знания его индекса или последовательного обхода, словари позволяют обращаться к данным напрямую по значению атрибута, которое служит однозначным идентификатором.

В общем виде ассоциативный массив можно рассматривать как отображение между двумя множествами. Пусть у нас есть некоторое множество D – множество объектов, и множество K – множество ключей. Обозначение D происходит от слова домен, который сам по себе также является довольно интересной концепцией, но подробнее о нем будет рассказано в других курсах. В качестве короткого определения можно сказать, что домен — это область определения атрибутов отношений.

На этих двух множествах определено инъективное отображение $h : D \to K$. (инъективное отображение говорит о том, что различным элементам D соответсвуют разные элементы из K). Значит, мы можем выделить подмножество K', которое является образом D, где существует обратное отображение h^{-1}. Это обеспечивает следующее:

$$\forall k \in K : h^{-1}(k) = d(k), \text{ если } k \in K' \text{ иначе } \perp$$

То есть для произвольного элемента k из K существует единственный элемент из D, который ему соответствует (если k входит в K') или такого прообраза нет. Иллюстрация таких отображений приведена на рисунке.

./ch-decision-trees/pic-injective.png

Рисунок 11.1 – Пример отображений

Структуру данных, описываемую таким отображением в некоторых источниках называют map. В математике с этим словом чаще связывают другое значение. Это либо, буквально, отображение, либо соотнесение. Можно например, сказать, что отображение h^{-1} соотносит элементы K с элементами D или отображение h соотносит элементы D с элементами из K.

Таким образом, получается, что h^{-1} частичное отображение на множестве K (определено не на всех элементах рассматриваемого множества) и тотально на множестве K'.

Рассмотрим теперь немного другую ситуацию. У нас также имеются множества D и K, подмножество K', отображение $h : D \to K' \subseteq K$. Однако теперь h является сюръективным на K'. Разница заключается в том, что в случае инъективного отображения разным прообразам из D соответствуют обязательно разные образы из K. В случае сюръективного отображения важно то, что для каждого элемента из K' найдется прообраз из D. Исходя из этого, инъективное отображение иногда называют погружением множества D в K, а сюръективные на-

зывают сужением D на K'.

Из этой концепции возникли функциональных моделей данных. В отличие от реляционных моделей, где объекты рассматриваются через призму характеризующих их атрибутов, функциональный подход является как бы разворотом этой концепции в сторону того, что каждый объект считается неким черным ящиком и структура его неизвестна, а атрибуты сводятся к выполнению каких-либо функций. Все это приводит к идее, что атрибут (независимо от того, хранимый он или вычислимый) – это функция (инъективная или сюръектиная), определенная на множестве объектов данных и с областью значений на том типе, который приписывается непосредственно рассматриваемому атрибуту.

Большинство атрибутов являются сюръекциями на подмножество формальных типов. Под формальными типами здесь понимаются, например, строки, целые числа и т.д. Важно понимать, что чаще всего эти множества покрываются только частично: не каждая строка является фамилией и далеко не каждое число можно считать возрастом. Именно из-за этого рассматривается в первую очередь подмножество K', а не сразу всё множество K. Это приводит к тому, что в некоторых информационных системах создают специальные справочники (пополняемые), в которых перечисляются всевозможные значения, чтобы минимизировать разночтения. Например, перечислить всевозможные фамилии, имена и отчества. Также удобство такого подхода будет заключаться в том, что в примере с фамилиями можно дополнительно указать различные словоформы, которые потом будут использовать, к примеру, в генерации документов.

Типовая ситуация с такими функциями заключается в том, что часто разным исходным обектам сопоставляются одинаковые значения атрибутов. При этом нас все равно может интересовать вопрос, например, какие объекты обладают заданным значением какого-либо атрибута. То есть, мы хотим построить

соответствие из K' в D (а глобально и из K в D), но оно не будет являться функцией – для элемента $k \in K'$ не существует единственного элемента из D, который ему можно сопоставить, так как прообразов может быть несколько.

Математически это можно формализовать следующим образом:

$$\forall k \in K' \exists \{d_1, ...d_{n_k}\}, n_k \geq 1 : h^{-1}(k) = \{d_1, ...d_{n_k}\}$$

Одному элементу из K' сопоставляется некоторое множество (неупорядоченное) из D. Это означает, что в случае инъективного отображения h областью значений h^{-1} является D, а в случае сюръективного отображения областью значений h^{-1} является булеан множества D – $\beta(D)$ (булеан – множество, состоящее из всех подмножеств данного множества). Именно использование булеана позволяет оставаться в концепции функций.

В итоге для любой рассматриваемой ситуации по ключу возможно получать какой-то однозначно определяемый элемент, но это может быть или отдельный взятый элемент D, или множество таких элементов. Значит, концепция взаимнооднозначного соответствия всегда сохраняется.

Подводя итоги предыдущим рассуждениям получается то, что существуют ключи (какой угодно простой или сложной природы, так как нигде выше не оговаривалось, что ключи обязаны быть атомарными), которым можно соспоставить один или несколько элементов. Это и назвается словарём.

Любой словарь параметризуется двумя типами: типами ключей и типами элементов, который с ключами связывается. В случае инъективного отображения, элементы – это просто элементы D, в противном случае – это булеан от D. (по-хорошему, это должно быть именно множество, но иногда используются списки или последовательности). Формальным типом при этом является множество K, а не множество K', до которого мы сузились в некоторый момент. Поэтому необходимо обрабатывать ситуацию, когда не найдется

элементов из D, соответствующих данному ключу. Поэтому $h^{-1} : K \to$ `Option(D)` или $h^{-1} : K \to$ `Option`$<\beta(D)>$.

Стоит отметить, что даже если мы говорим о булеане D, то значениями всё равно будет `Option`, так как важно разграничивать случаи, когда ключу нельзя сопоставить элемент и когда по ключу не нашёлся элемент в данный момент времени. Например, если рассматривать ФИО Иванов Иван Иванович, но в данный момент времени ни одного Иванова Ивана Ивановича не нашлось, необходимо вернуть пустое множество в качестве значения по ключу. В то же время, если в качестве ФИО рассмотреть строку ABCD#DEJZ, то такой строке в принципе нельзя сопоставить ни одну персону, поэтому значением будет не пустое множество, а именно `option None` (или `Null` из баз данных).

11.2 Хеш-таблицы

Самым непосредственным средством реализации словаря является структура данных, известная как хеш-таблица, поскольку обеспечивает доступ к элементу по ключу за константное время. Это достигается тем, что (а) элементы, а точнее пары ключ-значение, располагаются в непрерывной области памяти, аналогично обычным массивам, и (б) примененяется специальное преобразование к ключам, что позволяет сразу вычислить положение элемента в этой непрерывной области памяти.

Пусть заданы два множества: K и D. Между ними определяется инъективное отображение $f : K \to D$. Именно это отображение и лежит в основе ассоциативных типов данных. При этом элементы множества K выступают в роли ключей, а элементы множества D — в роли значений.

На практике ключи и значения чаще всего представляют собой скалярные типы данных, однако строгих ограничений на их природу не существует.

Также возможна ситуация, когда одному ключу соответствует несколько значений. Например, пусть D – множество студентов, K – множество фамилий. В таком случае, фамилии «Иванов» могут соответствовать несколько студентов, обладающих этой фамилией.

Для корректного описания подобных случаев множество значений D можно рассматривать не как скаляр, а как коллекцию элементов — то есть как булеан исходного множества (булеаном множества называется множество, состоящее из всех его подмножеств).

11.3 Альтернативные варианты реализации словарей

Множество – это совокупность элементов, в которой порядок не специфицирован. С математической точки зрения существует четкое определение множества, однако в контексте программной инженерии отсутствует единственный корректный способ его реализации. Можно зафиксировать некоторый абстрактный тип данных множество:

```
template <typename T>
class ISet
{
public:
    virtual int GetCount() = 0;
    virtual bool IsEmpty();
    virtual bool HasElement(T element) = 0;
public:
    virtual ISet<T>* Union(ISet<T>* set) = 0;
    virtual ISet<T>* Intersect(ISet<T>* set) = 0;
    virtual ISet<T>* Except(ISet<T>* set) = 0;
    virtual ISetPair<T,T>>* Product(ISet<T>* set)
= 0;
}
```

Но уже даже для такой простой спецификации возникает довольно много вопросов.

Рассмотрим, например, метод `GetCount`. Множество

с математической точки зрения может быть как конечным, так и бесконечным, причем бесконечное множество может быть счетным или несчетным. Поэтому возникает вопрос: корректно ли определять метод, возвращающий количество элементов, и какое значение он должен возвращать в случае бесконечного множества? Этот вопрос тесно связан с представлением множества в памяти, поскольку бесконечное множество невозможно полностью разместить в оперативной памяти компьютера. Единственным подходом к представлению бесконечного множества в конечном объёме памяти является использование порождающих функций. Однако применение порождающих функций порождает новые проблемы — в частности, как реализовать операции объединения, пересечения, разности и декартового произведения.

Рассмотрим пример: множество чисел, кратных трём, и множество чисел, кратных двум. Имея только порождающие функции этих множеств, весьма нетривиально определить их объединение, поскольку числа, кратные шести, принадлежат обоим множествам. Таким образом, хотя представление множеств через порождающие функции выглядит теоретически привлекательно, оно приводит к значительным сложностям при реализации операций над множествами.

Следовательно, реализация бесконечных множеств на практике сопряжена с серьёзными трудностями. Рациональным решением является ограничение объекта реализации только на конечные множества. Однако даже в этом случае остаётся открытым вопрос: каким образом следует хранить элементы множества?

Теоретически можно предложить создать несколько наследников для различных реализаций множества, каждый из которых использует различную структуру данных: массивы, хеш-таблицы или списки. На этом уровне можно отчасти решить проблему бесконечных множеств, реализуя отдельные структуры, предназначенные для определённых бесконечных мно-

жеств.

Ранее обсуждалось, что структуры данных могут быть изменяемыми или неизменяемыми. Неизменяемые структуры обладают преимуществом предсказуемости и отсутствием побочных эффектов. Однако они вызывают повышенные затраты памяти и времени при выполнении операций модификации. Следовательно, нельзя однозначно утверждать, что неизменяемые структуры превосходят изменяемые или наоборот. Поэтому в контексте множеств целесообразно разработать и еще два варианта реализации: как изменяемую, так и неизменяемую версию соответствующего интерфейса.

11.4 Построение алфавитного указателя

Рассмотрим интересный пример, на котором не только проиллюстрируем применение концепции словаря, но и целый ряд других концепций, в т.ч. применение принципа декомпозиции и разделения ответственности, а также изучим еще один пример составления объектных моделей.

Постановка задачи такая. На вход подается: (1) строка, состоящая из слов, разделенных пробелами и другими символами-разделителями, например скобками, запятыми, и др.; (2) параметры «верстки» текста. Программа должна построить постраничное представление текста и для этого представления – построить алфавитный указатель. Постраничное представление строится по следующим правилам:

1. Исходную строку разбить на отдельные слова, используя пробелы в качестве разделителей. В результате получится список строк, в котором каждый элемент – это слово.

2. Список слов «режется» на фрагменты так, чтобы каждый фрагмент умещался на станице. Если размер страницы указан в словах, то длина каждого фрагмента должна быть не больше этой величины. Если размер страницы

задан в количестве символов, то следует вычислить суммарную длину слов выделенного фрагмента (положим, что всего фрагмент содержит N слов) и прибавить величину $N - 1$ – количество пробелов, которое необходимо, что записать эти слова раздельно.

3. 3 При делении списка слов на страницы следует применять следующее правило: первая страница может быть заполнена не более, чем на половину, а каждая десятая – не более, чем на 3/4 от заданного размера страницы (конкретные цифры являются параметрами алгоритма).

Конечный результат удобно представить в виде пары объектов: первый описывает «книгу» – так будет называет упомянутое постраничное представление – и собственно алфавитный указатель.

11.4.1 Объектная модель документа: страницы, строки, слова

При таком подходе на первый план выходит построение объектной модели документа. Это позволяет отойти от примитивного представления текста как сплошной последовательности символов и перейти к структурированному представлению, удобному для дальнейшей обработки и анализа.

Объектная модель документа предполагает иерархическое представление его компонентов:

– **Книга (Book):** Содержит список страниц и алфавитный указатель.

– **Страница (Page):** Содержит список строк. Параметры страницы могут включать количество символов в строке и количество строк на странице. Важно исключать истории, связанные с переносами и прочими нюансами вёрстки для упрощения модели.

– **Строка (Line):** Содержит список слов. Теоретически, строка также может хранить дополнительную информацию, например, словарь позиций слов.

– **Слово (Word):** Может быть представлено как простой строковый объект или как более сложный объект с метаданными, такими как информация о частоте или контексте появления в тексте. Выделение слова как отдельного объекта становится целесообразным при необходимости связывать с ним дополнительную информацию, помимо его текстового значения.

Выше перечислены понятия, идентифицирующие структурные элементы книги. Но, например, представление каждого слова отдельным объектом может привести к значительным затратам (на выделение и освобождение памяти, например, и выполнение некоторых других операций). Это означает, что мелкость декомпозиции, или степень гранулирования объектного представления, необходимо соотносить с ограничениями и требованиями задачи. Слишком мелкое разбиение ведет к избыточным затратам по памяти и объему вычислений, слишком крупное разбиение – ведет к нарушению принципа разделения ответственностей, что усложняет создание и поддержку объектов, управление их жизненным циклом и т.д. В нашем конкретном примере это означает, что во многих случаях выделение отдельно класса для представления понятия «слово» может оказаться избыточным, и тогда достаточно считать, что строка Line содержит список слов как структуру типа `Sequence<string>`.

11.4.2 Разделение ответственностей: лексер, пагинатор, сборщик

Подобно тому, как решение обсуждаемой задачи требует правильно построить объектную модель – достаточно детальную, чтобы обеспечить нужную гибкость и объем функционала, и достаточно компактную, чтобы не сделать

реализацию неуправляемо громоздкой и избыточной по затратам, в т.ч. и с точки зрения исполнения, – точно так же важно и правильно декомпозицировать алгоритмическую составляющую задачи.

В целом, в задаче пагинации слов можно выделить несколько подзадач:

1. выделение слов в сплошной строке;
2. группирование слов в строки-линии (line);
3. группирование линий в страницы;
4. сбор последовательности страниц в целое и построение собственно алфавитного укзателя.

На рис. 11.2 изображена примерная схема объектной модели, которая получается, если следовать описываемой логике. Можно заметить некоторые, которые не были еще упомянуты, например, вертикальные блоки и блоки, из которых состоят слова. Появление первых связано с тем соображением, абзац является логической единицей, которая может быть поделена на части из-за того, что не умещается на одной странице (а в особых случаях, и на двух). Именно эти блоки «привязываются» к страницам. Аналогичные соображения связаны и с выделением блоков, из которых состоит слово: если слово требуется перенести, оно оказывается состоящим из двух блоков (один остается в конце одной строки, другой переносится на следующую). О необходимости соотносить затраты, связанные с введением этих дополнительных единиц, с приобретаемой на этапе разработки и эксплуатации выгодой – уже говорилось.

Лексер (лексический анализатор) отвечает за преобразование входного потока символов в список слов. Его основные функции:

– Разбиение входной строки на слова на основе разделителей (пробелы, запятые, точки).

ch-dictionary/pic-alphabet-index.png

Рисунок 11.2 – Объектная модель книги и порождающей ее системы обработки текста

– Обработка множественных пробелов и табуляций.

– Игнорирование знаков препинания (опционально, в зависимости от требований).

В контексте типов, лексер преобразует ‘Stream<Char>‘ в ‘Stream<Word>‘.

Формирователь строк (LineRenderer) принимает список слов, сгенерированных лексером, и формирует из них строки с учетом заданных ограничений, например, максимальной длины строки. Он преобразует ‘Stream<Word>‘ в ‘Stream<Line>‘.

Пагинатор отвечает за распределение сформированных строк по страницам в соответствии с заданными правилами (например, количество строк на странице, правила для висячих строк, правила для абзацев). На вход он получает множество строк и возвращает набор сформированных страниц. Его тип: ‘Stream<Line> -> Stream<Page>‘.

Сборщик книги (BookRenderer) является финальным этапом, который принимает список страниц и формирует из них объект книги, включая создание алфавитного указателя

и оглавления, если это предусмотрено. Это абстракция опциональна и может быть вызвана наличием различных правил формирования глав или других структурных элементов книги.

Все эти компоненты, хотя и могут быть реализованы как функции, в контексте ООП предпочтительнее представлять как отдельные классы с конструкторами, принимающими параметры для настройки их поведения (например, длина строки для LineRenderer, правила для Paginator).

11.4.3 Эффективные Структуры Данных: Словари и Хеш-Таблицы

Для реализации таких компонентов, как алфавитный указатель, где требуется быстрое нахождение элементов по ключу, необходимо использовать подходящие структуры данных. Обсуждаются следующие варианты:

Одним из простых, но не всегда эффективных способов реализации словаря является использование **отсортированной последовательности** (Sorted Sequence) пар ”ключ-значение”. Ключи сортируются, что позволяет использовать бинарный поиск для нахождения элементов за логарифмическое время $O(\log N)$. Однако вставка элементов в среднем будет занимать $O(N)$ времени.

Для достижения логарифмической сложности как для поиска, так и для вставки элементов используются **деревья поиска** (например, бинарные деревья поиска, B-деревья). Это более эффективное решение, чем Sorted Sequence, особенно для часто изменяющихся коллекций.

Хеш-таблицы обеспечивают константную амортизированную скорость поиска $O(1)$ и околоконстантную скорость вставки. Однако они имеют свои особенности:

– **Коллизии:** По мере наполнения таблицы возрастает вероятность коллизий (разные ключи дают один и тот же хеш-адрес), что приводит к росту цепочек элементов, связанных с одним адресом.

– **Перестроение таблицы:** Для поддержания эффективности хеш-таблицы требуется периодически ее перестраивать (увеличивать размер и перехешировать все элементы), когда заполненность достигает определенного порога (например, 70-80 процентов) или средняя длина цепочек становится слишком большой. Перестроение является дорогостоящей операцией, поскольку все элементы приходится перераспределять в новую таблицу.

– **Использование в базах данных:** В базах данных для индексации часто используются деревья (например, B-деревья), а не хеш-таблицы, потому что деревья лучше приспособлены к изменениям и обеспечивают более предсказуемую производительность при вставке и удалении данных.

11.5 Кеширование и индексирование данных

Индекс – специальная структура (типа ассоциативная память), ускоряющая поиск информационных объектов по ключу (значению атрибутов). Для построения индекса необходимо заранее зафиксировать структуру данных и перечень атрибутов, по которым допускается выполнять поиск. Это, в частности, означает, что над индексируемой коллекцией недопустимо произвольное выполнение операций сортировки или перестройки элементов, поскольку индекс должен оставаться согласованным с исходными данными.

Рассмотрим, что объекты обладают набором атрибутов различного типа. Например:

```
class Person {
    public:
      string GetFirstName();
      string GetMiddleName();
      string GetLastName();
      string GetFulltName();
```

```
        string GetFIO(); // Фамилия ИО..
        int GetBirthYear();
        int GetAge(int year); // вычислить возраст
на заданный момент
 };
```

В данном случае список атрибутов фиксированный, однако при практической реализации системы нельзя опираться на такой список. Требуется механизм, позволяющий получить перечень свойств произвольного объекта в процессе выполнения программы. Ранее подобный функционал обеспечивался посредством механизма RTTI (Run-Time Type Identification), который затем эволюционировал в более развитую концепцию — рефлексию.

Исходная коллекция объектов может храниться где угодно: в массиве, списке, файле или иной структуре с произвольным доступом. Однако элементы коллекции располагаются в случайном порядке, и поиск элемента без дополнительной структуры имеет трудоёмкость $O(n)$. Такая сложность допустима, если коллекция невелика или операции поиска выполняются редко. Однако в системах, где поиск по атрибутам — основная операция, линейный обход является недопустимо медленным.

Решение такого поиска можно реализовать, например, через следующий подход. В начале коллекцию необходимо отсортировать по выбранному атрибуту, а далее использовать бинарный поиск. Однако этот подход имеет ряд существенных недостатков. Во-первых, уже первичная сортировка больших наборов данных может быть затратной. Во-вторых, коллекция динамична: элементы регулярно добавляются или удаляются. Вставка в отсортированную структуру требует сдвига значительной части элементов, как и удаление. В-третьих, в результате чего элемент перестаёт находиться на корректной позиции, и требуется переместить его в другое место – операция нетривиальная и дорогостоящая. Более того, поиск может выполняться по разным атрибутам, поэтому заранее выбрать единственный

атрибут сортировки невозможно.

Исходя из этого, решение основано на построении дерева поиска по выбранному атрибуту. Рассмотрим случай, когда выбранный атрибут (или множество атрибутов) однозначно идентифицируют объект. Например, методы `GetFullName` или тройка `GetFirstName, GetMiddleName, GetLastName` образуют ключ, который однозначно определяет объект в коллекции. Тогда в узлах дерева хранятся пары вида (ключ, ссылка на объект).

Если же атрибут не является ключевым (например, год рождения или только имя), то множество объектов может иметь одинаковое значение. В этом случае в дереве поиска хранятся не одиночные элементы, а списки объектов с одинаковым ключом. Таким образом, индексирование делится на 2 основных типа: по ключевым атрибутам (результат поиска единственный) и по неключевым атрибутам (результатом является множество объектов).

Другая вариация задачи на идексирование заключается в индексировании по составным атрибутам. Если ключ состоит из нескольких значений, которые невозможно упорядочить строго линейно, возникает необходимость построения более сложной структуры — ориентированного ациклического графа. Поиск в подобных структурах сложнее, поскольку приходится рассматривать комбинации значений нескольких атрибутов.

Кэширование преследует ту же цель, что и индексирование, — ускорение доступа к данным. Однако кэш работает иначе.

Кэш изначально пуст. По мере обращений к данным он заполняется объектами, которые были запрошены. Если объём кэша ограничен и он переполнен, то удаляются элементы, к которым длительное время не было обращений. После большого числа операций в кэше остаются преимущественно те данные, которые используются чаще всего. При изменении частоты обращений состав кэша также меняется.

Кэширование особенно эффективно, когда получение данных напрямую является дорогостоящей операцией (например, данные хранятся на диске или во внешней системе). При наличии небольшого подмножества часто используемых объектов целесообразно держать их в оперативной памяти, обеспечивая быстрый доступ.

Различия между индексацией и кэшированием заключается в том, что индекс хорошо использовать, если имеется коллекция объектов. Кэширование же в первую очередь полезно, если операций поиска производится много, а также есть различия в частоте обращения к объектам.

Дополнительно стоит отметить, что размер кэша при этом мера весьма условная и может меняться в зависимости от условий. Иногда фиксируется объем памяти или число объектов, а иногда явных ограничений нет. В последнем имеется в виду, что ограничение может задаваться в формате обязательного периодического очищения кэша на определенный размер.

Как уже было сказано выше, элементы из кэша удаляются в зависимости от частоты их использования. Поэтому для принятия решения об удалении вместе с объектами хранится дополнительная информация: время последнего обращения, общее количество обращений и их частота.

Следует учитывать, что характер использования данных часто носит неравномерный характер: могут быть периоды интенсивного обращения к объекту и периоды полного простоя. Аналогично, если пользователь обращается к одному объекту, высока вероятность, что в ближайшее время ему понадобятся соседние данные. Поэтому современные кэши используют стратегию хранения не отдельных элементов, а блоков (страниц).

12. Анализ ситуаций с помощью древовидных структур

12.1 B-деревья

B-деревья (B от англ. *balanced*) представляют собой разновидность сбалансированных деревьев поиска, ориентированных прежде всего на локальную балансировку структуры. В отличие от бинарных деревьев, в которых глубины листьев могут различаться (по крайней мере на единицу), в B-дереве путь от корня до любого листа всегда имеет одинаковую длину, фиксированную для данного дерева.

Исторически необходимость появления таких структур была обусловлена особенностями организации данных на внешних носителях.

./ch-decision-trees/pic-disk.png

Рисунок 12.1 – Типичный дисковый носитель [6]

При работе с магнитными дисками приходилось учитывать конструкцию накопителей: наличие нескольких блинов,

отдельных считывающих головок и различие в доступе к дорожкам. Поскольку дорожки, расположенные на одинаковом расстоянии от оси вращения, образуют цилиндр, существовало практическое преимущество в том, чтобы данные одного поддерева полностью находились на одном цилиндре. Тогда за одно вращение диска считывался весь соответствующий объем данных, что значительно ускоряло операции доступа. В связи с этим возникла задача организовать данные так, чтобы они максимально эффективно распределялись по дискам и цилиндрам, что и привело к появлению описываемой структуры.

Типичное B-дерево выглядит следующим образом:

```
./ch-decision-trees/pic-b-tree.png
```

Рисунок 12.2 – B-дерево с согласными английского алфавита в качестве ключей [6]

B-дерево характеризуется параметром n, задающим минимальную степень узла. Каждый узел содержит от n до $2n - 1$ ключей и, соответственно, от $n + 1$ до $2n$ поддеревьев. Ключи в узле хранятся в отсортированном порядке, а поддеревья между последовательными ключами упорядочены таким образом, что любое поддерево между двумя соседними ключами содержит значения, строго большие левого ключа и строго меньшие правого.

Узел B-дерева хранит сами ключи, связанные с ними объекты данных, а также указатели на поддеревья. С неформальной точки зрения узел можно считать словарем, так как помимо

ключей в узле содержатся объекты данных (возможно не для всех ключей).

Поиск ключа в узле сводится к бинарному поиску по отсортированному набору ключей. Если ключ найден, возвращается соответствующий объект; если нет — поиск продолжается в выбранном поддереве. Благодаря фиксированной высоте дерева и ограниченному числу ключей в узлах, время поиска всегда логарифмическое.

Существует также модификация структуры, называемая B+-деревом. В ней ключи в внутренних узлах служат исключительно для формирование древовидной структуры, а полезная нагрузка — объекты данных — хранится только в листьях. Такая организация упрощает последовательный обход элементов и делает структуру особенно удобной для применения в файловых системах и базах данных.

12.2 Задача о рюкзаке

Задача о рюкзаке представляет собой классическую задачу дискретной оптимизации, в которой необходимо подобрать набор предметов с максимальной стоимостью при наличии ограничений на доступные ресурсы.

Пусть имеется двумерный рюкзак и набор предметов, пример которых показан на рисунке. Каждый предмет характеризуется рядом параметров, включая вес, объем и стоимость, и занимает определенное место в рюкзаке.

В зависимости от условий оптимизации можно выделить несколько вариантов постановки задачи:

1. Вариант A: суммарная стоимость предметов в рюкзаке максимальна, а суммарный объем не превосходит заданной величины. Считается, что форма предметов игнорируется;

2. Вариант B: суммарная стоимость предметов в рюкзаке

./ch-decision-trees/pic-backpack.png

Рисунок 12.3 – Пример формы рюкзака и предметов

максимальна, а объем и вес не превосходят заданных величин.

3. Вариант C: суммарная стоимость предметов максимальна, но ограничена грузоподъемность (суммарный вес не должен превосходить заданной величины) и следует заполнить рюкзак как можно более эффективно ($\Delta V = 0$ или $\Delta V \leq \epsilon$, где ΔV – объем рюкзака, который сотался незаполненным)

4. Вариант D: суммарная стоимость предметов максимальна, заполнение рюкзака должно быть максимальным (при ограничении вместимости), но ограничена грузоподъемность (суммарный вес не должен превосходить заданной величины).

5. Вариант E: суммарная стоимость предметов максимальна, а суммарный вес – минимальный (при ограниченной вместимости)

В большинстве практических случаев форма предметов игнорируется, однако существуют модификации задачи, где необходимо учитывать их геометрическое расположение. Учет формы существенно усложняет задачу, так как приводит к экспо-

ненциальному росту количества возможных конфигураций размещения предметов.

Формальная постановка задачи:

Задано множество предметов $\{s_1, ..., s_m\}$ (обязательно дискретное). Предметы характеризуются тремя параметрами: вес (W), стоимость (V), объем (H).

Существует ограничение на объем рюкзака: $\sum H(s_i) \leq h_0$.

Цель:

– $max(\sum V(s_i))$

– $min(\sum W(s_i))$

Решение задачи – это последовательность, в которой предметы помещаются в рюкзак, и состояние рюкзака.

Допустимое решение – это решение, удовлетворяющее всем ограничениям задачи.

Идея решения данной задачи заключается в построении дерева решений, в котором каждая вершина соответствует возможному состоянию рюкзака.

Начальным состоянием в нашей задаче является состояние пустого рюкзака.

Далее при добавлении каждого предмета формируется новое состояние, отличающееся набором предметов, текущим весом, объемом и суммарной стоимостью.

Из каждого состояния возможен переход к множеству новых состояний, соответствующих выбору следующего предмета.

Если новое состояние нарушает ограничения задачи (например, превышает допустимый объем), оно считается недопустимым и исключается из дальнейшего рассмотрения.

Таким образом, получается дерево, которое показывает каким образом выбранное состояние получено из начального.

Если учитывать форму предметов, то добавление одного элемента может порождать не одно, а целое множество новых

состояний, поскольку предмет можно разместить различными способами. Кроме того, если порядок размещения предметов не фиксирован, число возможных комбинаций возрастает многократно.

В этом случае дерево решений значительно разрастается в ширину, что делает невозможным полный перебор всех состояний из-за ограничений по памяти и времени вычислений. Поэтому на практике часто используется обход в глубину, который позволяет быстрее находить допустимое (пусть даже не оптимальное) решение. Дальнейшее улучшение может достигаться с помощью методов динамического программирования или жадных алгоритмов.

13. Графовые структуры данных

Графы являются одной из важнейших структур данных в информатике. Часто им не уделяется должного внимания, однако их роль простирается далеко за пределы решения классических задач, таких как раскраска или поиск компонент сильной связности. На практике графы широко используются для визуализации и моделирования систем, где ключевым является представление связей между объектами.

Вершины графа обычно представляют собой объекты, а дуги (или рёбра) — связи между ними. Природа этих объектов может быть самой разнообразной: от пунктов на карте в транспортных системах до компонентов в сложных инженерных проектах.

13.1 Объектная модель графа

Каноническим способом представления графа, преимущественно применяемым в учебниках и в целом при изложении алгоритмов на графах, является матрица смежности или матрица инциденций. В такой матрице наличие или отсутствие дуги между вершинами i и j отмечается ненулевым элементом: если граф невзвешенный, то это обычно просто число 1, а если взвешенный – то собственно вес дуги.

Однако на практике матричное представление, по крайней мере, без специальных ухищрений, оказывается крайне неэффективным. Основные причины этого:

– **Разреженность графов.** Большинство реальных графов являются разреженными, то есть содержат мало дуг по сравнению с максимально возможным их числом. В этом случае матрица смежности будет заполнена преимущественно нулями, что ведёт к нерациональному исполь-

зованию памяти. Например, если рассматривать графы, представляющие транспортные системы (т.е. системы, в которых вершины представляют какие-то пункты, а дуги – пути между этими пунктами), можно обнаружить, что обычно у каждой вершины в среднем около 3-4 дуг[1]. Например, уже при всего лишь 1000 вершин (а в реальности графы могут включать порядка 10^5 вершин, а иногда и более) количество дуг будет составлять доли процента от максимально возможного в случае полного графа, т.е хранение порядка 99% информации нецелесообразно. Использование матриц оправдано только для полных или почти полных графов, что встречается редко (например, в некоторых коммутационных системах).

– **Множественность атрибутов.** В практических задачах дуги и вершины часто обладают множеством атрибутов (например, в транспортной задаче: длина пути, стоимость, время, тип транспорта). Хранение таких наборов атрибутов в матричной структуре проблематично. Теоретически, можно создать отдельную матрицу для каждого атрибута, но это усложняет загрузку данных и управление ими, особенно при работе с базами данных (а это, пожалуй, наиболее частый случай).

Из-за этих недостатков на практике чаще применяются структуры, оптимизированные для разреженных графов. На уровне описания алгоритмов матричное представление делает изложение проще и нагляднее и позволяет абстрагироваться от деталей реализации; но на практике чаще предпочтительные другие решения.

[1] Для целей данного рассмотрения не очень важно, идет ли речь об ориентированном графе или неориентированном, а также достатоно рассматривать, в случае ориентированного графа, только исходящие дуги. Даже при таком грубом подсчете порядок величин будет верный.

В первую очередь, необходимо отметить, что приведенная критика матричного представления, в первую очередь, касается «физического» представления матрицами, а точнее – обычными двухмерными массивами. Мы уже обсуждали, что за логическим определением объектов (по иной терминологией – за их внешней спецификацией) стоит какая-то реализация, от которой пользователь абстрагирован. Так, раз мы установили, что графы чаще всего более или менее разрежены, целесообразным будет использовать какую-либо реализацию словаря, и лучше всего будет работать хеш-таблица, в которой ключом будет пара – упорядоченная в случае ориентированного графа и неупорядоченная в случае неориентированного графа.

Кроме того, матричное представление требует, чтобы вершины были пронумерованы сплошной последовательностью чисел от 0 до $N - 1$. Однако в реальных системах объекты (вершины), как правило, поступают из баз данных и имеют собственные идентификаторы (числовые, строковые), которые не образуют такой последовательности. Это создаёт необходимость введения искусственной нумерации, что усложняет реализацию. Требуется дополнительный механизм для сопоставления реальных идентификаторов и внутренних индексов. Стоит отметить, что ситуативно матричное представление все же может быть эффективным, если решить перечисленные проблемы.

Однако тут возникает другая сложность: многие алгоритмы оперируют таким понятием, как окрестность графа (а точнее, множество входящих или исходящих дуг); из матриц такую информацию сложно доставать эффективно. Это одно из соображений, подталкивающих к другому решению, о котором речь пойдет далее.

Дуги и вершины – это объекты данных, а значит ключевым, или, по крайней мере, важным является эффективное представление и оперирование их атрибутами. Рассмотрим пример транспортной системы. Дуга, представляющая маршрут,

может характеризоваться не только расстоянием, но и другими параметрами:

– **Стоимость:** Расход топлива, платные дороги.

– **Время в пути:** Зависит от типа дороги и транспорта.

– **Предпочтения:** Комфорт, личные предпочтения (боязнь самолётов), требования к перевозке грузов.

Когда у дуги множество атрибутов, центральным объектом данных становится сама дуга, а не матрица. Представление графа в виде списка дуг, где каждая дуга является сложной структурой, становится более предпочтительным.

Иногда не только дуги, но и вершины обладают важными атрибутами. Например, географические координаты вершин могут использоваться для эвристического ускорения алгоритмов поиска кратчайшего пути (например, в алгоритме A^*).

В наиболее сложных случаях отдельный атрибут превращается в сложную структуру, значение которой зависит от некоторого конктекста, например, момента времени. Пример – предполагаемая средняя скорость в зависимости от времени суток (определяемая статистически по имеющимся наблюдениям). Другой пример – в вершинах происходят процессы, имеющие свою стоимость. Рассмотрим транспортный узел, где возможна смена вида транспорта (например, с автомобиля на поезд). Такая перегрузка имеет свою стоимость (время, деньги).

Стандартные алгоритмы поиска пути не учитывают стоимость ”прохождения” вершины. Чтобы решить эту проблему, можно применить **трансформацию графа**:

– Одна вершина (транспортный узел) заменяется на целый подграф.

- В этом подграфе вершины представляют собой состояния (например, «разгрузка автомобиля», «погрузка на поезд»), а дуги — процессы перехода между состояниями.
- Веса этих новых дуг соответствуют стоимости перегрузки.

Такой подход позволяет использовать стандартные алгоритмы на изменённом графе, не усложняя их внутреннюю логику. Хотя количество вершин и дуг увеличивается, алгоритм остаётся однородным и простым для отладки и расширения.

Таким образом, мы выделили три основные сущности, необходимые для описания графа: собственно, граф, как целое, вершины графа, и дуги. Пока что отложим обсуждение того, что графы могут быть как ориентированные, так и неориентированные. Обычно, этот выбор фиксируется исходя из конкретного набора задач и выполняется подходящая специализированная реализация. Схематически, она выглядит так.

- **Узел (Node):**
  - Идентификатор; тип идентификатора в конкретном случае обычно фиксирован.
  - Полезная нагрузка (payload): данные, ассоциированные с узлом (координаты, матрица переходов и т.д.). Этот перечень обычно известен заранее и фиксируется на этапе написания программного кода.
  - Коллекция исходящих дуг (иногда и входящих). Обычно это коллекция указателей, в той или иной форме.
- **Дуга (Arc):**
  - Указатели на начальный и конечный узлы. Указатели – как в буквальном смысле, так и фигуральном:

например, в роли указателей иногда могут выступать идентификаторы узлов; выбор зависит от конкретной ситуации, однако «в среднем» можно считать использование фактического указателя или ссылочного типа предпочтительнее, т.к. не требует дополнительных операций для обращения к вершине. Использование идентификаторов требует обращения к словарю, а это – пусть и константная операция, но все же дополнительные накладные расходы, причем потенциально ощутимые, т.к. обращение к узлам происходит более или менее постоянно.

  - Полезная нагрузка (payload): данные, ассоциированные с дугой (вес, стоимость, тип и т.д.). Комментарии тут те же, что и в случае с атрибутами узлов: их перечень обычно фиксирован.
  - Может иметь собственный ID, но это не обязательно. В основном это имеет смысл, чтобы избежать различных сложностей, связанных с использованием составных ключей (как смысле словарей, так и в смысле баз данных).

- **Граф (Graph):**
  - Коллекция всех узлов графа. Скорее всего, не просто коллекция, а словарь или несколько словарей: у объектов, в частности, узлов, может быть несколько идентификаторов, а крмое того, иногда желательно иметь возможность быстро получить коллекцию узлов, обладающих определенным значением того или иного атрибута.
  - Коллекция всех дуг (часто отсутствует, чтобы избежать дублирования, так как дуги доступны через узлы).

– Методы для работы с графом, реализующие различные алгоритмы (поиск пути, обход и т.д.).

Примерные реализации приводятся на листингах ниже. Для узла (листинг 13.1) предусмотрены атрибуты – координаты (x, y) на гепотетической карте.

```
class TransportNetworkNode
{
private:
    int id;
    float x;
    float y;
    TransportNetworkNodeType nodeType;
    Sequence<TransportNetworkEdge*>* edges;
public:
    int GetId() const
    {
        return this->id;
    }
    float GetX() const
    {
        return this->x;
    }
    float GetY() const
    {
        return this->y;
    }
    virtual Sequence<TransportNetworkEdge*>*
   GetNeighbours() const
    {
        return this->edges;
    }
    TransportNetworkNodeType GetNodeType() const
    {
        return nodeType;
    }
public:
    bool operator==(TransportNetworkNode* node) {
        return this->id == node->id;
    }
```

```
};
```

Listing 13.1 – Примерная реализация АТД «Узел графа»

Из существенного стоит отметить неизменяемый характер этого объекта, так же как и структуры, описывающей дугу (листинг 13.2). Это обусловлено тем, что алгоритмы, даже такие, как раскраска или поиск остова или другие, требующие использования меток, не должны менять исходных данных. Любые метки, будь то используемые как промежуточные, рабочие данные, или как формируемые как результат работы алгоритма, должны быть внешними по отношению к исходным данным.

Это требование продикторвано как применением концепции разделения ответственностей, так и соображениями о потенциальной возможности использования одних и тех же исходных данных для решения серии однотипных задач, или даже набора различных, в т.ч. и в параллельном формате.

```
class TransportNetworkEdge
{
private:
    TransportNetworkNode* start;
    TransportNetworkNode* end;
    float length;
    float maxSpeed;
    float basePrice;
    int comfortLevel;
    TransportNetworkRoadType roadType;
public:
    TransportNetworkNode* GetStart() const
    {
        return this->start;
    }
    TransportNetworkNode* GetEnd() const
    {
        return this->end;
    }
    TransportNetworkRoadType GetRoadType() const
    {
        return this->roadType;
```

```
    }
    float GetLength() const
    {
        return this->length;
    }
    float GetMaxSpeed() const
    {
        return this->maxSpeed;
    }
    float GetBasePrice() const
    {
        return this->basePrice;
    }
    int GetComfortLevel() const
    {
        return this->comfortLevel;
    }
    bool IsIncidentTo(TransportNetworkEdge* edge)
   const
    {
        return this->end == edge->start ||
            edge->end == this->start;
    }
};
```

Listing 13.2 – Примерная реализация АТД «Дуга графа»

Структуры, описывающие типы вершин и дуг[2], могут быть определены различным образом, в т.ч и в виде явного перечисления, зафиксированного в программном коде (это зависит от ситуации). Диапазон возможнных вариантов – от простых перечислений (`enum`) до применения идиомы объект-как-глобальная-константа.

Нужно отметить немаловажное обстоятельство: описанный подход, в первую очередь, исходит из конктекста разработки в рамках создания некоторой информационной системы, когда объекты (вершины, дуги/ребра) оснащаются

[2] `TransportNetworkNodeType` и `TransportNetworkRoadType`, соответственно

набором атрибутов, зачастую большим (скажем, в больших транспортных системах количество атрибутов у дуг может исчисляться несколькими десятками, хотя обычно лишь часть из них использует в каждой конкретной задаче). И что еще важнее, данные о дугах и вершинах размещаются обычнов какой-либо базе данных, нередко декомпозировано (например, в соответствии с принципами нормализации в реляционной модели, но возможны и другие ситуации) – а не в структурированных файлах, когда информация о каждом объекте обычно локализована и считывание некоторого целостного, относительно небольшого фрагмента позволяет получить относительную полную информацию об объекте (или даже сразу нескольких объектах)[3].

В других случаях, например, в некоторых коммутационных системах, когда с дугой связывается обычно один числовой атрибут а сам граф связей между узлами либо близок к полному, либо не слишком большой, – может быть оправданным даже использование двухмерных массивов весов. Но это не отменяет общего принципа абстрагирования: какая бы форма реализации ни оказалась удобной, она должна быть прозрачна для пользователей; и даже если используются двухмерные массивы, описанная объектная модель все еще может быть использована для внешнего представления для пользователей.

13.2 Обходы графа: в глубину и в ширину

Большинство задач на графах так или иначе сводятся к процедуре обхода – систематического посещения вершин и дуг. Основных стратегий обхода две.

При обходе в ширину алгоритм сначала посещает все вер-

[3]Сравнительная характеристика различных способов хранения информации о (графовых) структурах выходит за рамки данного обсуждения и обычно рассматривается в продвинутых курсах по базам данных и информационным системам.

шины, смежные с текущей (соседние), и только после этого переходит к вершинам следующего «уровня» удалённости.

– Для реализации используется структура данных **очередь (Queue)**. При посещении очередной вершины, «раскрывается» исходящая окрестность этой вершины, и дуги помещаются в очередь. Это приводит к тому, что эти дуги будут обработаны лишь после посещения дуг, соседних с данной по исходящей окрестности.

– Гарантирует нахождение кратчайшего пути в невзвешенном графе.

– Является более надёжным вариантом, когда требуется найти оптимальное решение путём полного перебора.

При обходе в глубину алгоритм старается уйти как можно дальше от начальной вершины по одному пути, и только дойдя до тупика, возвращается назад и исследует альтернативные ветви.

– Для реализации используется структура данных **стек (Stack)**, в который помещаются дуги из исходящей окрестности очередного посещенного узла.

– Часто применяется для очень больших графов, когда полный обход невозможен, и целью является быстрое нахождение любого допустимого решения.

– Позволяет быстрее дойти до удалённых вершин, что может быть выгодно, если известно, что решения находятся далеко от старта.

Обе процедуры обхода (и многие другие, включая алгоритм Дейкстры) можно обобщить. Разница между ними заключается лишь в *порядке выбора* следующей дуги для посещения. Т.е.

мы идентифицировали новую ответственность, а значит, можно (и нужно) выделить новый компонент, который назовем, например, алгоритмом разрешения очередности просмотра решения (для краткости – Resolver). Причем это абстракция, за которой скрывается несколько различных алгоритмов на выбор. Удобным способом реализации будет – выделить интерфейс с несколькими доступными реализациями. При этом формально речь идет о коллекции, в которую элементы добавляются по одному как бы в один конец, а очередность выдачи как и определяется упомянутым алгоритмом.

Это означает, что одним из подходов к реализации может стать использование очереди с приоритетами, в которой процедура приписывания приоритетов является параметром, и в качестве такого параметра можно как раз передавать нужную реализацию Resolver'а.

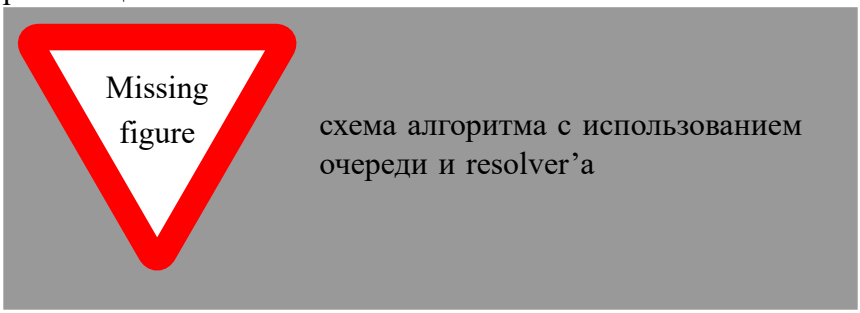

схема алгоритма с использованием очереди и resolver'а

- В эту очередь добавляются все доступные для посещения дуги.
- На каждом шаге из очереди извлекается дуга с наивысшим приоритетом.
- Функция вычисления приоритета определяет стратегию обхода:
  - для **BFS** необходимо использовать принцип FIFO — первым пришёл, первым вышёл;

- для **DFS** порядок извлечения обратный порядку попадания – LIFO, последним пришёл, первым вышёл;
- для **алгоритма Дейкстры** приоритет определяется минимальным весом пути до вершины.

Такой подход позволяет создать универсальный алгоритм обхода, параметризуемый стратегией выбора следующего шага, что является мощным и элегантным решением, допускающим, к тому же, потенциально дальнейшее расширение за счет использования других правил приписывания приоритетов. В целом это позволяет сделать итоговое решение более модульным и при расширении или модификации изменения максимально локальные и не затрагивают другие части программы. Это проявление принципа локализации зависимостей.

Тогда обобщенный алгоритм обхода можно представить следующим образом:

1. Начать с произвольной (или заданной) стартовой вершины, пометив её как посещённую.
2. Поместить все исходящие из неё дуги в специальную коллекцию — **Resolver**.
3. Пока Resolver не пуст:
   - Извлечь из него очередную дугу согласно внутренней логике Resolver'а.
   - Если дуга ведёт в ещё не посещённую вершину, пометить эту вершину как посещённую, а все её исходящие дуги (не ведущие в уже посещённые вершины) добавить в Resolver.

Таким образом, всё многообразие алгоритмов обхода сводится к различным реализациям Resolver'а:

– **Обход в ширину:** Resolver реализован как очередь (FIFO).

– **Обход в глубину:** Resolver реализован как стек (LIFO).

– **Алгоритм Дейкстры:** Resolver реализован как очередь с приоритетами, где приоритет определяется длиной пути (до очередной выбираемой вершины).

Интерфейс Resolver'а напоминает конепцию **Итератор**. Напомним, итератор — это объект, который позволяет последовательно обходить элементы коллекции, не раскрывая её внутреннего устройства.

Стандартный итератор инкапсулирует логику перехода к следующему элементу. В случае с графом можно создать специализированный итератор, который будет реализовывать ту или иную стратегию обхода (в ширину, в глубину), инкапсулируя внутри себя не только логику выбора, но и состояние обхода (например, коллекцию посещённых вершин). Примерное определение интерфейса Resolver'а приведено на листинге 13.3.

```
template <class T>
class Resolver {
public:
    virtual void Add(T item) = 0;
    virtual void Add(T* items) = 0;
    virtual void Add(Sequence<T>* items) = 0;
    virtual bool HasNext() = 0;
    virtual T Next() = 0;
    virtual bool IsEmpty() = 0;
};
```

Listing 13.3 – Примерный интерфейс процедуры, определяющей очередность посещения дуг

Алгоритм Дейкстры является классическим представителем класса т.н. жадных алгоритмов и на его примере обсудим особенности практической реализации и применения описанной

выше схемы. Идея алгоритма состоит в последовательном расширении подграфа уже посещённых вершин, при этом очередная дуга, по которой нужно пройти, выбирается из числа всех дуг, исходящих из уже посещенных вершин и ведущих в какую-либо непосещенную вершину.

Стандартное изложение этого алгоритма выглядит примерно так:

1. Для каждой вершины графа хранится текущая минимальная длина пути от стартовой вершины (изначально 0 для самой стартовой вершины и бесконечность для всех остальных).

2. Создаётся очередь с приоритетами, в которую помещается стартовая вершина. Приоритет определяется минимальной длиной пути.

3. На каждом шаге из очереди извлекается вершина u с наименьшей текущей длиной пути. Эта вершина считается посещённой.

4. Для каждой непосещённой соседней вершины v для u выполняется операция **релаксации**: если путь в v через u короче текущего известного пути в v, то значение пути для v обновляется, и v добавляется в очередь с новым приоритетом.

5. Процесс повторяется до тех пор, пока очередь не опустеет или пока не будут найдены пути до всех целевых вершин.

Во-первых, обратим внимание, что в этой схеме требуется предварительная инициализация матрицы расстояний. Дополнительно отметим, что обычно говорят именно о матрице расстояний; но искомым является как правило именно (кратчайший) путь, а не только его длина. Восстановление пути требует дополнительных усилий. Использование именно матрицы тут

оправдано тем, что при поиске кратчайшего пути, в отсутствии каких-либо дополнительных укзаний или подсказок, приходится так или иначе посетить все вершины. Дополнительаня сложность – в выборе подходящего представления о бесконечности: не всегда областью определения является какой-либо числовой ряд или вещественная прямая (ее неотрицательная половина). Иногда веса берутся из ограниченного множества, в т.ч. и имеющего наибольший элемент (в смысле частичных порядков), а значит нужны дополнительные меры, чтобы отличать непосещеннные вершины от тех, до которых путь действительно максимально возможный.

Алгоритм работает следующим образом:

1. Изначально подграф посещённых вершин состоит из одной стартовой вершины A. Диаметр этого подграфа равен нулю.

2. На каждом шаге рассматривается исходящая окрестность всего посещённого подграфа (т.е. все дуги, исходящие из уже посещённых вершин в ещё не посещённые).

3. Из этой окрестности выбирается та дуга, которая приводит к **минимальному увеличению диаметра** посещённого подграфа. Диаметр здесь понимается как длина самого длинного кратчайшего пути от стартовой вершины A до любой другой вершины в подграфе.

4. Новая вершина и дуга, ведущая к ней, добавляются в посещённый подграф.

5. Процесс повторяется до тех пор, пока не будет достигнута целевая вершина B, или пока не будут посещены все достижимые вершины.

Важным условием корректной работы алгоритма Дейкстры является **неотрицательность весов** всех дуг.

В таких алгоритмах использование объектной модели, в которой в узлах содержится исходящая окрестность этого узла (а точнее, список исходящих дуг) – оказывается очень удобным. Среди прочего, это позволяет избежать вообще решения вопроса о том, как помечать посещенные вершины: для этого можно использовать дерево путей, описанное ниже. Все посещенные вершины будут в этом дереве располагаться выше текущего узла, так что достаточно просмотреть лишь некоторую часть узлов. Кроме того, следует предусмотреть, чтобы в очередь, стек или очердь с приоритетами не помещались дубликаты. Впрочем, приходится делать выбор, что экономить: количество выполняемых операций или объем используемой памяти. Для наибольшего выигрыша по количеству операций, а следовательно, и по времени, приходит хранить в каждом узле дерева решений копию множества посещенных узлов (лучше, разумеется, указателей на них, в широком смысле этого слова).

Для восстановления найденных кратчайших путей и эффективной реализации самого алгоритма неудобно использовать матрицы. Более эффективной структурой является **дерево кратчайших путей**.

- **Структура узла дерева:** Каждый узел в этом дереве хранит ссылку на родительский узел, дугу, по которой был осуществлён переход в этот узел, и суммарную длину пути от стартовой вершины.

- **Построение:** Дерево строится динамически в процессе работы алгоритма. Корнем является стартовая вершина. Когда алгоритм выбирает очередную дугу для расширения подграфа, в дерево добавляется новый дочерний узел.

- **Преимущества:**

  - **Экономия памяти:** Общие начальные участки разных маршрутов хранятся в дереве только один раз, что позволяет избежать дублирования.

– **Поиск нескольких путей:** Данная структура позволяет легко находить не только один, но и k кратчайших путей, продолжая процесс построения дерева до тех пор, пока не будет найдено нужное количество маршрутов до целевой вершины.

– **Восстановление пути:** Путь легко восстанавливается движением от конечного узла в дереве вверх к корню по ссылкам на родительские элементы.

Это приводит к вопросу, который оказался последним в порядке обсуждения, но является далеко не последним в смысле важности: что является результатом работы алгоритма?

Ответ на самом деле очевидный, но у некоторых, как показывает практика, вызывает затруднения. Разумеется, результатом работа алгоритма по поиску кратчайшего пути (или k-кратчайших путей) будет *множество* или *коллекция* путей – причем *путей как объектов*. Поясним.

Во-первых, даже если ставится задача поиска одного кратчайшего пути, такового может и не найтись – точнее, может не найтись *единственного* такого. А значит, нужна какая-то процедура выбора. Будет ли этот выбор осуществлять человек или алгоритм (программа) – это точно не должен делать тот же алгоритм, который ищет эти кратчайшие пути: это не его ответственность. Это отдельная задача, потенциально имеющая несколько (алгоритмов) решений(я), следовательно, это отдельная ответственность.

Во-вторых, путь в графе – это тоже полноценный объект со своей внешней спецификацией и определенными на множестве таких объектов операциями.

13.3 Динамическое программирование. Мемоизация частичных результатов

Динамическое программирование (ДП) — это обобщённая методика эффективного решения дискретных оптимизацион-

ных задач, многие из которых можно свести к поиску оптимального пути в графе состояний.

Основной принцип ДП — **принцип оптимальности Беллмана**: оптимальный путь обладает тем свойством, что какова бы ни была последняя вершина на этом пути, предшествующий ей участок также является оптимальным путём до этой вершины.

Это позволяет разбить сложную задачу на более простые подзадачи и избежать повторных вычислений, сохраняя уже найденные оптимальные решения для подзадач (мемоизация). При этом вершины графа представляют состояния некоторой системы, причем эти состояния имеют комбинационный характер – являются дискретными и могут быть получены сочетанием дискретного множества элементов (возможно, разнотипных). Целевая характеристика при этом не обязательно монотонная (см. обсуждение в 12.2).

Существует два основных подхода к ДП:

- **Нисходящий процесс (сверху-вниз):** Решение основной задачи разбивается на подзадачи, те, в свою очередь, на ещё более мелкие, и так далее, пока не будут достигнуты тривиальные случаи. Этот подход естественно реализуется с помощью рекурсии. Это соответствует **обходу графа в глубину**.

- **Восходящий процесс (снизу-вверх):** Сначала решаются все самые мелкие подзадачи, затем на их основе решаются подзадачи более крупного уровня, и так далее, пока не будет решена исходная задача. Этот подход обычно реализуется итеративно. Это соответствует **обходу графа в ширину**.

На практике часто оказывается, что «обратный» обход (от конечной вершины к начальной) с использованием входящих окрестностей может быть эффективнее прямого, так как

позволяет быстрее отсекать бесперспективные ветви графа. За подробностями отсылаем к [6].

Алгоритм Флойда-Уоршелла — это классический алгоритм, основанный на методе динамического программирования, который находит длины кратчайших путей между всеми парами вершин во взвешенном ориентированном графе. Его преимуществом является способность корректно работать с рёбрами отрицательного веса (при условии отсутствия циклов с отрицательным суммарным весом).

Принцип Работы. Алгоритм работает итеративно. На k-й итерации он находит кратчайшие пути между всеми парами вершин (i, j), которым разрешено использовать в качестве промежуточных только вершины из множества $\{1, 2, \ldots, k\}$.

Для этого используются две матрицы:

- D: матрица кратчайших расстояний. Изначально $D[i][j]$ содержит вес ребра от i до j, если оно существует, и бесконечность в противном случае.

- S: матрица предшественников, используемая для восстановления путей.

Основное рекуррентное соотношение, применяемое на каждой итерации для каждой пары (i, j):

$$D_k[i][j] = \min(D_{k-1}[i][j], D_{k-1}[i][k] + D_{k-1}[k][j])$$

Это означает, что кратчайший путь от i до j с использованием промежуточных вершин до k — это либо уже известный кратчайший путь (без использования k), либо путь, проходящий через вершину k.

Восстановление Кратчайшего Пути. После того как матрица расстояний D полностью вычислена, кратчайший путь от u до v можно восстановить с помощью матрицы предшественников S. Если $S[u][v] = k$, это означает, что кратчайший путь от u

до v проходит через вершину k. Процедура восстановления рекурсивно вызывается для участков (u, k) и (k, v), пока не будут найдены все промежуточные вершины.

13.4 Оснащение метаданными

Отправной точкой для нашего обсуждения послужит задача о раскраске графа.

Задача раскраски графа заключается в том, чтобы присвоить каждой вершине цвет таким образом, чтобы никакие две смежные вершины не имели одинакового цвета, используя при этом минимально возможное количество цветов.

Поскольку задача о нахождении минимальной раскраски является NP-трудной, на практике часто используют эвристические (приближённые) алгоритмы. Один из таких подходов – алгоритм Ершова:

1. Выбрать вершину с наибольшей степенью и окрасить её в первый цвет.

2. Из 2-окрестности какой-либо окрашенной вершины выбрать неокрашенную; .окрасить ее в тот же цвет, если только она не инцидентна какой-либо вершине, окрашенной в этот же цвет. На втором шаге это часто так (хотя может быть и «танзитивное» ребро из первой вершины). В противном случае ничего не делает с этой вершиной (пока).

3. Повторяем процедуру, пока есть нераскрашенные вершины в 2-окрестности «укрупненной» вершины.

4. Если остались неокрашенные вершины, выбрать одну такую и выбрать новый цвет; разумно выбрать в первую очередь вершину с наибольшей степенью. Окрасить выбранную вершину в выбранный цвет и повторить предыдущие шаги. Всякий раз среди вершин из 2-окрестности

от новой укрупненной выбираем те, которые можно окрасить в тот же цвет.

5. Повторять, пока все вершины не будут окрашены.

Этот алгоритм не гарантирует оптимального результата, но является относительно простым в реализации и часто даёт приемлемое приближение.

Один из существенных аспектов, с точки реализации – это вопрос, как отметить цвет вершины.

Топологическое упорядочение — это линейное размещение вершин направленного ациклического графа (DAG) в виде последовательности, при котором для любой дуги, ведущей из вершины u в вершину v, вершина u в этой последовательности идёт раньше, чем вершина v.

Иными словами, это способ ”вытянуть” граф в линию так, чтобы все дуги были направлены слева направо. Такое упорядочение, как правило, не единственно, поскольку несравнимые между собой вершины могут располагаться в произвольном взаимном порядке. Процедуру можно сделать детерминированной, если зафиксировать правила выбора начальных вершин и порядок обхода исходящих дуг.

Алгоритм Кана (на основе входящих степеней). Один из классических алгоритмов топологической сортировки основан на последовательном исключении вершин.

1. Для всех вершин графа вычислить их входящую степень (количество входящих дуг).

2. Создать очередь и добавить в неё все вершины с нулевой входящей степенью. Это минимальные элементы графа.

3. Пока очередь не пуста:

   – Извлечь вершину u из очереди и добавить её в итоговый отсортированный список.

- Для каждой смежной с u вершины v:
  - Уменьшить её входящую степень на единицу (что эквивалентно мысленному удалению дуги $u \to v$).
  - Если входящая степень v стала равна нулю, добавить v в очередь.

Если после завершения алгоритма итоговый список содержит не все вершины исходного графа, это означает, что в графе присутствует цикл. Алгоритмическая сложность этого подхода составляет $O(V + E)$, где V — число вершин, а E — число рёбер.

Алгоритм на основе обхода в глубину (DFS). Альтернативный подход использует обход в глубину:

1. Запустить обход в глубину (DFS) из произвольной, ещё не посещённой вершины.

2. В процессе обхода отслеживать состояние каждой вершины: не посещена, в процессе посещения, полностью посещена.

3. Когда обход из вершины u полностью завершён (т.е. все достижимые из неё вершины посещены), добавить u в начало итогового списка.

4. Повторять шаг 1, пока все вершины не будут посещены.

Итоговый список, построенный таким образом, будет представлять собой обратный топологический порядок. Его достаточно инвертировать, чтобы получить результат. Этот метод также позволяет обнаруживать циклы: если в процессе обхода встречается вершина, уже находящаяся в состоянии ”в процессе посещения”, значит, в графе есть цикл. Сложность этого алгоритма также составляет $O(V + E)$.

13.5 Продвинутые идеи при обходе графов. Алгоритмы A* и лойда-Уоршелла

Алгоритм A* — это обобщение алгоритма Дейкстры, которое использует **эвристическую функцию** для ускорения поиска. Эвристика даёт оценку оставшегося расстояния до цели и помогает алгоритму выбирать более перспективные направления для исследования.

Например, при поиске пути на географической карте в качестве эвристики можно использовать прямое евклидово расстояние до цели. Алгоритм будет приоритизировать те пути, которые не только имеют малую уже пройденную длину, но и ведут в направлении цели.

Важно понимать, что эвристика не всегда приводит к улучшению. В графах со специфической топологией (например, с препятствиями) она может направить поиск по ложному, неоптимальному пути. Однако статистически, на "средних" графах, A* даёт значительный прирост производительности.

Алгоритм Флойда-Уоршелла – это классический алгоритм, основанный на методе динамического программирования, который находит длины кратчайших путей между всеми парами вершин во взвешенном ориентированном графе. Его преимуществом является способность корректно работать с рёбрами отрицательного веса (при условии отсутствия циклов с отрицательным суммарным весом).

Алгоритм работает итеративно. На k-й итерации он находит кратчайшие пути между всеми парами вершин (i, j), которым разрешено использовать в качестве промежуточных только вершины из множества $\{1, 2, \ldots, k\}$.

Для этого используются две матрицы:

- D: матрица кратчайших расстояний. Изначально $D[i][j]$ содержит вес ребра от i до j, если оно существует, и бесконечность в противном случае.

- S: матрица предшественников, используемая для восста-

новления путей.

Основное рекуррентное соотношение, применяемое на каждой итерации для каждой пары (i, j):

$$D_k[i][j] = \min(D_{k-1}[i][j], D_{k-1}[i][k] + D_{k-1}[k][j])$$

Это означает, что кратчайший путь от i до j с использованием промежуточных вершин до k — это либо уже известный кратчайший путь (без использования k), либо путь, проходящий через вершину k.

После того как матрица расстояний D полностью вычислена, кратчайший путь от u до v можно восстановить с помощью матрицы предшественников S. Если $S[u][v] = k$, это означает, что кратчайший путь от u до v проходит через вершину k. Процедура восстановления рекурсивно вызывается для участков (u, k) и (k, v), пока не будут найдены все промежуточные вершины.

13.6 Обобщение объектной модели

В большинстве проектов, в которых требуется решать какие-либо графовые задачи, чаще всего применяются решения, специфичные для этого проекта, – примерно так, как это описано в разделе 13.1: реализуются структуры данных с конкретным набором атрибутов, которыми в данном приложении характеризуются вершины и дуги.

Однако сами алгоритмы, в целом, являются достаточно общими, так же как и некоторой повторяемостью, из проекта в проект, являются и другие элементы, в т.ч. и общая логика организации структур данных. Повысить продуктивность разработки за счет повторного использования ранее полученных решений – важный фактор в пользу выбора более общих решений.

С другой стороны, большие проекты разрабатываются обычно на протяжении продолжительного отрезка времени, за

который по разным причинам могут измениться требования к разрабатываемой системе – не обязательно значительно, но все же достаточно, чтобы корректировать структуры данных и некоторые детали алгоритмов (например, функции оценки). И если предсказать, какими конкретно будут изменения, сложно, то вот то, что такие изменения в принципе возникут, следует считать ожидаемым и скорее нормой в практике современной разработки, чем исключением. Происходить это может в силу множества разных причин, в т.ч. объективного и производственного характера, но это предмет отдельного обсуждения, выходящего за рамки текущего круга вопросов.

Все эти соображения приводят к вопросу о том, как можно было бы обобщить объектную модель графа (и соответственно с этим – алгоритмы, их использующие). Мы рассмотрим два основных приема, один из которых достаточно универсален, чтобы его можно было непосредственно реализовать на почти любом объектно-ориентированном языке, а другой использует специальную возможность C++ – концепты. В то же время, второй подход частично использует идеи из первого, являясь в некотором роде его «продвинутой версией».

13.6.1 Декомпозиция и абстрагирование

Наиболее очевидным способом сделать реализацию более общей – это абстрагироваться от типов используемых объектов данных. А чтобы формализовать, что эти объекты должны удовлетворять определенным требованиям, сводящимся к тому, что у них должны быть определенные атрибуты/методы, можно применить подтипизацию, что в контексте большинства объектно-ориентированных языков означает потребовать, чтобы типы объекты были подтипами (в данном случае, наследниками) определеных интерфейсов.

Выше мы установили, что узлы необходимо каким-то образом идентифицировать, а значит, у них должен быть уникальный идентификатор, тип которого заранее фиксировать проблематично, поскольку в разных приложениях могут

использоваться различные типы: числа различных категорий, строки, GUID'ы[4], иногда встречаются и другие варианты.

Во-вторых, у узлов обычно есть входящая или/и исходящая окрестности (в теории графов обычно обозначаемые Γ^+ и Γ^-, соответственно), но чаще нужна лишь одна из них, и в разных задачах – разная: где-то исходящая (например, для обычных алгоритмов обхода в ширину или глубину), а где-то – входящая (например, для динамического программирования). А еще, в некоторых случаях не требуется хранить окрестность узла в самом узле, иногда ее удобнее вычислять специальные структуры типа словарей.

Поэтому целесообразно выделить три отдельные ответственности: (1) нечто, обладающие идентификатором, (2) нечто, обладающие входящей окрестность, и (3) нечто, обладающее исходящей окрестностью, – и представить эти три концепции каждую своим интерфейсом.

```
template <typename TIdentifier>
// не забываем, что тип идентификатора -- тоже
   параметр
class INode
{
public:
    virtual TIdentifier GetId() const = 0;
};

template <typename TIdentifier>
class INodeInboundVicinity // входящая окрестность
{
public:
```

[4]GUID – Globally Unique Identifier (глобально уникальный идентификатор), иногда называемый UUID (Universally Unique Identifier, т.е. универсально уникальный идентификатор). Логически, это 128-битное беззнаковое целое, обычно записывамое в 16-чной системе в формате «{xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx}», $x \in \{0, \ldots, 9, A, \ldots, F\}$. Смысл использования таких идентификаторов состоит в том, что в рамках отдельно взятой системы при минимальных ограничениях можно гарантировать уникальность каждого такого генерируемого идентификатора.

```
    virtual ReadonlySequence<ILink<TIdentifier>>*
   GetInboundEdges() const = 0;
};

template <typename TIdentifier>
class INodeOutboundVicinity  // исходящая окрестность
{
public:
    virtual ReadonlySequence<ILink<TIdentifier>>*
   GetOutboundEdges() const = 0;
};
```

Обратите внимание, что окрестности не наследуются от `INode`, поскольку это не обязательно. Окрестности задаются перечислением дуг (входящих, исходящих) или ребер, обозначенных в приведенных фрагментах кода интерфейсом `ILink`, с помощью которого мы попытаемся обобщить понятие дуг и ребра. На самом деле, выполнять такое обобщение не слишком необходимо, поскольку в рамках того или иного класса информационных или в целом программных систем используются обычно либо только ориентированные, либо только неориентированные графы, и потребности (унифицированно) работать и с теми, и с другими обычно не возникает. Тем не менее, попробуем такое обощение проделать, поскольку это само по себе поучительное упражнение.

Отправной точкой для нашего решения послужит наблюдение, что по сути дуга – это упорядоченная пара, а ребро – тоже пара, но неупорядоченная, а общим для них является наличие в точности двух элементов в составе совокупности. К сожалению, лаконичного способа гарантировать, что вершин будет ровно 2, найти не удается; зато можно получить обобщение, пригодное для использования в гиперграфах[5]

```
template <typename TIdentifier>
```

[5]Гиперграф – конструкция, допускающая, что соединяться могут не 2, а произвольные подмножества вершин. Впрочем, гиперграфы обычно можно свести к (псевдо-)графам.

```
class ILink
{
public:
    virtual bool Constains(TIdentifier nodeId) const;
};
```

Если же все-таки хочется иметь гарантию того, что элементов будет строго 2, то придется ввести дополнительную абстракцию – 2-множество; его реализация в целом похожа на реализацию обычного множества, с поправкой на то, что конструкторы будут требовать передать ровно 2 элемента, а теоретико-множественные операции не замкнуты на этой категории: для объединения, перечения и других операций в общем случае нельзя гарантировать, что результатом будет 2-множество. Оставим это читателю в качестве самостоятельного упражнения.

Отметим, что решение вопроса о равенстве и идентичности мы выносим за скобки. На самом же деле, вероятно, интерфейсы `ILink` и `INode` должны содержать объявления (скорее всего, абстрактного, или, по терминологии C++, чисто вирутального) оператора `operator==`. Кроме того, раз `ILink` – хоть и 2-элементное, но все же множество, а значит, коллекция, вероятно, этот тип должен быть унаследован от `IEnumerable`. Однако в данном изложении, в целях компактности, позволим себе опустить эти и некоторые другие детали, подразумевая, что на практике им должно быть уделено достаточное внимание.

С логической точки зрения дуга и ребро – частные случаи связи `ILink`, т.е. являются подмножествами или подтипами. А значит, для них нужно зафиксировать собственные подклассы.

```
template <typename TIdentifier>
class IArc : public ILink<TIdentifier>
{
public:
    INode<TIdentifier>* GetStart() const = 0;
    INode<TIdentifier>* GetEnd() const = 0;
protected:
```

```
    using ILink::Constains;
};

template <typename TIdentifier>
class IEdge : public ILink<TIdentifier>
{
};
```

Обратите внимание, что в нашем случае интерфейс `IEdge` указался пустым – в том смысле, что не добавляет никаких новых методов помимо унаследованных от родителя `ILink`. На практике, скорее всего, какие-то новые методы в этом интерфейсе будут, например, переопределенный оператор `operator==`; однако мы намеренно в приведенном выше фрагменте кода оставили этот интерфейс пустым.

Этим мы хотим подчеркнуть: даже если ничего нового, в дополнение к тому, что наследуется от `ILink` интерфейс `IEdge` не добавляет, он все равно должен быть определен и структуры, реализующие ребра графа, должны быть унаследованы именно от него, а не от `ILink`. Это обусловлено логикой: ребра являются подмножеством связей, даже если мы рассматриваем только связи с двумя окончаниями. А наследование должно отражать именно подтипизацию, а никак не лень разработчика и желание повторно использовать уже написанный код. В тех случаях, когда повторно использовать ранее реализованный функционал хочется, а подтипизация неприменима, можно использовать инкапсуляцию с делегированием функционала.

Еще один аспект, который пока остался без надлежащего освещения, но при том чрезвычайно важен – это вопрос о том, как представить атрибуты узлов и связей. Решений, по крайней мере, простых, тут можно предложить всего два.

Первый вариант заключается в том, чтобы, следуя базовым определениям из теории графов, отметить, что узлы и связи в графах могут быть помеченными или нет; для этого просто введем интерфейс `IMarked`, который могут поддерживать (или не поддерживать) как узлы, так и связи.

```
template <typename TMark>
class IMarked
{
public:
    TMark GetMark() const = 0;
};
```

Предполагается, что в качестве параметра `TMark` будет передаваться структура, содержащая необходимый набор атрибутов. Преимущество такого подхода в том, что благодаря статической типизации удается гарантировать, что, например, функции, вычисляющие оценку связей или путей в графе, будут использовать именно тот набор атрибутов, которые ожидается. Недостаток же подхода – в дополнительной декомпозиции объектов (что создает дополнительные накладные расходы с точки зрения создания и уничтожения этих объектов), а также менее удобной работе с объектами с точки зрения обращения к атрибутам, а также при чтении данных из внешних источников.

13.6.2 Использование подтипизации без наследования

Особенность предложенного в предыдущем подразделе подхода состоит в необходимости наследования фактически используемых структур данных от определенных интерфейсов. Но иногда это бывает нежелательным или проблематичным, и C++ предлагает способ обойти это требование, а заодно и демонстрирует наиболее продвинутые средства задания ограничений на т*и*повых параметрах (точнее – ограничения на параметры, от которых зависят типы, а это могут быть как т*и*повые переменные, так и термы).

Идея этого подхода достаточно проста: описанные ранее интерфейсы заменяются на аналогичные концепты.

```
template <  typename TNode,
            typename TIdentifier>
concept CNode = requires(TNode node)
{
```

```
    { node.GetId() } -> std::convertible_to<
   TIdentifier>;
};

template <  typename TNode,
            typename TEdge,
            typename TIdentifier>
concept COutboundNodeVicinity =
    CEdge<TNode, TEdge, TIdentifier>
        && CNode<TNode, TIdentifier>
        && requires(TNode node) {
    { node.GetOutboundEdges()} ->
        std::convertible_to<ReadonlySequence<TEdge
   >*>;
};
template <  typename TNode,
            typename TEdge,
            typename TIdentifier>
concept CInboundNodeVicinity =
    CEdge<TNode, TEdge, TIdentifier>
        && CNode<TNode, TIdentifier>
        && requires(TNode node) {
    { node.GetInboundEdges()} ->
        std::convertible_to<ReadonlySequence<TEdge
   >*>;
};
```

13.7 Материализация элементов графа по требованию

Ленивое раскрытыие окрестности вершины графа в некоторых задачах, например, при оценке решений

Список литературы

1. *Страуструп Б.* Язык программирования C++. Специальное издание. — Москва : Издательство Бином, 2002. — 1136 с.
2. *Элджер Д.* C++. Библиотека программиста. — Санкт-Петербург : Питер, 1998. — 320 с.
3. *Лавров И.*, *Максимова Л.* Задачи по теории множеств, математической логике и теории алгоритмов. — Изд. 5-е, испр. — Москва : ФИЗМАТЛИТ, 2011. — 256 с.
4. *Бурков В.*, *Новиков Д.* Теория активных систем: состояние и перспективы. — Москва : Синтег, 1999. — 128 с.
5. Теория выбора и принятия решений: учебное пособие / И. Макаров [и др.]. — Москва : Наука, 1982. — 328 с.
6. Алгоритмы. Построение и анализ / Т. Кормен [и др.]. — 3е издание, пер. с англ. — Москва : ООО «И.Д. Вильямс», 2013. — 1328 с.